

FRI

UNIVERZA V LJUBLJANI | Fakulteta za računalništvo in informatiko

Vaje pri predmetu Osnove programiranja

Ana Halužan Vasle

Verzija 2.0

Študijsko leto 2026/2027

Izjava o avtorstvu in uporabi velikih jezikovnih modelov

Pričujoča skripta je v pretežni meri avtorsko delo Ane Halužan Vasle, nastalo v okviru izvajanja vaj pri predmetu Osnove programiranja na Fakulteti za kemijo in kemijsko tehnologije Univerze v Ljubljani.

Izjema so naloge za vaje v poglavjih 1.5, 2.4, 3.4, 4.6, 5.7, 6.3, 7.6, 8.4, 9.6, 10.5, Naloge za ponavaljanje - Maraton, pripadajoče rešitve in testne skripte, ki so jih pripravili nekdanji izvajalci tega predmeta. Omenjene naloge in njihove testne skripte so v skripti deloma prirejene, preoblikovane ali dopolnjene, nekatere pa so ohranjene v izvirni obliki z manjšimi spremembami.

Pri pripravi Dodatnih vsebin sem si delno pomagala z orodji umetne inteligence (veliki jezikovni modeli), pri čemer sem vse predloge vsebinsko preverila in jih ustrezno priredila. Preostala vsebina skripte, vključno z jedrnim delom vaj, je nastala brez uporabe tovrstnih orodij.

Pri pripravi skripte so bili v poglavjih Uvod in navodila za vaje ter Kako se (na)učiti programiranja uporabljeni veliki jezikovni modeli (Claude in ChatGPT) za jezikovno urejanje, izboljšanje izražanja in kot pomoč pri urejanju oblikovanja v LaTeX-u. V preostalem delu skripte so bili veliki jezikovni modeli uporabljeni kot pomoč pri urejanju oblikovanja v LaTeX-u.

Vsebina skripte je nastala brez uporabe tovrstnih orodij - jezikovni modeli niso bili uporabljeni za generiranje vsebinskih delov skripte, temveč izključno kot podporno orodje pri zgoraj navedenih nalogah v omenjenih poglavjih.

Vse predloge, pridobljene z uporabo jezikovnih modelov, je avtorica kritično ovrednotila, preverila in ustrezno prilagodila.

Kazalo

Uvod in navodila za vaje **9**
Skripta . . . 9
Repozitorij OP_FKKT na GitHub-u . . . 9
Uporaba avtomatskih testov . . . 9
Kako se (na)učiti programiranja? . . . 10

1. vaje: Python in IDLE **11**
1.1 Namestitev Pythona . . . 11
1.2 IDLE . . . 11
1.3 Uporaba testnih skript . . . 12
1.4 Podatkovni tipi, operacije in pogoste funkcije . . . 14
1.5 Naloge . . . 17
1.6 Rešitve . . . 19

2. vaje: Pogojni stavek *if* **20**
2.1 Operatorji za opis pogojev . . . 20
2.2 Načini uporabe pogojnih stavkov `if` . . . 21
2.3 Zgledi . . . 23
2.4 Naloge . . . 26
2.5 Rešitve . . . 30

3. vaje: Zanka *while* **32**
3.1 Možnosti izvedbe zanke *while* . . . 32
3.2 Uporaba *break* in *continue* . . . 33
3.3 Zgled: Uporaba zanke *while* na primeru titracije . . . 34
3.4 Naloge . . . 38
3.5 Rešitve nalog . . . 42

4. vaje: Seznami in zanka *for* **45**
4.1 Seznami . . . 45
4.2 Zanka *for* . . . 47
4.3 Gnezdenje zank *for* . . . 48
4.4 *Hitra izpeljava seznama (angl. *list comprehension*) . . . 49
4.5 Zgledi z nalogami za utrjevanje . . . 50
4.6 Naloge . . . 54
4.7 Rešitve nalog . . . 57

5. vaje: Funkcije **59**
5.1 `return` vs. `print()` . . . 59
5.2 Položaj `return` v funkciji . . . 60

5.3 Opcijski argument . . . 61
5.4 Uporaba `pass` . . . 61
5.5 Zgledi . . . 62
5.6 Naloge za utrjevanje . . . 64
5.7 Naloge . . . 65
5.8 Rešitve . . . 69

6. vaje: (Ne)spremenljivi podatkovni tipi in terke **72**
6.1 Spremenljivost podatkovnih tipov . . . 72
6.2 Terke . . . 74
6.3 Naloge . . . 77
6.4 Rešitve . . . 80

7. vaje: Slovarji **83**
7.1 Metode `.keys()`, `.values()` in `.items()` . . . 84
7.2 Seznami in množice kot vrednosti v slovarjih . . . 85
7.3 Slovar slovarjev . . . 86
7.4 *Hitra izpeljava slovarja (angl. *dictionary comprehension*) . . . 88
7.5 Dodatne naloge . . . 90
7.6 Naloge . . . 91
7.7 Rešitve nalog . . . 95

8. vaje: Množice in metode **98**
8.1 Množice . . . 98
8.2 Metode . . . 100
8.3 Naloge za utrjevanje . . . 102
8.4 Naloge . . . 103
8.5 Rešitve nalog . . . 107

9. vaje: Datoteke **109**
9.1 Odpiranje datotek . . . 109
9.2 Branje podatkov . . . 109
9.3 Pisanje v datoteko . . . 111
9.4 Zapiranje datoteke . . . 111
9.5 Zgledi . . . 112
9.6 Naloge . . . 116
9.7 Rešitve nalog . . . 120

10. vaje: Knjižnice **124**
10.1 Nameščanje knjižnic . . . 124
10.2 Uporaba knjižnic . . . 124
10.3 Uporaba lastnih modulov . . . 125
10.4 Utrjevanje . . . 126

10.5 Naloge 127
10.6 Rešitve 136

Razhroščevanje **141**
Prva pomoč pri nedelujočem programu 141
Pythonova sporočila o napakah 141
Razhroščevanje s testi 142
Razhroščevanje s funkcijo `print()` 142

Naloge za ponavljanje **150**
Naloge 150
Rešitve nalog 157

Uvod in navodila za vaje

Vaje pri predmetu Osnove programiranja so zasnovane tako, da naloge rešujete sami, v svojem tempu, saj je samostojno reševanje edini način, s katerim se dejansko naučite programirati. Če se vam pri kakšni nalogi zatakne, ne napredujete ali ne razumete snovi, vprašajte[1]. Bistvo vaj je, da se pri razlagi nalog/odpravljanju težav prilagodimo vašemu načinu razumevanja in razmišljanja, saj vsak programer razmišlja na svoj način.

Skripta

Skripta vsebuje deset sklopov vaj, ki sledijo snovi na predavanjih in se stopnjujejo po zahtevnosti. Poglavjem z vajami sledi še poglavje *Razhroščevanje*, kjer je razloženo, kako brati Pythonova sporočila o napakah in kako si pri iskanju napak pomagati s testi.

Snov, ki pride v poštev za pisni izpit, obsega sklope vaj 1–9[2], skupaj s poglavjem *Razhroščevanje*.

Naslovi, označeni z zvezdico (`*`), vsebujejo dodatne/zahtevnejše tematike, ki niso predmet izpita.

Repozitorij OP_FKKT na GitHub-u

Vsa gradiva za predmet so zbrana v repozitoriju `OP_FKKT`[3] na GitHubu. Opis vsebine repozitorija in navodila za delo z njim so objavljena v datoteki `README.md` v repozitorju.

Spletna učilnica se uporablja predvsem za obvestila in oddajo nalog, medtem ko so vaje in dodatna gradiva ažurneje vzdrževana na GitHub-u. V primeru razlik med gradivi se držite gradiv iz repozitorija GitHub.

Uporaba avtomatskih testov

Za vsak sklop vaj se na repozitoriju v mapi `naloge_testi` nahaja podmapa z avtomatskimi testi, ki preverjajo pravilnost vaše rešitve.

Avtomatsko testiranje svoje kode izvedete tako, da:

(1) v ustrezno podmapo shranite svojo rešitev, poimenovano natanko tako, kot je določeno v navodilih naloge[4],
(2) v isti podmapi odprete in poženete datoteko `test.py`.

[1] Gotovo ste že neštetokrat slišali stavek: "Neumnih vprašanj ni." Ko boste v dilemi, ali vprašati ali ne, se spomnite nanj ;)
[2] Do in vključno s poglavjem o datotekah.
[3] Dostopen na povezavi: https://github.com/ahaluzan/OP_FKKT
[4] Če poimevanje datoteke ni pravilno, je test ne bo prepoznal.

Iz ukazne vrstice (macOS/Linux: Terminal) to storite tako, da delovni direktorij prestavite v mapo s testom, nato pa v terminal vpišete ukaz: `python test.py`. V operacijskih sistemih macOS in Linux je ukaz: `python3 test.py`.

Kako se (na)učiti programiranja?

Programiranje je veščina, ki zahteva veliko treninga. Zgolj razumevanje neke rešitve še ne pomeni, da jo boste znali sami tudi sprogramirati – zato je nujno, da čim več vadite in se učite na napakah. Najboljši recept je, da nalogo najprej rešite sami, nato primerjajte svojo rešitev s pravilno, razmislite, kako bi svojo kodo lahko izboljšali, in nato nalogo poskusite rešiti znova.

Zelo koristno je tudi sprotno anotiranje kode, pisanje psevdokode za sledenje poteku in sledenje izvajanja kode – bodisi neposredno v programu ali pa na list papirja. V primeru sledenja izvajanja kode si za vsako vrstico zapišite vrednosti spremenljivk po njeni izvedbi. Vsi ti koraki pomagajo pri grajenju občutka za izvajanje kode in delovanje algoritmov, kar je temelj dobrega razumevanja programiranja.

Pri pripravah na pisna preverjanja znanja priporočam reševanje starih kolokvijev in izpitov z uporabo časovnika, nastavljenega na trajanje izpita (običajno 90 minut). Veliko tipov nalog se ponavlja, zato si njihove postopke in primere kode dobro poglejte.

Zelo vam bo koristilo, če se navadite delati s testi in sporočili o napakah, ki jih Python izpiše ob zagonu kode (glejte poglavje Razhroščevanje). Priporočljivo je, da se učite tudi tako, da namenoma vnašate napake in opazujete, kako jih Python javi. Sporočila o napakah berite od spodaj navzgor – zadnja vrstica običajno pove, kaj je dejanski problem, medtem ko je preostanek sporočila pogosto le sled, kako je Python prišel do napake.

Najprej napišite "grdo", a delujočo rešitev, šele nato se lotite izboljševanja kode. Vztrajanje pri iskanju popolne rešitve od začetka pogosto pripelje do tega, da naloga ostane nedokončana.

Ločite med tem, ali imate težavo z razumevanjem postopka reševanja ali zgolj s sintakso. Če znate razložiti rešitev z besedami, a je ne znate zapisati v kodi, je to precej manjši problem, kot če ne veste, kako bi se naloge sploh lotili.

Redno, sprotno delo, četudi krajše, je veliko učinkovitejše od enkratnega dolgega učenja tik pred izpitom – številne veščine, predvsem razhroščevanje in branje kode, se gradijo postopoma s ponavljanjem.

Na izpitih uporabljajte samo izraze, funkcije in metode, za katere ste prepričani, da jih razumete in obvladate njihovo sintakso. Če druge rešitve ne vidite, neznano metodo najprej preverite na preprostem primeru izven vaše glavne kode.

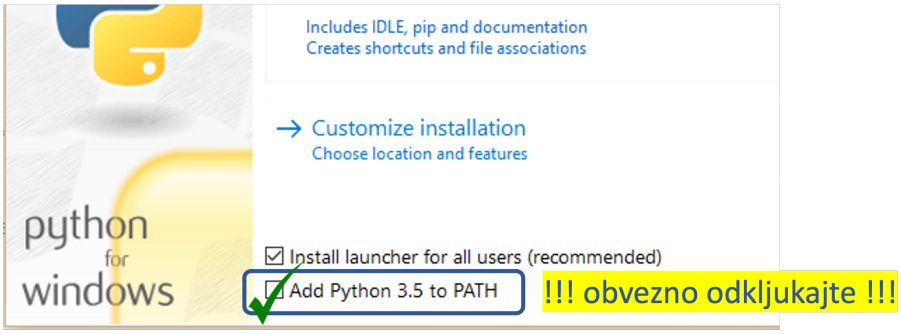
Če vam programiranje na začetku dela težave, ni nič nenavadnega – večina študentov, ki se s programiranjem srečuje prvič, se sooča s podobnimi izzivi. Če naletite na težavo ali nejasnost, vprašajte čim prej – snov se nadgrajuje in drobna težava, ki jo spregledate zdaj, lahko kasneje postane zelo velik problem.

1. vaje: Python in IDLE

Programski jezik Python je eden najpogosteje uporabljenih programskih jezikov. Zaradi berljive kode, ki je podobna psevdokodi, je zelo primeren za začetnike. Za njegovo uporabo ni potrebno podrobno poznavanje delovanja računalnika. Poleg številnih vgrajenih funkcij, podatkovnih struktur in modulov, ki omogočajo pisanje programov s kompleksnimi funkcionalnostmi, nudi Python tudi močno podporo za podatkovne vede, vizualizacijo podatkov in strojno učenje prek knjižnic, kot so Pandas, NumPy, SciPy, Seaborn, scikit-learn, Keras in PyTorch.

1.1 Namestitev Pythona

Na spletni strani https://www.python.org/downloads/ izberite verzijo programja, ki je kompatibilna z vašim operacijskim sistemom. Namestitveno datoteko zaženite. Če nameščate Python na Windows, ob nameščanju odkljukajte možnost "Add Python to PATH" (Slika 1). Uporabniki iOS in Linux te možnosti nimate, saj se v večini primerov Python avtomatsko doda v pot.

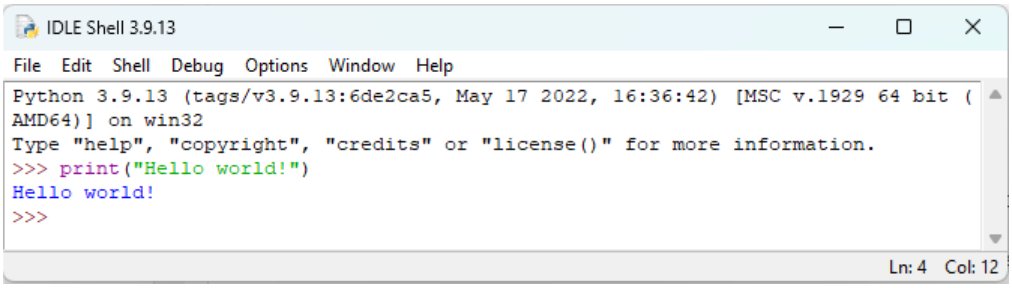


Slika 1: Pri nameščanju Pythona na računalnike z operacijskim sistemom Windows odkljukajte možnost "Add Python to PATH", ki se pojavi ob zagonu namestitvene datoteke. V zgornjem primeru se je nameščal Python verzije 3.5

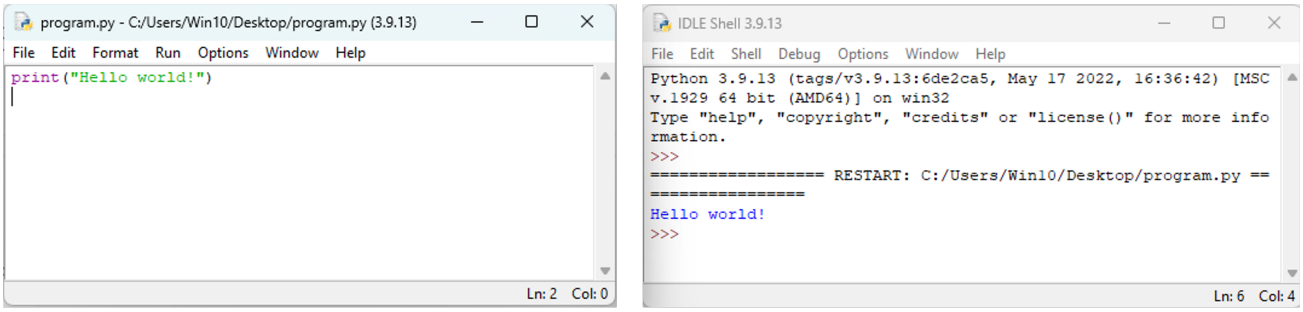
1.2 IDLE

IDLE (angl. *Integrated Development and Learning Environment*) je integrirano razvojno okolje, ki ga pridobimo skupaj z distribucijo Python. Na fakultetnih računalnikih ga zaženemo preko namizne ikone. Odpre se t.i. lupina (angl. *Shell*), ki omogoča izvajanje ukazov v jeziku Python (Slika 2).

Če želimo izvesti več ukazov naenkrat, lahko s klikom na "File" v orodni vrstici IDLE odpremo meni, kjer izberemo "New File". Vanj vpišemo kodo, ki jo poženemo s klikom na "Run Module" ali tipko F5. Ukaz "Run Module" se nahaja v spustnemu meniju pod ukazom "Run" v orodni vrstici. Kode ne bomo mogli pognati, če datoteke nismo shranili. Izhod naše kode se izpisuje v IDLE Shell (Slika 3).



Slika 2: IDLE lupina (angl. *Shell*). Ob zagonu IDLE se odpre t.i. lupina, v kateri lahko izvajamo krajše ukaze v jeziku Python.



Slika 3: Primer shranjenega programa (levo) in njegovega izhoda v lupini (desno). Daljše ukaze ali zaporedje ukazov lahko shranimo v datoteko kot je `program.py` na levi sliki. Ko datoteko zaženemo, se rezultat programa izpiše v lupini IDLE.

Običajno lahko `.py` datoteke s programom IDLE odpremo tako, da nanje kliknemo z dvoklikom. V primeru, da to ne deluje, na izbrano datoteko kliknemo z desnim klikom, poiščemo možnost "Show more options", nato "Edit with IDLE" in ponovno "Edit with IDLE".

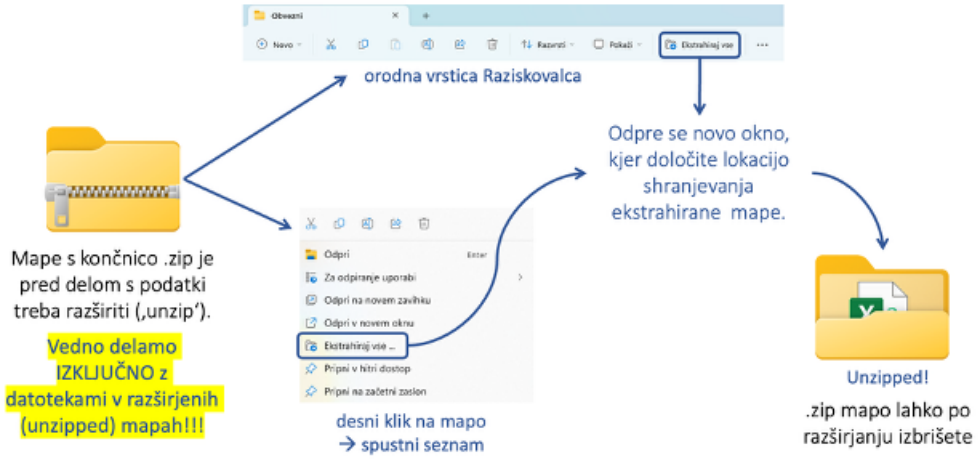
1.3 Uporaba testnih skript

S testiranjem preverjamo, ali naš program vrača pravilen rezultat. Pri ročnem testiranju poznamo vhode in pričakovane rezultate ter preverjamo, ali je pri danih vhodih izhod našega programa enak pričakovanemu rezultatu. Poleg ročnega poznamo tudi avtomatsko testiranje s t.i. enotskimi testi (angl. *unit tests*), kjer definiramo nabor vhodov s pričakovanimi izhodi in vse testiramo naenkrat. Pri obeh oblikah testiranja preverjamo, kako se naš program odziva v robnih pogojih, kot so minimalni in maksimalni vhodi, odziv na nepričakovane vnose, ipd.

Vajam, kolokvijem in izpitom so priložene testne skripte, ki vam pomagajo pri testiranju programa, lažji identifikaciji napak in pomanjkljivosti kode ter preverjanju upoštevanja navodil - vključno s poimenovanjem datoteke, ki jo želimo testirati.

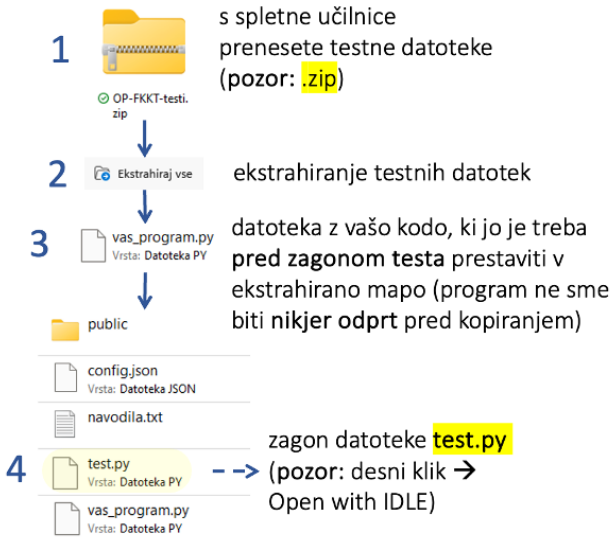
Testne skripte na računalnik prenesemo v obliki stisnjene mape, običajno poimenovane `testi`. Preden jih lahko začnemo uporabljati, moramo mapo nujno razširiti oziroma ekstrahirati njeno vsebino. V operacijskem sistemu iOS se ekstrakcija zgodi avtomatsko s klikom na mapo, v

operacijskem sistemu Windows pa moramo izvesti desni klik na mapo in izbrati možnost "Ekstrahiraj vse"/"Extract all". Odprlo se bo novo okno, ki bo spraševalo po želeni lokaciji ekstrahirane mape - najbolje je izbrati Namizje/Desktop, saj tako preprečite, da bi pomotoma delali s stisnjeno mapo, ki se nahaja v Prenosih/Downloads (Slika 4).

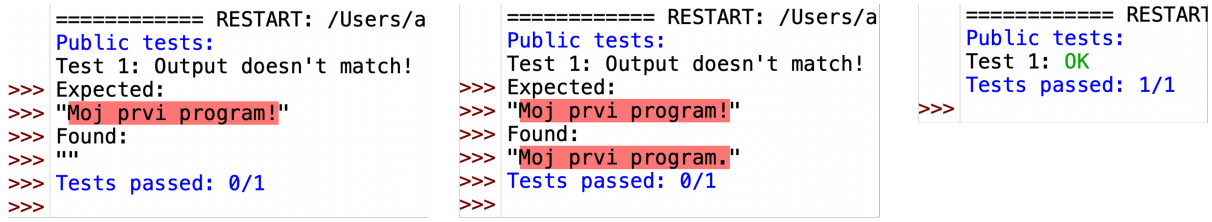


Slika 4: Potek ekstrahiranja mape v operacijskem sistemu Windows.

Če želite teste uporabiti, preverite, ali ste svojo datoteko poimenovali kot je zahtevano v navodilih. To datoteko shranite v mapo `testi`, nato pa poženite datoteko `test.py` (Slika 5). Odprla se bo lupina IDLE, v kateri boste videli, ali vaš program deluje pravilno (Slika 6). Testi so v prvem delu vaj zelo občutljivi na znake!



Slika 5: Potek zagona testne skripte, ki preverja pravilnost vaše rešitve naloge.



Slika 6: Primer testiranja programa, ki mora na zaslon izpisati `Moj prvi program!`. Prvi dve testiranji nista bili uspešni - pod "Expected" lahko vidimo, kaj naloga od nas pričakuje, pod "Found" pa je zapisano, kaj vrača naš program. V prvem primeru naš program ne vrača ničesar, v drugem pa je uporabljeno napačno ločilo.

1.4 Podatkovni tipi, operacije in pogoste funkcije

Podatkovni tipi, s katerimi bomo delali so:

- cela števila (angl. *integer*, funkcija `int()`),
- realna števila (angl. *float*, funkcija `float()`),
- nizi (angl. *string*, funkcija `str()`) in
- Boolovi operatorji (angl. *Boolean*, funkcija `bool()`).

Če nismo prepričani, katerega tipa je podatek, shranjen v neki spremenljivki, lahko to preverimo s funkcijo `type()`.

Seštevanje, odštevanje in množenje lahko uporabljamo pri operacijah s celimi števili, realnimi števili, pa tudi nizi. Potenciranje (znak `**`), deljenje (znak `/`), celoštevilsko deljenje (znak `//`) in vračanje ostanka pri deljenju (znak `%`) lahko izvajamo le nad številskimi podatkovnimi tipi.

Funkcije so oblike `funkcija(argumenti)` in izvajajo določene operacije nad argumenti. Vsaka funkcija ima določeno število argumentov, ki jih lahko sprejme, hkrati pa je tudi omejena s podatkovnimi tipi - nekatere funkcije delujejo samo, če je argument točno določenega podatkovnega tipa (npr. niz) in/ali oblike (npr. seznam). Pogosto uporabljene osnovne funkcije v Pythonu so zbrane v Tabeli 1.

Tabela 1: Pogosto uporabljene osnovne funkcije v Pythonu.

Funkcija:	Opis:
`input()`	uporabnik lahko vnese poljuben podatek
`abs(a)`	vračanje absolutne vrednosti števila, shranjenega v spremenljivki `a`
`pow(a,b)`	vrne rezultat potenciranja a^b
`min(s)`	vrne najmanjše število v seznamu `s`
`max(s)`	vrne največje število v seznamu `s`
`round(a,b)`	vrne število `a`, zaokroženo na `b` decimalnih mest
`len()`	vrne dolžino niza ali seznama
`sum(s)`	vrne vsoto števil v seznamu `s`
`range(a,b)`	vrne zaporedje celih števil med `a` in `b`, ne vključujoč `b`

Funkciji input() in print()

Funkcija `input()` kot argumente sprejme niz, ki uporabniku namigne, kakšen vnos se pričakuje. Če v kodi zapišemo:

```
>>>ime = input("Vpiši ime: ")
```

in kodo zaženemo, se v lupini pojavi naš namig, ki smo ga podali kot argument in utripajoča črtica, ki čaka na uporabnikov vnos - v tem primeru je uporabnik vpisal ime `Ivan`:

```
Vpiši ime: Ivan
>>> ime
'Ivan'
```

Takoj, ko uporabnik vpiše niz (v tem primeru ime `Ivan`), se ta shrani v spremenljivko `ime`. Ko pokličemo to spremenljivko (druga vrstica v zgornji kodi), Python vrne ime, označeno z narekovanjema, ki uporabniku sporočata, da gre za podatkovni tip niza. Funkcija `input()` vedno shrani vnos kot niz. Če je uporabnik vpisal število, se bo to prikazalo kot niz in v primeru, da želimo z njim računati, moramo spremeniti podatkovni tip (glej 4. vrstico):

```
Vpiši število: 4
>>> stevilo
'4'
>>> int(stevilo)
4
```

S pomočjo funkcije `print()` lahko vsebino spremenljivke `ime` izpišemo na zaslon brez oznak za podatkovni tip. Argumenti funkcije `print()` so lahko nizi ali spremenljivke, ki vsebujejo poljuben podatkovni tip.

Spremenljivko `ime` lahko na zaslon izpišemo na različne načine, ki so povzeti spodaj. Bodite pozorni na uporabo presledkov znotraj funkcije `print()` in izpis le-teh na zaslonu!

1. **Uporaba konkatenacije**

   ```
   >>> print("Dolžina imena " + ime + " je " + str(len(ime)) + ".")
   Dolžina imena Ivan je 4.
   ```

   Pri konkatenaciji je potrebno presledke vnesti kot del nizov, sicer jih `print()` ne bo izpisal na zaslon. Paziti moramo, da seštevamo le podatkovni tip niza, sicer funkcija ne bo delovala.

2. **Nizanje argumentov z vejico**

   ```
   >>> print("Dolžina imena", ime, "je",(len(ime)))
   Dolžina imena Ivan je 4
   ```

   Uporaba vejice v `print()` samodejno vnese presledek na mesto, kjer je ta vnesena. S poenotenjem podatkovnega tipa se za razliko od konkatenacije tukaj ne rabimo ukvarjati.

3. Uporaba f-nizov (angl. *f-strings*)

```
>>> print(f"Dolžina imena {ime} je {len(ime)}.")
Dolžina imena Ivan je 4.
```

Pri uporabi f-string imamo največjo kontrolo nad spremenljivkami. Kakor v primeru uporabe vejice, se tudi tukaj ne rabimo ukvarjati s poenotenje podatkovnih tipov spremenljivk. Paziti moramo, da vedno na začetek želenega izpisa vnesemo `f` in narekovaj, ki je lahko bodisi enojni (’), bodisi dvojni (").

Pri uporabi funkcije ***`print()`*** *se odločite za enega od zgoraj naštetih načinov in se tistega držite. Odlična vaja za razumevanje vseh treh načinov je pretvarjanje iz enega v preostala dva načina ter sprotno preverjanje enotnosti izpisov na zaslon. Zaradi enostavnosti in razširjenosti priporočam uporabo f-strings.*

1.5 Naloge

Pri reševanju pazite, da se vaši izpisi ujemajo z izpisi v navodilih (vključno s presledki).

ZIP-datoteko s testi razpakirajte v enega od imenikov, kjer imate pravice za pisanje. Vsaka naloga ima svoj podimenik, v katerem so testi. Svoj program ustrezno poimenujte in shranite v imenik poleg testne skripte `test.py`. Teste poženete tako, da poženete program `test.py` (lahko preko IDLE ali konzole).

1.5.1 Kako ti je ime?

Napišite program `ime.py`, ki uporabnika povpraša najprej po imenu in na zaslon izpiše dolžino tega imena.

Primer pravilnega delovanja programa:

```
Vpiši ime: Janez
Dolžina imena Janez je 5
```

1.5.2 Pitagorov izrek

Napišite program `pitagorov_izrek.py`, ki uporabnika vpraša po dolžinah katet pravokotnega trikotnika in izpiše dolžino hipotenuze. Program naj omogoča vnos decimalnih števil. Rezultat naj bo zaokrožen na eno decimalko.

Primer pravilnega delovanja programa:

```
Vpiši dolžino prve katete: 3
Vpiši dolžino druge katete: 4
Dolžina hipotenuze: 5.0
```

1.5.3 Ploščina pravokotnega trikotnika

Program iz prejšnje naloge spremenite tako, da izračuna in izpiše ploščino pravokotnega trikotnika. Program shranite kot `ploscina.py`. Oba rezultata v izpisu zaokrožite na dve decimalni mesti natančno.

Primer pravilnega delovanja programa:

```
Vpiši dolžino prve katete: 3
Vpiši dolžino druge katete: 4
Dolžina hipotenuze: 5.0
Ploščina trikotnika: 6.0
```

1.5.4 Molska masa

Napišite program `molska_masa.py`, ki na podlagi uporabnikovega vnosa celega števila molekul vode, izračuna molsko maso $CuSO_4(H_2O)_x$.

Pri izračunu uporabite naslednje relativne atomske mase:

```
Cu = 63.5
S = 32.1
O = 16.0
H = 1.0
```

Rezultat naj bo zaokrožen na eno decimalko.

Primer pravilnega delovanja programa:

```
Stopnja hidratacije: 5
Molska masa je 249.6 g/mol.
```

1.6 Rešitve

1.5.1 Kako ti je ime?

```
ime = input("Vpiši ime: ")
print("Dolžina imena", ime, "je", len(ime))
```

1.5.2 Pitagorov izrek

```
a = float(input("Vpiši dolžino prve katete: "))
b = float(input("Vpiši dolžino druge katete: "))

c = a ** 2 + b ** 2
c = c ** (1 / 2)

print("Dolžina hipotenuze: ", str(round(c, 1)))
```

1.5.3 Ploščina pravokotnega trikotnika

```
a = float(input("Vpiši dolžino prve katete: "))
b = float(input("Vpiši dolžino druge katete: "))

c = a ** 2 + b ** 2
c = c ** (1 / 2)
p = a * b / 2

print("Dolžina hipotenuze:", round(c,2))
print("Ploščina trikotnika:", round(p,2))
```

1.5.4 Molska masa

```
hidratacija = int(input('Stopnja hidratacije: '))

Cu = 63.5
S = 32.1
O = 16.0
H = 1.0

mm = Cu + S + 4 * O + hidratacija * (2 * H + O)
mm = round(mm,1)

print('Molska masa je', mm, 'g/mol.')
```

2. vaje: Pogojni stavek *if*

Pri pogojnem stavku `if` se srečamo z zamiki - ob `if` zapišemo pogoj in dodamo dvopičje (`:`), v naslednji vrstici, ki je zamaknjena za 4 presledke ali 1 tabulatorsko mesto, pa definiramo kaj naj se zgodi v primeru, da je pogoj ob `if` izpolnjen.

Splošna psevdokoda uporabe pogojnega stavka `if`:

```
if pogoj1:
    naredi to
elif pogoj2:
    naredi raje to
else:
    za vse ostalo naredi to
```

> *Ko se odločite za način zamika - 4 presledki ali 1 tabulatorsko mesto - se skozi celoten program držite istega principa. Mešanje presledkov in tabulatorskih mest je lahko problematično pri izvajanju kode in iskanju napak.*

Nizamo lahko poljubno število stavkov `if` oziroma `elif`, le `else` je omejen na 1 na kombinacijo.

2.1 Operatorji za opis pogojev

Pogoje v pogojnih stavkih `if` postavimo tako, da dodamo neko relacijo z izbrano spremenljivko. Te relacije so lahko primerjalne (Tabela 2) ali Boolove (Tabela 3).

Tabela 2: Primerjalni operatorji.

Operator:	Opis:
`==`	je enako
`!=`	ni enako
`<, >`	manjše/večje
`<=, >=`	manjše/večje ali enako

Tabela 3: Boolovi operatorji.

Operator:	Opis:
`or`	logični ali
`and`	logični in
`not`	logični ne
`in`	je vsebovano v
`not in`	ni vsebovano v

Pri preverjanju pogojev moramo biti pozorni na sintakso. Spremenljivka `x` ima prirejeno neko vrednost, za katero nas zanima, ali je enaka kateri od naslednjih spremenljivk: `a = 1`, `b = "a"` in `c = (1,"a")`. To bomo preverili s pogojnim stavkom *if*.

Če kodo napišemo takole:

```
if x == a or b or c:
    print("Se ujema.")
```

bo Python to razumel kot vprašanje, ali je `x` enak `a` oziroma ali je v spremenljivki `b` neničelna vrednost oziroma ali je v spremenljivki `c` neničelna vrednost, kar bi v kodi lahko zapisali kot

`if x == a or b == True or c == True`. Ker sta spremenljivkama `b` in `c` prirejeni pozitivni vrednosti, bo Python pogoja `b == True` in `c == True` vedno označil kot pravilna. Koda pod podanim pogojnim stavkom `if` se bo torej izvedla vedno, kadar vsaj ena od spremenljivk `b` ali c vsebuje neničelno vrednost.

Pravilna sintaksa pogojnega stavka *if*, ki bo preverjala, ali je `x` enak bodisi `b` ali c se glasi:

```
1 if x == a or x == b or x == c:
2     print("Se ujema.")
```

2.2 Načini uporabe pogojnih stavkov `if`

Nizamo lahko poljubno število pogojnih stavkov `if`. Če nizamo samo `if`, se bo vsak pogoj preveril neodvisno od pravilnosti ali nepravilnosti predhodnega. Če se odločimo za kombinacijo `if-elif`, se izvede le tisti del kode, ki sledi prvemu izpolnjenemu pogoju - preostali pogoji se ne preverjajo več. Koda pod `else` v kombinaciji `if-elif-else` ali `if-else` se izvede za vse primere, kjer noben od predhodnih pogojev ni bil izpolnjen.

1. Uporaba zaporednih pogojnih stavkov `if`

```
1 if x < 3:
2     print("a")
3 if x < 2:
4     print("b")
5 if x < 1:
6     print("c")
```

Če je `x = 0`, bo program izpisal sledeče:

```
a
b
c
```

Ker so nanizani 3 pogojni stavki *if*, se bodo navedeni pogoji vseh treh preverili neodvisno eden od drugega, saj imamo 3 sklope *if*. Ker so za `x = 0` vsi trije pogoji pravilni, se bodo izvedle vse vrstice pod *if*.

2. Uporaba *if-elif-else*

```
1 if x < 3:
2     print("a")
3 elif x < 2:
4     print("b")
5 else:
6     print("c")
```

V kombinaciji *if-elif-else* se vedno izvede le koda pod stavkom, ki vsebuje prvi pravilen pogoj. Če je `x = 0`, sta sicer pravilna tako pogoj pri *if* kot pri *elif*, vendar se bo na zaslonu izpisal le `a`, saj Python v tej kombinaciji izvede le koda pod prvim pravilnim pogojem, naslednjih pogojev pa ne gleda več.

3. Uporaba *if-if-else*

```
1 if x < 3:
2     print("a")
3 if x < 2:
4     print("b")
5 else:
6     print("c")
```

Če je `x = 2.5`, se bo na zaslonu izpisalo:

```
a
c
```

Kombinacijo *if-if-else* Python tolmači kot 2 neodvisna sklopa pogojnih stavkov - prvi sklop se začne v 1. vrstici, drugi sklop pa v 3. vrstici. Pogoj pri pogojnem stavku *if* je pravilen, zato se izvede koda pod tem stavkom. V drugem sklopu `x` ne izpolnjuje pogoja, navedenega pod *if*, zato se izvede *else*. Zadnji *else* je vezan na zadnji *if* (torej *if* v zadnjem sklopu, ki se začne v 3. vrstici) in ni povezan z nobenih predhodnim sklopom pogojnih stavkov *if*.

Nerazumevanje delovanja zgoraj opisanih kombinacij je zelo pogosta napaka pri programiranju, zato dobro preučite razlike med njimi.

2.3 Zgledi

2.3.1 Določanje genotipa

Pri gnezdenju stavkov `if` bodite pozorni na zamike - skupaj spadajo tisti sklopi `if`, ki so na istem nivoju zamikov.

```
1  genotip = input("Vpišite genotip: ")
2
3  if len(genotip) == 2:
4      if 'A' in genotip:
5          if "AA" == genotip:
6              print("Dominantni homozigot.")
7          else:
8              print("Heterozigot.")
9      else:
10         print("Recesivni homozigot.")
11 else:
12     print("Vnesli ste neveljavno dolžino genotipa.")
```

Prvi sklop pogojnih stavkov sestavljata `if` in `else` v 3. in 11. vrstici, drugi sklop `if` in `else` se nahaja v 4. in 9. vrstici, tretji sklop pa v 5. in 7. vrstici.

Program najprej uporabnika vpraša po genotipu, nato pa v prvem sklopu `if-else` preveri, ali je dolžina vnešenega genotipa enaka 2. Če je, se koda pomakne k drugemu sklopu `if-else`. V primeru, da je črka `"A"` prisotna v nizu, prirejenem spremenljivki `genotip`, se pomaknemo na zadnji nivo odločanja.

2.3.2 Določanje formule ogljikovodikov

```
1  acyclic_ch = input("Iščem formulo za (alkan/alken/alkin): ")
2  c_atoms = int(input("Število C atomov: "))
3
4  if acyclic_ch == 'alkan':
5      h_atoms = 2*c_atoms + 2
6  elif acyclic_ch == 'alken':
7      h_atoms = 2*c_atoms
8  elif acyclic_ch == 'alkin':
9      h_atoms = 2*c_atoms - 2
10 else:
11     h_atoms = 0
12     print('Vpisali ste neveljaven tip ogljikovodika.')
13
14 if h_atoms:
15     print(f'{acyclic_ch.capitalize()} ima formulo C{c_atoms}H{h_atoms}.')
```

V vrstici 14 Python zapis `if h_atoms:` tolmači kot `if h_atoms == True:`. Če je v spremenljivki `h_atoms` shranjena katerakoli spremenljivka, ki ni enaka 0, bo zgornji stavek veljaven. Če pa velja `h_atoms = 0`, se zadnji stavek `if` ne izvede. S takšno obliko se elegantno izognemo potrebi po večkratnemu ponavljanju stavkov `print`.

2.3.3 Izračun pH pri titraciji

Določene funkcije niso nativno na voljo v Pythonu, temveč so zbrane v modulih - paketih funkcij, razredov in spremenljivk. Pri uporabi funkcije, ki izvira iz modula, moramo modul najprej uvoziti s stavkom `import`, ki mu sledi ime modula, kot v vrstici 1.Izbrano funkcijo v kodi nato kličemo z imenom modula, ki mu dodamo piko in izpišemo ime želene funkcije (glejte vrstici 12 in 16).

```
1  import math
2
3  cb = float(input("Vnesite koncentracijo NaOH [M]: "))
4  Vb = float(input("Vnesite volumen porabljenega NaOH [L]: "))
5  ck = float(input("Vnesite koncentracijo HCl [M]: "))
6  Vk = float(input("Vnesite volumen raztopine HCl [L]: "))
7
8  nb = cb*Vb
9  nk = ck*Vk
10
11 if nb < nk:
12     pH = -math.log(((nk-nb)/(Vk+Vb)),10)
13 elif nb == nk:
14     pH = 7
15 else:
16     pH = 14-math.log(((nb-nk)/(Vk+Vb)),10)
17
18 if pH == 7:
19     print("Dosegli ste ekvivalenčno točko. Titracija je končana.")
20 elif pH < 7:
21     print("pH je " + str(round(pH, 2)) + ". Nadaljujte z dodajanjem baze.")
22 else:
23     print("pH je " + str(round(pH, 2)) + ". Raztopina je pretitrirana.")
```

2.4 Naloge

2.4.1 Črka v nizu

Napišite program `crka_v_nizu.py`, ki ugotovi, ali nek niz vsebuje podano črko (ali nek drug znak). Vaš program naj uporabnika najprej vpraša po nizu, nato še po črki. Program naj izpiše, ali je ta črka vsebovana v nizu. Da bo naloga lažja, naj program velike in male črke smatra kot različne znake.

Primer delovanja programa:

```
Vpišite niz: fakulteta
Vpišite črko: a
Niz vsebuje črko a.

Vpišite niz: univerza
Vpišite črko: ž
Niz ne vsebuje črke ž.
```

Pri tej nalogi testi niso občutljivi na presledke.

2.4.2 Največje in najmanjše število

Napišite program `min_maks.py`, ki izpiše največjo in najmanjšo izmed treh (celih) števil, ki jih vnese uporabnik. Ne uporabite funkcij `min()` in `max()`.

Primer delovanja programa:

```
Vpišite 1. število: 5
Vpišite 2. število: 8
Vpišite 3. število: 7
Minimum: 5, Maksimum: 8
```

2.4.3 Pretvarjanje temperatur

Napišite program `temperature.py`, ki uporabnika vpraša po stopinjah Celzija in v katero enoto naj jih spremeni (Fahrenheit ali Kelvin). Program izpiše pretvorjeno vrednost. Če uporabnik vnese napačno enoto, mu izpiše opozorilo: `Vnesli ste napačno enoto!`

Pretvorbi izračunamo po formulah: $K = C + 273.15$ in $F = C \cdot \frac{9}{5} + 32$.

Primer delovanja:

```
Vpiši temperaturo [°C]: 32
Želite pretvoriti v Kelvine (vpiši K) ali Fahrenheite (vpiši F)? K
32.0 °C je enako 305.15 K

Vpiši temperaturo [°C]: 23
Želite pretvoriti v Kelvine (vpiši K) ali Fahrenheite (vpiši F)? F
23.0 °C je enako 73.4 °F

Vpiši temperaturo [°C]: 13
Želite pretvoriti v Kelvine (vpiši K) ali Fahrenheite (vpiši F)? C
Vnesli ste napačno enoto!
```

2.4.4 Indeks telesne mase

Napišite program `ITM.py`, ki uporabnika vpraša po telesni višini v in masi. ITM izračunajte po formuli $ITM = teza/visina^2$, pri čemer vnesete težo v enoti kg, višino pa v m. Rezultat zaokrožite na dve decimalni mesti.

Če je indeks manjši od 18,5, uporabniku sporočite, da indeks kaže na prenizko telesno težo. Če je indeks večji od 25, sporočite, da kaže na previsoko telesno težo. V nasprotnem primeru sporočite, da kaže na normalno telesno težo.

Primer delovanja:

```
Telesna višina [cm]: 189
Teža [kg]: 70
Vaš indeks telesne mase je: 19.6
Indeks kaže na normalno telesno težo.

Telesna višina [cm]: 170
Teža [kg]: 53
Vaš indeks telesne mase je: 18.34
Indeks kaže na prenizko telesno težo.

Telesna višina [cm]: 165
Teža [kg]: 70
Vaš indeks telesne mase je: 25.71
Indeks kaže na previsoko telesno težo.
```

2.4.5 Kvadratna enačba

Napišite program `kvadratna_enacba.py`, ki izračuna vse realne rešitve kvadratne enačbe $ax^2 + bx + c = 0$ na štiri decimalna mesta za decimalno vejico. Uporabnik naj vnese vrednosti parametrov a, b in c.

Pri iskanju rešitev kvadratne enačbe si pomagajte z izračunom diskriminante: $D = b^2 - 4ac$.

Pri tem velja:

- $D > 0$: kvadratna enačba ima dve realni rešitvi, $x_{1,2} = \frac{-b \pm \sqrt{D}}{2a}$,
- $D = 0$: kvadratna enačba ima natanko eno realno rešitev, $x = \frac{-b}{2a}$ in
- $D < 0$: kvadratna enačba nima realnih rešitev.

Primer pravilnega delovanja programa:

```
Vpiši a: 1
Vpiši b: 2
Vpiši c: 1
Enačba ima eno realno rešitev: -1.0

Vpiši a: 1
Vpiši b: 2
Vpiši c: 0
Enačba ima dve realni rešitvi: 0.0 in -2.0

Vpiši a: 1
Vpiši b: 2
Vpiši c: 2
Enačba nima realnih rešitev.
```

2.5 Rešitve

2.4.1 Črka v nizu

```
niz = input('Vpišite niz: ')
crka = input('Vpišite črko: ')
if crka in niz:
    print('Niz vsebuje črko '+crka+'.')
else:
    print('Niz ne vsebuje črke '+crka+'.')
```

2.4.2 Največje in najmanjše število

```
st1 = int(input("Vpišite 1. število: "))
st2 = int(input("Vpišite 2. število: "))
st3 = int(input("Vpišite 3. število: "))
maksimum = st1
minimum = st1
if st2 > maksimum:
    maksimum = st2
if st3 > maksimum:
    maksimum = st3
if st2 < minimum:
    minimum = st2
if st3 < minimum:
    minimum = st3
print("Minimum:", str(minimum)+", Maksimum:", maksimum)
```

2.4.3 Pretvarjanje temperatur

```
temp_C = float(input('Vpiši temperaturo [°C]: '))
pretvorba = input('Želite pretvoriti v Kelvine (vpiši K) ali Fahrenheite (vpiši
↪ F)? ')
if pretvorba == 'K':
    temp_K = temp_C + 273.15
    print(str(temp_C) + '°C je enako ' + str(temp_K) + ' K')
elif pretvorba == 'F':
    temp_F = temp_C * 9/5 + 32
    print(str(temp_C) + '°C je enako ' + str(temp_F) + ' °F')
else:
    print('Vnesli ste napačno enoto!')
```

2.4.4 Indeks telesne mase

```
visina = float(input("Telesna višina [cm]: "))
teza = float(input("Teža [kg]: "))
indeks = teza / (visina/100) ** 2
print("Vaš indeks telesne mase je: ", round(indeks,2))
if indeks > 25:
    print("Indeks kaže na previsoko telesno težo.")
elif indeks < 18.5:
    print("Indeks kaže na prenizko telesno težo.")
else:
    print("Indeks kaže na normalno telesno težo.")
```

2.4.5 Kvadratna enačba

```
a = float(input('Vpiši a: '))
b = float(input('Vpiši b: '))
c = float(input('Vpiši c: '))

d = b**2 - 4 * a * c
if d < 0:
    print('Enačba nima realnih rešitev.')
elif d == 0:
    x = -b / (2 * a)
    print('Enačba ima eno realno rešitev:', x)
else:
    x1 = (-b + pow(d, 1/2)) / (2 * a)
    x2 = (-b - pow(d, 1/2)) / (2 * a)
    print('Enačba ima dve realni rešitvi:', round(x1, 4), 'in', round(x2, 4))
```

3. vaje: Zanka *while*

3.1 Možnosti izvedbe zanke *while*

Zanko *while* lahko izvedemo na več načinov. Posebna pozornost naj bo namenjena načrtovanju izstopnih pogojev, saj lahko pri zanki *while* zelo hitro zaidemo v neskončno zanko. Ta se običajno pojavi, ker smo bodisi pozabili definirati pogoj za zaključek izvajanja zanke ali pa smo ga napačno definirali.

Spodaj so na primeru programa iz prejšnjih vaj (Zgled 2.3.1) prikazani trije načini izvedbe zanke, ki se razlikujejo v vstopnih in izstopnih pogojih.

- V izogib ponavljanju `genotip = input("Vpišite genotip: ")` pred zanko `while` definiramo spremenljivko `genotip` in ji priredimo vrednost, za katero smo prepričani, da bo povzročila vstop v zanko. V tem primeru je pogoj za vstop v zanko dolžina genotipa, ki je različna od 2, zato bo katerakoli dolžina niza, prirejenega spremenljivki `genotip`, različna od 2, povzročila vstop v zanko.

```
genotip = "a"
while len(genotip) != 2:
    genotip = input("Vpišite genotip: ")
    if len(genotip) != 2:
        print("Vnesli ste neveljavno dolžino genotipa. Poskusite
        ↪ ponovno.")
```

- Pri določanju vstopnega pogoja lahko uvedemo pomožno spremenljivko. V tem primeru bomo uporabili spremenljivko `x`, ki ji je prirejena vrednost `True`. Dokler bo `x` enak `True`, se bo zanka izvajala. Ko bo uporabnik vnesel genotip dolžine 2 črk, spremenimo vrednost spremenljivke `x` v `False` in na ta način ustavimo zanko.

```
x = True
while x:
    genotip = input("Vpišite genotip: ")
    if len(genotip) == 2:
        x = False
    else:
        print("Vnesli ste neveljavno dolžino genotipa. Poskusite
        ↪ ponovno.")
```

- Z uporabo `break` zanko predčasno zaključimo (glej poglavje 3.2). Na ta način se lahko izognemo določanju pogoja in vpeljavi pomožne spremenljivke. Pozorni moramo biti le na to, da je brez uporabe `break` zanka z `while True` neskončna.

```
while True:
    genotip = input("Vpišite genotip: ")
    if len(genotip) == 2:
        break
    print("Vnesli ste neveljavno dolžino genotipa. Poskusite ponovno.")
```

Pri zanki *while* v osnovi težimo k temu, da uporabimo čim manj funkcij `input()` za isto vsebino. V primeru uporabe `x = True` ali `while True` bodite pozorni na izstopne pogoje iz zanke - nastavitev `x = False` v prvem primeru ali uporabo *break* v drugem.

> *Ko opazite ali načrtujete zanko* `while` *najprej poiščite oziroma določite vstopni in izstopni pogoj.*

3.2 Uporaba *break* in *continue*

V zankah *while* in *for* lahko za nadzor poteka zanke uporabljamo *break* in *continue*. *break* povzroči izhod iz zanke in nadaljevanje programa za zanko, *continue* pa povzroči, da se koda za tem stavkom v dani ponovitvi preskoči in zanka vstopi v naslednjo ponovitev (Slika 7).



Slika 7: Ponazoritev delovanja `break` in `continue`. Z uporabo `break` povzročimo izhod iz zanke, `continue` pa omogoči, da se preostali del kode preskoči in se pomaknemo na naslednjo iteracijo - ostanemo v zanki.

Pri zagonu spodnje kode:

```
stevec = 0

while stevec < 10:
    stevec += 1

    if stevec == 3:
        print("Preskočimo!")
        continue
```

... se nadaljuje na naslednji strani.

```
9     if stevec == 7:
10        print("Gremo iz zanke!")
11        break
12
13    print(f'Število: {stevec}')
14
15 print("Zanka je zaključena.")
```

dobimo izhod:

```
Število: 1
Število: 2
Preskočimo!
Število: 4
Število: 5
Število: 6
Gremo iz zanke!
Zanka je zaključena.
```

3.3 Zgled: Uporaba zanke *while* na primeru titracije

Spodnji program izračuna pH kisle raztopine po vsaki dodani kapljici baze. Zanka `while`, ki se začne v vrstici 15, se izvaja, dokler je razlika med celotno količino baze in količino, že dodano kislini, večja od volumna ene kapljice.

```
1  import math
2
3  cb = float(input("Vnesite koncentracijo NaOH [M]: "))
4  Vb = float(input("Vnesite celoten porabljen volumen NaOH [mL]: "))
5
6  Vdrop = round(float(input("Vnesite volumen kapljice NaOH [mL]: ")),2)
7
8  ck = float(input("Vnesite koncentracijo HCl [M]: "))
9  Vk = float(input("Vnesite volumen raztopine HCl [mL]: "))
10
11 nk = ck*Vk
12
13 Vdrops = 0
14
15 while (Vb-Vdrops)>=Vdrop:
16     Vdrops += Vdrop
17     ndrops = Vdrops*cb
18
19     if ndrops == nk:
```

```
20          print(f'Volumen dodane baze: {round(Vdrops,2)}  pH raztopine: 7.00')
21      elif ndrops < nk:
22          h3o = (nk-ndrops)/(Vk + Vdrops)
23          pH = -math.log10(h3o)
24          print(f'Volumen dodane baze: {round(Vdrops,2)}  pH raztopine:
            ↪  {round(pH,2)}')
25      else:
26          oh = (ndrops-nk)/(Vk + Vdrops)
27          pH = 14-math.log10(oh)
28          if pH >= 14:
29              print(f'Volumen dodane baze: {round(Vdrops,2)}  pH raztopine:
                ↪  14.0')
30              break
31          else:
32              print(f'Volumen dodane baze: {round(Vdrops,2)}  pH raztopine:
                ↪  {round(pH,2)}')
33
34  if Vb-Vdrops != 0:
35      nb = Vb*cb
36      if nb > nk:
37          pH = 14-math.log10((nb-nk)/(Vk + Vb))
38      else:
39          pH = -math.log10((nk-nb)/(Vk+Vb))
40      if pH >= 14:
41          pH = 14
42      print(f'Volumen baze: {round(Vb,2)} pH raztopine: {round(pH,2)}. Porabljena
        ↪  je vsa baza.')
```

V vrstici 34 imamo stavek `if`, ki izračuna končni pH, ko razlika med celotnim volumnom baze in seštevkom vseh dodanih kapljic ni enaka nič. To se lahko zgodi v primeru, ko je `Vdrop` nastavljen na 1 mL, celotne baze pa je 24.9 mL. Brez tega stavka `if` bi program po zanki `while` zadnjo točko določil pri 24 mL in ne pri 24.9 mL.

Izpis programa, kjer celoten porabljen volumen NaOH znaša 25 mL:

```
Vnesite koncentracijo NaOH [M]: 0.2
Vnesite celoten porabljen volumen NaOH [mL]: 25
Vnesite volumen kapljice NaOH [mL]: 1
Vnesite koncentracijo HCl [M]: 0.1
Vnesite volumen raztopine HCl [mL]: 50
Volumen dodane baze: 1.0  pH raztopine: 1.03
Volumen dodane baze: 2.0  pH raztopine: 1.05
Volumen dodane baze: 3.0  pH raztopine: 1.08
Volumen dodane baze: 4.0  pH raztopine: 1.11
Volumen dodane baze: 5.0  pH raztopine: 1.14
Volumen dodane baze: 6.0  pH raztopine: 1.17
Volumen dodane baze: 7.0  pH raztopine: 1.2
Volumen dodane baze: 8.0  pH raztopine: 1.23
Volumen dodane baze: 9.0  pH raztopine: 1.27
Volumen dodane baze: 10.0  pH raztopine: 1.3
Volumen dodane baze: 11.0  pH raztopine: 1.34
Volumen dodane baze: 12.0  pH raztopine: 1.38
Volumen dodane baze: 13.0  pH raztopine: 1.42
Volumen dodane baze: 14.0  pH raztopine: 1.46
Volumen dodane baze: 15.0  pH raztopine: 1.51
Volumen dodane baze: 16.0  pH raztopine: 1.56
Volumen dodane baze: 17.0  pH raztopine: 1.62
Volumen dodane baze: 18.0  pH raztopine: 1.69
Volumen dodane baze: 19.0  pH raztopine: 1.76
Volumen dodane baze: 20.0  pH raztopine: 1.85
Volumen dodane baze: 21.0  pH raztopine: 1.95
Volumen dodane baze: 22.0  pH raztopine: 2.08
Volumen dodane baze: 23.0  pH raztopine: 2.26
Volumen dodane baze: 24.0  pH raztopine: 2.57
Volumen dodane baze: 25.0  pH raztopine: 7.00
```

Izpis programa, kjer celoten porabljen volumen NaOH znaša 24.9 mL:

```
Vnesite koncentracijo NaOH [M]: 0.2
Vnesite celoten porabljen volumen NaOH [mL]: 24.9
Vnesite volumen kapljice NaOH [mL]: 1
Vnesite koncentracijo HCl [M]: 0.1
Vnesite volumen raztopine HCl [mL]: 50
Volumen dodane baze: 1.0  pH raztopine: 1.03
Volumen dodane baze: 2.0  pH raztopine: 1.05
Volumen dodane baze: 3.0  pH raztopine: 1.08
Volumen dodane baze: 4.0  pH raztopine: 1.11
Volumen dodane baze: 5.0  pH raztopine: 1.14
Volumen dodane baze: 6.0  pH raztopine: 1.17
Volumen dodane baze: 7.0  pH raztopine: 1.2
Volumen dodane baze: 8.0  pH raztopine: 1.23
Volumen dodane baze: 9.0  pH raztopine: 1.27
Volumen dodane baze: 10.0  pH raztopine: 1.3
Volumen dodane baze: 11.0  pH raztopine: 1.34
Volumen dodane baze: 12.0  pH raztopine: 1.38
Volumen dodane baze: 13.0  pH raztopine: 1.42
Volumen dodane baze: 14.0  pH raztopine: 1.46
Volumen dodane baze: 15.0  pH raztopine: 1.51
Volumen dodane baze: 16.0  pH raztopine: 1.56
Volumen dodane baze: 17.0  pH raztopine: 1.62
Volumen dodane baze: 18.0  pH raztopine: 1.69
Volumen dodane baze: 19.0  pH raztopine: 1.76
Volumen dodane baze: 20.0  pH raztopine: 1.85
Volumen dodane baze: 21.0  pH raztopine: 1.95
Volumen dodane baze: 22.0  pH raztopine: 2.08
Volumen dodane baze: 23.0  pH raztopine: 2.26
Volumen dodane baze: 24.0  pH raztopine: 2.57
Volumen baze: 24.9 pH raztopine: 3.57. Porabljena je vsa baza.
```

3.4 Naloge

3.4.1 Poštevanka

Napišite program `postevanka.py`, ki uporabnika pozove k vnosu števila, nato pa na zaslon izpiše večkratnike podanega števila do 10.

Primer delovanja:

```
Vnos števila: 5
5
10
15
20
25
30
35
40
45
50
```

3.4.2 Vse po pet

Napišite program `vse_po_pet.py`, ki zahteva, da morajo stranke v trgovini vedno kupiti natanko pet artiklov. Program uporabnika vpraša po petih cenah; ko jih uporabnik vnese, program izpiše vsoto.

Primer delovanja:

```
Cena artikla: 2
Cena artikla: 4
Cena artikla: 1
Cena artikla: 6
Cena artikla: 3
Vsota: 16
```

3.4.3 Konkurenca

Program `vse_po_pet.py` popravite tako, da blagajnika najprej vpraša, koliko izdelkov je v košarici, nato vpraša po cenah teh izdelkov in na koncu izpiše vsoto. Program poimenujte `konkurenca.py`.

Primer delovanja:

```
Število izdelkov: 3
Cena artikla: 2
Cena artikla: 4
Cena artikla: 1
Vsota: 7
```

3.4.4 Top shop

Popravite program `konkurenca.py` tako, da ta uporabnika sprašuje po cenah toliko časa, dokler mu uporabnik ne vnese ničle. Program poimenujte `top_shop.py`.

Primer delovanja:

```
Cena artikla: 2
Cena artikla: 4
Cena artikla: 1
Cena artikla: 0
Vsota: 7
```

3.4.5 Povprečna cena

Popravite program `top_shop.py` tako, da bo izpisal tudi povprečno ceno. To zaokrožite na 5 decimalnih mest. Program poimenujte `povprecna_cena.py`.

Primer delovanja:

```
Cena artikla: 2
Cena artikla: 4
Cena artikla: 1
Cena artikla: 0
Vsota: 7
Povprečna cena: 2.33333
```

3.4.6 Trojni stop

Napišite program `trojni_stop.py`, ki mu uporabnik vnaša cene in ki se neha izvajati, ko uporabnik vnese 0 (ne bo več kupoval), ko je vnešenih deset števil ali ko vsota cen doseže ali preseže 100 evrov.

Primer delovanja, ko uporabnik vnese 0:

```
Cena: 10
Cena: 5
Cena: 0
Porabili boste 15 EUR za 2 stvari.
```

Primer delovanja, ko uporabnik preseže 100 evrov:

```
Cena: 10
Cena: 5
Cena: 90
Porabili boste 105 EUR za 3 stvari.
```

Primer delovanja, ko uporabnik kupi 10 artiklov:

```
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Cena: 1
Porabili boste 10 EUR za 10 stvari.
```

3.4.7 Tekoči račun

Napišite bančni program, poimenovan `tekoci_racun.py`, kamor uporabniki vtipkavajo prejemke in izdatke v obliki pozitivnih in negativnih zneskov na svojem tekočem računu. Program jim sproti izpisuje stanje in se ustavi, ko je uporabnik v minusu za 100 evrov ali več.

Primer delovanja:

```
Sprememba: 23
Stanje: 23
Sprememba: 15
Stanje: 38
Sprememba: -30
Stanje: 8
Sprememba: 10
Stanje: 18
Sprememba: 100
Stanje: 118
Sprememba: -200
Stanje: -82
Sprememba: -50
Stanje: -132
Bankrot!
```

3.5 Rešitve nalog

3.4.1 Poštevanka

```
st = int(input("Vnos števila: "))
i = 1
while i <= 10:
    print(i * st)
    i += 1
```

3.4.2 Vse po pet

```
vsota = 0
i = 0
while i < 5:
    vsota += int(input('Cena artikla: '))
    i += 1
print('Vsota:', vsota)
```

Rešitev z zanko *for*:

```
vsota = 0
for i in range(5):
    vsota += int(input('Cena artikla: '))
print('Vsota:', vsota)
```

3.4.3 Konkurenca

```
vsota = 0
i = int(input('Število izdelkov: '))
while i > 0:
    vsota += int(input('Cena artikla: '))
    i -= 1
print('Vsota:', vsota)
```

Rešitev z zanko *for*:

```
vsota = 0
for i in range(int(input('Število izdelkov: '))):
    vsota += int(input('Cena artikla: '))
print('Vsota:', vsota)
```

3.4.4 Top shop

```
vsota = 0
# Spremenljivko cena najprej nastavimo na 1, da se zanka sploh začne izvajati.
cena = 1
while cena != 0:
    cena = int(input('Cena artikla: '))
    vsota += cena
print('Vsota:', vsota)
```

3.4.5 Povprečna cena

```
artiklov = -1
vsota = 0
cena = 1
while cena != 0:
    cena = int(input('Cena artikla: '))
    vsota += cena
    artiklov += 1
print('Vsota:', vsota)
if artiklov > 0:
    povprecje = vsota / artiklov
else:
    povprecje = 0.0
print('Povprečna cena:', round(povprecje, 5))
```

3.4.6 Trojni stop

```
artiklov = 0
vsota = 0
cena = 1
while cena != 0 and vsota < 100 and artiklov < 10:
    cena = int(input('Cena: '))
    vsota += cena
    artiklov += 1

if cena == 0:
    artiklov -= 1

print('Porabili boste', vsota, 'evrov za', artiklov, 'stvari.')
```

3.4.7 Tekoči račun

```
stanje = 0
while stanje > -100:
    stanje += int(input('Sprememba: '))
    print('Stanje:', stanje)
print('Bankrot!')
```

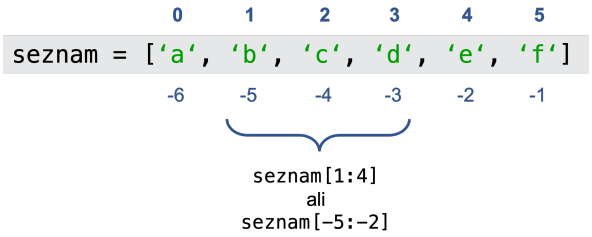
4. vaje: Seznami in zanka *for*

4.1 Seznami

Seznami so zbirke elementov, ki so lahko različnih podatkovnih tipov. Ker gre za spremenljiv podatkovni tip, lahko elemente in njihov vrstni red znotraj seznama prosto dodajamo, urejamo ali odstranjujemo.

Elementi v seznamu so urejeni, do njih pa lahko dostopamo s pomočjo indeksiranja. Kadar se želimo po seznamu pomikati iz leve proti desni, se indeksiranje vedno začne z 0, če pa se želimo po seznamu pomikati iz desne proti levi, ima zadnji element indeks -1. Iz seznama lahko z indeksiranjem izluščimo en ali več elementov. Pri slednjem lahko uporabimo rezine, kjer indekse želenih elementov zapišemo po sintaksi:

Pri uporabi rezin je sintaksa naslednja: `[od:do:korak]` (angl. `[start:stop:step]`). Pri tem pazite, kaj rezina dejansko vsebuje. Naj bosta `x` in `y` indeksa v poljubno dolgem seznamu `s`, ki označujeta meje želene rezine. Če rezino zapišemo kot `s[x:y]`, bo ta vsebovala elemente med indeksoma `x` in `y-1`, saj rezinjenje vedno poteka od prvega izbranega elementa do ne vključno zadnjega (gre za polodprt interval: $[x, y-1)$). Če želimo zajeti tudi element na indeksu `y`, moramo rezino zapisati `s[x,y+1]`[5] (Slika 8).



Slika 8: Indeksiranje in rezine pri seznamih. Če indeksiramo od leve proti desni, bo prvi indeks enak `0`, zadnji pa `n`; če pa z desne proti levi, bo imel zadnji element seznama indeks `-1`, prvi pa `-n` pri čemer je `n` število elementov seznama.

Če bi želeli izpisati vse elemente od indeksa `y` pa do konca seznama, lahko to storimo z rezino `s[y:]`, pri čemer so vključeni vsi elementi vključno od indeksa `y` dalje. Če želimo izpisati le elemente do indeksa `y`, to storimo z rezino `s[:y]`. Ponovno bodite pozorni, da bo takšna rezina zajela le elemente do vključno elementa z indeksom `y-1`, ne pa tudi elementa z indeksom `y`.

Vrstni red elementov v seznamu lahko obrnemo z uporabo rezine `[::-1]`:

```
1 seznam = ['a','b','c','d']
2 seznam = seznam[::-1]
3 print(seznam)
```

[5] Enako velja pri uporabi funkcije `range(a,b)`, ki bo zajela števila na polodprtem intervalu $[a, b)$.

Zgornji program na zaslon izpiše:

```
['d','c','b','a']
```

Ker so seznami spremenljivi[6], lahko elemente znotraj seznama spremenimo. Če želimo na indeksu `0` zamenjati niz `'d'` z nizom `'e'`, to storimo na sledeč način:

```
seznam = ['d','c','b','a']
seznam[0] = 'e'
print(seznam)
```

in dobimo:

```
['e','c','b','a']
```

Nov element lahko v seznam dodamo na dva načina: z metodo `.append()` ali pa z operacijo seštevanja:

```
seznam.append('f')
print(f'V seznam smo dodali "f": {seznam}')

seznam += 'g'
print(f"V seznam smo dodali 'g': {seznam}")

seznam += ['h']
print(f'V seznam smo dodali "h": {seznam}')
```

Zgornji program izpiše:

```
V seznam smo dodali "f": ['e', 'c', 'b', 'a', 'f']
V seznam smo dodali 'g': ['e', 'c', 'b', 'a', 'f', 'g']
V seznam smo dodali "h": ['e', 'c', 'b', 'a', 'f', 'g', 'h']
```

Z uporabo pogojnega stavka *if* in operacij ***in*** ter ***not in***, lahko hitro preverimo, ali se nek element nahaja v seznamu ali ne:

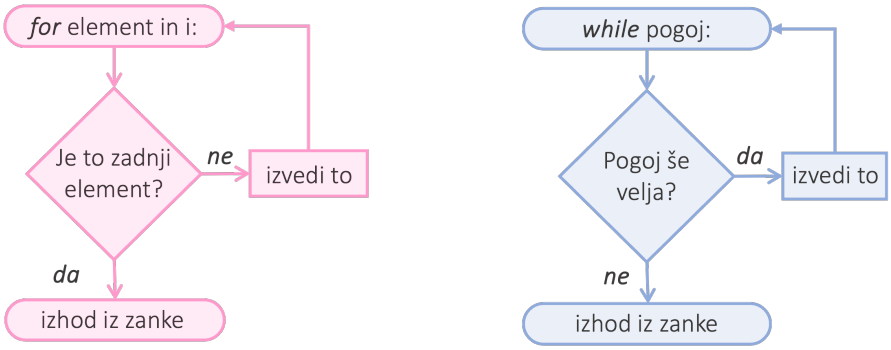
```
if 'c' in seznam:
    print("'c' se nahaja v seznamu.")

if 'c' not in seznam:
    print("'c' se ne nahaja v seznamu.")
```

[6]Več o spremenljivosti na 6. vajah.

4.2 Zanka *for*

Zanka *for* je nekoliko podobna zanki *while*. Zanka *while* se izvaja, dokler velja pogoj, zanka *for* pa je omejena s številom elementov v seznamu ali nizu, po katerem se premikamo (Slika 9).



Slika 9: Primerjava izvajanja zank *for* in *while*. Zanka *for* teče tako dolgo, dokler ne pridemo do zadnjega elementa v seznamu ali nizu `i`. Zanka *while* pa se izvaja tako dolgo, dokler je navedeni pogoj veljaven.

Pri zanki *while* smo opozarjali na nevarnost neskončne zanke, ki je pri zanki *for* ni. V praksi lahko marsikatero zanko *while* preoblikujemo v zanko *for* in obratno - to je še posebej uporabno, kadar ne najdemo dobrega pogoja za *while*.

Pomikanje po seznamu lahko z zanko *for* izvedemo na dva načina:

1. po elementih

```
1 for element in seznam:
2     print(element)
```

Pri tem načinu nam beseda `element` v 1. vrstici kode ponazarja element v tem seznamu. Poimenovanje je lahko poljubno, saj bo Python v kombinaciji z `for` ter `in ime_seznama` vedel, da je tako označen posamezen element.

2. z indeksiranjem

```
1 for i in range(len(seznam)):
2     print(seznam[i])
```

Funkcija `range()` v ozadju pripravi seznam števil med 0 in dolžino seznama[7]. Do posameznega elementa seznama dostopamo z indeksiranjem seznama: `seznam[i]`.

Enak program bi z zanko *while* zapisali na sledeč način:

```
1 i = 0
2 while i < len(seznam):
3     print(seznam[i])
4     i += 1
```

[7]Če je `len(seznam)` enako `n`, bo funkcija `range(len(seznam))` pripravila seznam števil na intervalu $[0, n)$.

Vsi trije programi vrnejo izpis elementov, enega za drugim, na zaslonu:

```
e
c
b
a
f
g
h
```

> *Med zelo pogostimi napakami je mešanje pomikanja po elementih, kjer je* `element` *že element v seznamu, in indeksiranjem, kjer je* `i` *zgolj indeks (število).*

Če želimo spreminjati vrednosti v celotnem seznamu, moramo zanko *for* izvesti z indeksiranjem[8].

4.3 Gnezdenje zank *for*

Z zanko *for* se želimo sprehoditi po istoležnih elementih seznamov `s1 = [0, 1, 2]` in `s2 = ['a', 'b', 'c']`.

Zanka *for* ne omogoča uporabe več kot ene kombinacije "`element in seznam`" naenkrat - povedano drugače: Python ne sprejme sintakse: `for i in s1 and j in s2`. Za izvedbo sprehoda po istoležnih elementih seznamov uporabimo indeksiranje - ob pogoju, da sta enako dolga (`len(s1) == len(s2)`):

```
1 for i in range(len(s1)):
2     print(s1[i], s2[i])
```

Če bi uporabili ugnezdeni zanki:

```
1 for i in s1:
2     for j in s2:
3         print(i, j)
```

bi program izpisal vse možne kombinacije elementov obeh seznamov:

```
0 a
0 b
0 c
1 a
1 b
1 c
2 a
2 b
2 c
```

Zadnje število bo torej `n-1`.

[8]To je povezano s spremenljivostjo podatkovnih tipov, o kateri bomo govorili na 6. vajah.

To se zgodi, ker notranja zanka teče hitreje od zunanje - šele, ko se sprehodimo skozi celoten `s2`, se lahko premaknemo iz `s1[0]` na `s1[1]`.

4.4 *Hitra izpeljava seznama (angl. *list comprehension*)

Sezname lahko hitro ustvarjamo na način, znan kot *list comprehension*. Hitri način sledi sintaksi `seznam = [element for element in y]`, pri čemer je `y` lahko zbirka elementov (npr. seznam, slovar, ...).

Zelo hitro lahko pripravimo sezname iz števil ali pa pripravimo nove, s spremenjenimi vrednostmi:

```
>>> seznam1 = [x for x in range(0,11)]
>>> seznam1
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> seznam2 = [x**2 for x in seznam1]
>>> seznam2
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Pri uporabi hitrega načina ne rabimo predhodno pripraviti praznega seznama, saj se bo tvoril avtomatsko.

4.5 Zgledi z nalogami za utrjevanje

4.5.1 Izpis kvadratov elementov seznama

```
1 seznam = [-3, 7, -9, 14, -19, 21]
2 for stevilo in seznam:
3     print(stevilo**2)
```

Program na zaslon izpiše kvadrate števil, ki se nahajajo v seznamu:

```
9
49
81
196
361
441
```

Utrjevanje: zgornji program preoblikujte na dva načina:

1. namesto zanke *for* uporabite zanko *while*,
2. kvadrate števil iz podanega seznama shranite v nov seznam.

4.5.2 Ločevanje negativnih in pozitivnih števil

```
1  seznam = [-3, 7, -9, 14, -19, 21]
2
3  negativna = []
4  pozitivna = []
5
6  for stevilo in seznam:
7      if stevilo <= 0:
8          negativna.append(stevilo)
9      else:
10         pozitivna.append(stevilo)
11
12 print(f'Seznam pozitivnih števil: {pozitivna}')
13 print(f'Seznam negativnih števil: {negativna}')
```

Izpis programa:

```
Seznam pozitivnih števil: [7, 14, 21]
Seznam negativnih števil: [-3, -9, -19]
```

Utrjevanje: zgornji program napišite z uporabo zanke *while*.

4.5.3 Računanje povprečja

```
seznam = [15, 40, 5, 1, 12]

vsota = 0

for element in seznam:
    vsota += element

if len(seznam) != 0:
    povprecje = round(vsota/len(seznam), 5)
else:
    povprecje = 0

print(f'Povprečje seznama {seznam} je {povprecje}.')
```

Program izpiše:

```
Povprečje seznama [15, 40, 5, 1, 12] je 14.6.
```

Utrjevanje: zgornji program preoblikujte na naslednje načine:

1. namesto zanke *for* uporabite zanko *while*,
2. z uporabo funkcij `eval` in `input` omogočite, da uporabnik sam vnese poljuben seznam.

4.5.4 Štetje nukleotidov

```
dna_zaporedje = 'TCGCGGTTCGTTGGA'

a = 0
t = 0
g = 0
c = 0

for nukleotid in dna_zaporedje:
    if nukleotid == "A":
        a += 1
    elif nukleotid == "T":
        t += 1
    elif nukleotid == "G":
        g += 1
    elif nukleotid == "C":
        c += 1

print(f'Št. A: {a} \nŠt. T: {t} \nŠt. G: {g} \nŠt. C: {c}')
```

Izpis programa:

```
Št. A: 1
Št. T: 5
Št. G: 6
Št. C: 3
```

Utrjevanje: zgornji program preoblikujte na sledeče načine:

1. omogočite, da uporabnik vnese poljubno dolgo zaporedje,
2. če je v zaporedju črka, ki je različna od A, T, G in C, naj program preneha s štetjem in uporabnika pozove k vnosu pravilnega zaporedja,
3. dodajte izračun, koliko odstotkov celotnega zaporedja sestavlja pojavnost posameznega nukleotida,
4. kodo skrajšajte, kolikor je mogoče. Pomagajte si z metodo `.count()`.

4.6 Naloge

4.6.1 Iskanje minimuma

Napišite program `iskanje_minimuma.py`, ki poišče in izpiše najmanjši element v seznamu celih števil brez uporabe funkcije `min()` ali sortiranja! Seznam števil naj poda uporabnik, pri čemer lahko za interpretacijo vhodnega niza kot seznam uporabite funkcijo `eval()`, ki omogoča avtomatsko prepoznavo vnesenega podatkovnega tipa[9].

Primer delovanja:

```
Vpišite seznam števil: [23, 42, 87, 34, 1, -3, 2]
-3
```

4.6.2 Niz v seznamu

Napišite program `niz_v_seznamu.py`, ki ugotovi, ali je nek niz vsebovan v predhodno definiranem seznamu nizov, definiranem kot:
`seznam = ["beseda", "spremenljivka", "niz", "zanka", "stavek", "slovar"]`.

Program naj uporabnika vpraša po nizu, ki ga išče, nato pa izpiše, ali je ta niz vsebovan v seznamu.

Nalogo poskusite rešiti z uporabo zanke in brez nje.

Primera delovanja programa:

```
Vpišite iskani niz: zanka
Seznam vsebuje niz "zanka".

Vpišite iskani niz: vrednost
Seznam ne vsebuje niza "vrednost".
```

Testi pri tej nalogi niso občutljivi na presledke.

[9] V primeru seznamov je nujno treba vnesti tudi oklepaje.

4.6.3 Števila v seznamu

Napišite program `stevilo_v_seznamu.py`, ki ugotovi, ali je število, ki ga vpiše uporabnik, v predhodno definiranem seznamu celih števil. Ta seznam je: `seznam = [3, 35, 7, 68, 9, 10, 12, 481, 17, 12, 31, 21, 98, 33]`.

Nalogo poskusite rešiti z uporabo zanke in brez nje.

Primera delovanja programa:

```
Vpišite število: 9
Seznam vsebuje število 9

Vpišite število: 4
Seznam ne vsebuje števila 4
```

4.6.4 Vsota in povprečje

Napišite program `vsota_in_povprecje.py`, ki za podani seznam izračuna vsoto (brez uporabe funkcije `sum`) in povprečje elementov, zaokroženo na 5 decimalk. Seznam števil naj vnese uporabnik.

Primer delovanja:

```
Vpišite seznam števil: [23, 42, 87, 34, 1, -3, 2]
186
26.57143
```

4.6.5 Iskanje večkratnikov

Napišite program `iskanje_veckratnikov.py`, ki preveri, ali seznam vsebuje vsaj en večkratnik izbranega števila. Seznam in izbrano število poda uporabnik. Nalogo poskusite rešiti tudi z uporabo `break`.

Primera delovanja:

```
Vpišite seznam števil: [23, 42, 87, 34, 1, -3, 2]
Vnesite število: 3
Vsebuje.

Vpišite seznam števil: [23, 42, 87, 34, 1, -3, 2]
Vnesite število: 8
Ne vsebuje.
```

4.6.6 Samo večkratniki

Program `iskanje_veckratnikov.py` spremenite tako, da bo preverjal, ali seznam vsebuje zgolj večkratnike izbranega števila. Seznam in izbrano število poda uporabnik, nalogo pa poskusite rešiti tudi z uporabo `break`.

Primera delovanja:

```
Vpišite seznam števil: [27, 21, 3, 33, 60, -3]
Vnesite število: 3
Vsebuje.

Vpišite seznam števil: [27, 21, 3, 33, 60, -3]
Vnesite število: 9
Ne vsebuje.
```

4.6.7 Izris trikotnika

Po programerski tradiciji je eden prvih programov, ki jih napišemo v določenem programskem jeziku, izris trikotnika iz zvezdic. Napišite program `izris_trikotnika.py`, ki vpraša uporabnika po višini trikotnika, nato pa izpiše takšen trikotnik iz zvezdic:

```
Vpiši višino: 4
 *
 * *
 * * *
 * * * *
```

*4.6.8 *Izris smrekice*

Napišite program `izris_smrekice.py`, ki bo namesto trikotnikov izrisoval „smrekice“.

Namig: prazni prostori in zvezdice so povezani s formulo za opis zaporedja.

```
Vpiši višino: 4
       *
     * * *
   * * * * *
 * * * * * * *
```

Testi so občutljivi na presledke, zato jih ne uporabljajte tam, kjer niso potrebni.

4.7 Rešitve nalog

4.6.1 Iskanje minimuma

```
seznam = eval(input('Vpišite seznam števil: '))
minimum = seznam[0]
for elt in seznam:
    if elt < minimum:
        minimum = elt
print(minimum)
```

4.6.2 Niz v seznamu

```
seznam = ["beseda", "spremenljivka", "niz", "zanka", "stavek", "slovar"]
niz=input('Vpišite iskani niz: ')
if niz in seznam:
    print('Seznam vsebuje niz "'+niz+'".')
else:
    print('Seznam ne vsebuje niza "'+niz+'".')
```

4.6.3 Števila v seznamu

```
seznam = [3, 35, 7, 68, 9, 10, 12, 481, 17, 12, 31, 21, 98, 33]
st=int(input('Vpišite število: '))
if st in seznam:
    print('Seznam vsebuje število',st)
else:
    print('Seznam ne vsebuje števila', st)
```

4.6.4 Vsota in povprečje

```
seznam = eval(input("Vpišite seznam števil: "))
vsota = 0
for elt in seznam:
    vsota += elt
print(vsota)
if len(seznam) != 0:
    print(round(vsota/len(seznam), 5))
else:
    print(0)
```

4.6.5 Iskanje večkratnikov

```
seznam = eval(input("Vpišite seznam števil: "))
st = int(input("Vnesite število: "))

vsebuje = False
for elt in seznam:
    if elt % st == 0:
        vsebuje = True
        break
if vsebuje:
    print("Vsebuje.")
else:
    print("Ne vsebuje.")
```

4.6.6 Samo večkratniki

```
seznam = eval(input("Vpišite seznam števil: "))
st = int(input("Vnesite število: "))

samo_veckratniki = True
for elt in seznam:
    if elt % st != 0:
        samo_veckratniki = False
        break
if samo_veckratniki:
    print("Vsebuje.")
else:
    print("Ne vsebuje.")
```

4.6.7 Izris trikotnika

```
n = int(input('Vpiši višino: '))

for i in range(1, n + 1):
    print('*' * i)
```

4.6.8 Izris smrekice

```
n = int(input('Vpiši višino: '))

for i in range(1, n + 1):
    print(' ' * (n - i) + '*' * (2 * i - 1))
```

5. vaje: Funkcije

Funkcije nam omogočajo, da določeno kodo ponovno uporabimo, brez da jo moramo ponovno v celoti izpisati. Kodo, ki jo želimo večkrat uporabiti, zamaknemo in pred njo zapišemo `def`, želeno ime funkcije, s katerim bomo to kodo klicali ter v oklepaju navedemo argumente:

```
def ime_funkcije(argumenti):
    sekvenca ukazov
    return rezultat_funkcije
```

5.1 return vs. print()

Za razliko od kode, ki smo jo pisali do sedaj, tukaj pri vračanju rezultata (in zaključku funkcije) uporabimo besedico `return` namesto `print()`. Funkcija `print()` namreč rezultat samo izpiše na zaslon in ga ne more prirediti spremenljivki, medtem ko `return` rezultat lahko priredi spremenljivki in s tem rezultatom lahko operiramo dalje. To najlepše ponazorimo s primerom dveh funkcij, pri katerih je ena zaključena z `return`, druga pa s `print()`:

```
def povprecje(seznam):
    if len(seznam) != 0:
        povprecje = sum(seznam)/len(seznam)
    else:
        povprecje = 0
    return povprecje

def povprecje_print(seznam):
    if len(seznam) != 0:
        povprecje = sum(seznam)/len(seznam)
    else:
        povprecje = 0
    print(povprecje)
```

Ko poženemo `.py` datoteko, lahko v lupini pokličemo funkciji in jima podamo argument:

```
>>> seznam = [1, 5, 6, 7, 1, 10]
>>> povprecje(seznam)
5.0
>>> povprecje_print(seznam)
5.0
```

Do sem izhoda funkcij izgledata enako, vendar se situacija spremeni, ko želimo izhoda funkcij prirediti spremenljivkama:

```
>>> seznam = [1, 5, 6, 7, 1, 10]
>>> a = povprecje(seznam)
>>> a
5.0
>>> b = povprecje_print(seznam)
5.0
>>> b
>>>
```

V primeru uporabe funkcije `povprecje` izračunano brez težav priredimo spremenljivki `a` in ob klicu spremenljivke vidimo rezultat funkcije. Pri funkciji `povprecje_print` pa se ob poskusu prirejanja vrednosti spremenljivki `b` kar takoj izpiše rezultat, v spremenljivki `b` pa ob njenem klicu vidimo, da ni prirejenega rezultata. Še lepše to ponazorimo, ko želimo računati naprej:

```
>>> a + 0.5
5.5
>>> b + 0.5
Traceback (most recent call last):
  File "<pyshell7>", line 1, in <module>
    b + 0.5
TypeError: unsupported operand type(s) for +: 'NoneType' and 'float'
```

Besedica `return` namreč zagotovi, da rezultat funkcije ohrani podatkovni tip, funkcija `print` pa tega ne omogoča.

5.2 Položaj `return` v funkciji

`return` hkrati deluje kot `break` ter zaključi izvajanje funkcije. V primeru zgornje funkcije `povprecje()`, bi lahko `return` uporabili prej in nekoliko poenostavili kodo:

```
1 def povprecje(seznam):
2     if len(seznam) != 0:
3         return sum(seznam)/len(seznam)
4     return 0
```

Tako zapisana funkcija preverja, ali seznam vsebuje elemente - če jih, bo stavek v 3. vrstici vrnil izračunano povprečje in zaključil funkcijo. Nobena nadaljnja koda znotraj te funkcije se ne bo izvajala, saj `return` deluje kot `break`. Če pogoj o polnosti seznama ne velja, pa bo funkcija vrnila 0 (glej 4. vrstico) - `else` v takšnem primeru ni potreben.

5.3 Opcijski argument

Argumente funckij delimo na obvezne in opcijske. Brez obveznih se funkcija ne more izvesti, opcijski pa omogočajo dodatne funcionalnosti ali nastavitve.

Poglejmo si funkcijo za izračun molske in/ali masne koncentracije:

```
1 def izracun_c(m, M, V, c=False):
2     if c == True:
3         konc = m/V # izračun masne koncentracije
4     else:
5         konc = m/(M*V) # izračun množinske koncentracije
6     return konc
```

Obvezni argumenti zgornje funkcije so `m` (masa topljenca), `M` (molska masa topljenca) in `V` (volumen raztopine). Molska koncentracija, `c`, je opcijski argument. Če pri klicu funkcije opcijskega argumenta eksplicitno ne podamo, bo ta ostal nastavljen na privzeto vrednost - v našem primeru `False`. Če opcijski argument spremenimo, se bo izvedla koda pod `else` (glej 4. vrstico) in funkcija bo vrnila množinsko koncentracijo.

```
>>> izracun_c(10, 58.44, 0.1)
1.7111567419575633
>>> izracun_c(10, 58.44, 0.1, c=False)
1.7111567419575633
>>> izracun_c(10, 58.44, 0.1, c=True)
100.0
```

5.4 Uporaba pass

V primeru, da pisanja funkcije nismo dokončali, dodamo besedo `pass`, saj s tem omogočimo, da se program (in testi) izvede(jo) normalno, brez izvajanja nedokončane funkcije. Ukaz `pass` lahko vnesemo kamorkoli znotraj funkcije, pod pogojem, da nismo uporabili `return`.

```
1 def nedokoncana_funkcija1(arg1, arg2):
2     pass
3
4 def nedokoncana_funkcija2(arg1, arg2):
5     b = "abcd"
6     if arg1 in b:
7         pass
```

... se nadaljuje na naslednji strani.

```
8  def nedokoncana_funkcija3(arg1, arg2):
9      b = "abcd"
10     if arg1 in b:
11         b += arg1
12     pass
```

5.5 Zgledi

5.5.1 Iskanje minimuma in maksimuma v seznamu

Pogosto želimo poiskati minimum ali maksimum v nekem seznamu. Python ima že vgrajeni funkciji `min()` in `max()`, ki počneta ravno to, vendar se pogosto zgodi, da ju v določenih primerih ne moremo uporabiti.

```
1  def maksimum(seznam):
2      najvecji = seznam[0]
3      for element in seznam:
4          if element > najvecji:
5              najvecji = element
6      return najvecji
```

```
1  def minimum(seznam):
2      najmanjsi = seznam[0]
3      for element in seznam:
4          if element < najmanjsi:
5              najmanjsi = element
6      return najmanjsi
```

Bistvo funkcij je, da jih poskušamo spisati čim bolj splošno - na konkretnem primeru to pomeni, da je lahko seznam, ki ga bo vnesel uporabnik, seznam bodisi nenegativnih števil, negativnih števil ali vseh. Zato pri iskanju maksimuma in minimuma uvedemo dve novi spremenljivki (glej vrstico 2), ki ju nastavimo na prvo vrednost seznama. Nato se z zanko *for* pomikamo po seznamu in primerjamo posamezen element s trenutno vrednostjo, shranjeno v `najvecji` oziroma `najmanjsi`. Če trenutni element izpolnjuje pogoja pri pogojnih stavkih *if* v 4. vrstici, v spremenljivki shranimo novi vrednosti.

Za lažjo ponazoritev in sledenje izvajanju funkcije dodamo `print()` stavke[10]:

```
1  def maksimum(seznam):
2      najvecji = seznam[0]
3      print(f'Trenutna vrednost v najvecji: {najvecji}')
4      for element in seznam:
5          print(f'Element: {element}')
6          if element > najvecji:
7              najvecji = element
8              print(f'Sprememba vrednosti v najvecji!')
9      return najvecji
```

[10]Glej poglavje Razhroščevanje.

Datoteko s funkcijo zaženemo in pokličemo funkcijo `maksimum`:

```
>>> s = [-1, 2, 20, -4, 5]
>>> maksimum(s)
Trenutna vrednost v najvecji: -1
Element: -1
Element: 2
Sprememba vrednosti v najvecji!
Element: 20
Sprememba vrednosti v najvecji!
Element: -4
Element: 5
20
```

5.5.2 Uporaba pomožnih funkcij

Včasih je smiselno del večje funkcije spisati kot samostojno oziroma pomožno funkcijo, še posebej, če se ta del kode večkrat ponovi. Takšne pomožne funkcije izvajajo enostavne naloge in izboljšajo berljivost kode. Obenem nam lahko olajšajo testiranje, razhroščevanje in vzdrževanje kode, saj popravke ob morebitnih spremembah naredimo samo na enem mestu. Na ta način gradimo hierarhijo funkcij, kjer se kompleksnejše rešitve opirajo na enostavnejše gradnike.

V spodnjem primeru funkcija `mnozina` opravi osnovni izračun, ki se nato uporabi v funkciji `koncni_volumen`, ta pa se naprej uporabi v funkciji `dodatek_volumna`.

```
def mnozina(koncentracija, volumen):
    return round(koncentracija*volumen, 2)

def koncni_volumen(c1, v1, c2):
    v2 = mnozina(c1, v1)/c2
    return round(v2,2)

def dodatek_volumna(konc1, vol1, konc2):
    vol2 = koncni_volumen(konc1, vol1, konc2)
    vol2 = round(vol2,2)
    if vol2 > vol1:
        return f'K začetni raztopini dodajte {vol2-vol1} topila.'
    return f'Iz začetne raztopine odstranite {vol1-vol2} topila.'
```

5.6 Naloge za utrjevanje

1. Funkcijo `izracun_c` iz podpoglavja 5.3 skrajšajte z uporabo dveh `return`. Pomagajte si z zgledom iz podpoglavja 5.2.
2. Napišite funkcijo `generiraj_dna`, ki generira zaporedje DNA z dolžino, ki jo kot argument funkciji poda uporabnik. Za naključno dodajanje nukleotidov boste potrebovali modul `random`, v katerem je funkcija `random.choice()`[11] Dodajte tudi opcijski argument, ki omogoča generiranje RNA zaporedja.
3. V poglavju 3.3.1 je zapisan program za izračun pH med titracijo kisle raztopine z bazo. Program zapišite v obliki funkcije `titracija` z obveznimi argumenti `cb` (koncentracija baze), `Vb` (volumen baze), `Vdrop` (volumen kapljice), `ck` (koncentracija kisline) in `Vk` (volumen raztopine) ter opcijskim argumentom `izpis`, ki uporabniku omogoča izpisovanje pH na zaslon po vsaki dodani kapljici. Če uporabnik ne izbere izpisovanja pH, naj se na zaslonu izpiše samo končna točka titracije.

[11] Modul uvozite na začetku datoteke z ukazom `import random`. Lastnosti funkcije `random.choice()` poiščite v dokumentaciji: https://docs.python.org/3/library/random.html.

5.7 Naloge

5.7.1 Preproste funkcije

Vse funkcije iz tega sklopa pišite v datoteko `preproste_funkcije.py`.

Iskanje minimuma

Napišite funkcijo `minimum(seznam)`, ki poišče in vrne najmanjšo vrednost v podanem seznamu števil, brez da uporabite funkcijo `min()` ali sortiranje.

Primer pravilnega delovanja funkcije:

```
>>> s = [23, 42, 87, 34, 1, -3, 2]
>>> minimum(s)
-3
```

Trikotniška neenakost

Napišite funkcijo `trikotniska_neenakost(a, b, c)`, ki preveri, ali lahko s podanimi dolžinami stranic (a, b in c) tvorimo trikotnik. Trikotniška neenakost pravi, da trikotnik obstaja, če je vsota dolžin poljubnih dveh stranic večja od dolžine tretje stranice.

Primer pravilnega delovanja funkcije:

```
>>> trikotniska_neenakost(3, 4, 5)
True
>>> trikotniska_neenakost(2, 3, 8)
False
```

Samoglasniki

Napišite funkcijo `samoglasnik(s)`, ki prejme znak, tj. niz dolžine 1, in vrne `True`, če je znak samoglasnik, v nasprotnem primeru pa `False`.

Primera pravilnega delovanja funkcije:

```
>>> samoglasnik('a')
True
>>> samoglasnik('f')
False
```

pH

Napišite funkcijo `pH(koncentracija)`, ki prejme molarno koncentracijo oksonijevih ionov v raztopini, vrne pa vrednost pH, izračunano po formuli: $pH = -log_{10}(koncentracija)$.

Primer pravilnega delovanja funkcije:

```
>>> pH(0.0000001)
7.0
```

Izračun desetiškega logaritma omogoča funkcije `log()`, ki se nahaja v knjižnici `math`. Knjižnico na začetku datoteke uvozite z vrstico: `import math`.

Prekrivajoča se seznama

Napišite funkcijo `prekrivajoca_seznama(seznam1, seznam2)`, ki kot argumenta sprejme dva seznama in vrne `True`, če imata seznama vsaj en skupen element. V nasprotnem primeru naj funkcija vrača `False`.

Primera pravilnega delovanja funkcije:

```
>>> prekrivajoca_seznama([1, 2, 3], [4, 5, 6])
False
>>> prekrivajoca_seznama(['Ana', 'Peter', 'Miha'], ['Matej', 'Eva', 'Ana'])
True
```

Namig: najprej razmislite, kako ugotoviti, ali se nek podan element nahaja v seznamu.

5.7.2 Razcep na prafaktorje

Vse funkcije iz tega sklopa pišite v datoteko `razcep.py`.

Napisati želimo funkcijo `razcep_na_prafaktorje(n)`, ki vrača razcep števila na prafaktorje. Do celotne funkcije bomo prišli z uporabo več pomožnih funkcij.

Praštevilo

Napišite funkcijo `prastevilo(n)`, ki za vnešeno število vrne, ali je praštevilo[12].

Primera pravilnega delovanja funkcije:

```
>>> prastevilo(13)
True
>>> prastevilo(22)
False
```

[12] Število 1 ni praštevilo.

Praštevila

Napišite funkcijo `prastevila(n)`, ki vrne seznam vseh praštevil med `2` in `n`, vključno z `2` in `n`. Pri tem si pomagajte s funkcijo `prastevilo(n)` iz prejšnje naloge.

Primera pravilnega delovanja funkcije:

```
>>> prastevila(59)
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59]
>>> prastevila(42)
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
```

Deljivost

Napišite funkcijo `deljivost(n, x)`, ki vrne, kolikokrat je `n` deljiv z `x`.

Primeri pravilnega delovanja funkcije:

```
>>> deljivost(756, 2)
2
\begin{minted}[breaklines, bgcolor=gray_light]{text}
>>> deljivost(756, 3)
3
>>> deljivost(756, 5)
0
```

Namig: Dokler je `n` deljiv z `x`, ga delite z `x` in sproti štejte, kolikokrat ste ga delili. Ne pozabite zmanjševati `n`.

Razcep na prafaktorje

Napišite funkcijo `razcep_na_prafaktorje(n)`, ki razcepi število `n` na prafaktorje. Razcep na prafaktorje je zapis števila s produktom praštevil, npr. 252 lahko zapišemo s produktom $2^2 \cdot 3^2 \cdot 7^1$. Funkcija naj vrne seznam seznamov, ki predstavljajo osnovo in potenco. Pri pisanju si pomagajte s funkcijama `prastevila(n)` in `deljivost(n, x)`.

Primera pravilnega delovanja funkcije:

```
>>> razcep_na_prafaktorje(252)
[[2, 2], [3, 2], [7, 1]]
>>> razcep_na_prafaktorje(1944)
[[2, 3], [3, 5]]
```

Namig: Funkcija naj za vsa praštevila med `2` in `n` (vključno z `2` in `n`) ugotovi, kolikokrat delijo podano število in če ga delijo vsaj enkrat, to doda v seznam.

5.7.3 Fibonaccijevo zaporedje

Vse funkcije iz tega sklopa pišite v datoteko `fibonacci.py`.

Fibonaccijeva števila

Napišite funkcijo `fibonaccijeva_stevila(n)`, ki prejme število `n`, vrne pa seznam Fibonaccijevih števil manjših od `n`.

Primer pravilnega delovanja funkcije:

```
>>> fibonaccijeva_stevila(90)
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Liha Fibonaccijeva števila

Napišite funkcijo `liha_fibonaccijeva_stevila(n)`, ki prejme število `n`, vrne pa vsoto vseh lihih Fibonaccijevih števil manjših od `n`.

Primer pravilnega delovanja funkcije:

```
>>> liha_fibonaccijeva_stevila(90)
188
```

5.8 Rešitve

5.7.1 Preproste funkcije

Iskanje minimuma

```
def minimum(seznam):
    minimum = seznam[0]
    for elt in seznam:
        if elt < minimum:
            minimum = elt
    return minimum
```

Trikotniška neenakost

```
def trikotniska_neenakost(a, b, c):
    if a + b > c and a + c > b and b + c > a:
        return True
    return False
```

Samoglasniki

```
def samoglasnik(s):
    if s.lower() in ['a', 'e', 'i', 'o', 'u']:
        return True
    else:
        return False
```

pH

```
from math import log10

def pH(koncentracija):
    return -log10(koncentracija)
```

Prekrivajoča se seznama

```
def prekrivajoca_seznama(seznam1, seznam2):
    for elt in seznam1:
        if elt in seznam2:
            return True
    return False
```

5.7.2 Razcep na prafaktorje

Praštevilo

```
def prastevilo(n):
    if n == 1:
        return False
    prastevilo = True
    for j in range(2, n//2 + 1):
        if n % j == 0:
            prastevilo = False
            return prastevilo
    return prastevilo
```

Praštevila

Pomagamo si s funkcijo prestevilo(n) iz prejšnje naloge.

```
def prastevila(n):
    seznam_prastevil = []
    for i in range(2, n + 1):
        if prastevilo(i):
            seznam_prastevil += [i]
    return seznam_prastevil
```

Deljivost

```
def deljivost(n, x):
    stevec = 0
    while n % x == 0:
        n /= x
        stevec += 1
    return stevec
```

Razcep na prafaktorje

Pomagamo si s funkcijama prastevila(n) in deljivost(n, x).

```
def razcep_na_prafaktorje(n):
    seznam = []
    seznam_prastevil = prastevila(n)
    for prastevilo in seznam_prastevil:
        deli = deljivost(n, prastevilo)
        if deli > 0:
            seznam += [[prastevilo, deli]]
    return seznam
```

5.7.3 Fibonaccijevo zaporedje

Fibonaccijeva števila

```
def fibonaccijeva_stevila(n):
    f1 = 1
    f2 = 1
    stevila = [1, 1]
    while f1 + f2 < n:
        stevila += [f1 + f2]
        f2, f1 = f1, f1 + f2
    return stevila
```

Liha Fibonaccijeva števila

```
def liha_fibonaccijeva_stevila(n):
    stevila = fibonaccijeva_stevila(n)
    suma = 0
    for st in stevila:
        if st % 2 == 1:
            suma += st
    return suma
```

6. vaje: (Ne)spremenljivi podatkovni tipi in terke

6.1 Spremenljivost podatkovnih tipov

Podatkovni tipi v Pythonu so lahko nespremenljivi ali spremenljivi. Med nespremenljive štejemo cela števila (`int`), realna števila (`float`), nize (`str`), Boolove izraze (`bool`) in terke (`tuple`). Spremenljivi podatkovni tipi so seznami (`list`), množice (`set`) in slovarji (`dict`).

6.1.1 Spremenljivost seznamov

Spremeniti želimo seznam `s = ['a', 'b', 'c', 'd']` tako, da je niz `'a'` zapisan z veliko tiskano črko.

Če poskušamo spremembo uveljaviti s spodnjo kodo:

```
for crka in s:
    if crka == 'a':
        crka = crka.upper()
print(s)
```

dobimo:

```
['a', 'b', 'c', 'd']
```

Seznama s tem načinom torej nismo uspeli spremeniti, saj nismo dobro označili, da želimo zamenjati element na točno določenem mestu. Oznako mesta elementa lahko dosežemo z uporabo indeksiranja:

```
for i in range(len(s)):
    if s[i] == 'a':
        s[i] = s[i].upper()
print(s)
```

in dobimo:

```
['A', 'b', 'c', 'd']
```

6.1.2 Spremenljivost pri funkcijah

(Ne)spremenljivost podatkovnih tipov je pomembna pri funkcijah - spodnja funkcija `spremeni` na zaslonu izpiše, kaj se zgodi s spremenljivko `x` znotraj funkcije:

```
def spremeni(x,y):
    x += y
    print("Objekt v funkciji:", x)
```

Poglejmo si, kaj se dogaja z različnimi podatkovnimi tipi znotraj in izven funkcije - za spremljanje slednjega bomo poleg zgornje funkcije uporabili še `print("Objekt izven funkcije:", x)`:

1. **spreminjanje seznama:**

```
>>> x = [1,2,3]
>>> y = [4,5]
>>> spremeni(x,y)
Objekt v funkciji: [1, 2, 3, 4, 5]
>>> print("Objekt izven funkcije:", x)
Objekt izven funkcije: [1, 2, 3, 4, 5]
```

2. **spreminjanje terke:**

```
>>> x = (1,2,3)
>>> y = (4,5)
>>> spremeni(x,y)
Objekt v funkciji: (1,2,3,4,5)
>>> print("Objekt izven funkcije:", x)
Objekt izven funkcije: (1,2,3)
```

3. **spreminjanje števila (`int`):**

```
>>> x = 1
>>> y = 2
>>> spremeni(x,y)
Objekt v funkciji: 3
>>> print("Objekt izven funkcije:", x)
Objekt izven funkcije: 1
```

4. **spreminjanje niza:**

```
>>> x = 'a'
>>> y = 'b'
>>> spremeni(x,y)
Objekt v funkciji: ab
>>> print("Objekt izven funkcije:", x)
Objekt izven funkcije: a
```

V vseh primerih se znotraj funkcije objekt spremeni, izven funkcije pa spremembo ohrani le, če gre za seznam.

6.2 Terke

Terke so nespremenljiv podatkovni tip, ki je zelo uporaben pri shranjevanju vrednosti, za katere ne želimo, da se tekom izvajanja programa hote ali nehote spreminjajo.

Obravnavali bomo seznam terk oblike: `(spojina, molska_masa, SMILES, temp_t)`, pri čemer je `spojina` niz z imenom spojine, `molska_masa` realno število, `SMILES` niz s formulo spojine in `temp_t` temperatura tališča kot celo število.

```
1 spojine = [("H2O", 18.01528, "O", 0),
2             ("CO2", 44.01, "C(=O)=O", -57),
3             ("NaCl", 58.44, "[Na+].[Cl-]", 801),
4             ("CH4", 16.042, "C", -182),
5             ("NH3", 17.031, "N", -78)]
```

Kadar operiramo s seznamom terk, moramo biti pozorni na indeksiranje in razpakiranje terk. V 1. vrstici spodnje kode z indeksiranjem seznama dostopamo do celotne terke na izbranem indeksu. Če želimo dostopati le do določenega elementa znotraj terke, moramo v tem primeru uporabiti dvojno indeksiranje - prvi indeks označuje element seznama, drugi indeks pa element znotraj terke (glej 3. vrstico v spodnji kodi).

```
1 >>> spojine[2]
2 ("NaCl", 58.44, "[Na+].[Cl-]", 801)
3 >>> spojine[2][0]
4 "NaCl"
```

Terko lahko razpakiramo, pri čemer vsakemu elementu terke priredimo ime. To lahko storimo le za terke, za katere vemo točno število elementov.

```
1 >>> snov = spojine[4]
2 >>> snov
3 ("NH3", 17.031, "N", -78)
4 >>> spojina, molska_masa, SMILES, temp_t = snov
5 >>> print(f'Spojina {spojina} (SMILES: {SMILES}) ima molsko maso {molska_masa}
  ↪  g/mol in temperaturo tališča {temp_t} °C.')
6 Spojina NH3 (SMILES: N) ima molsko maso 17.031 in temperaturo tališča -78 °C.
```

Razpakiranje terk (glej 4. vrstico zgornje kode) je še posebej uporabno v kombinaciji z zanko *for*, ko želimo dostopati do elementov v vseh terkah. To lahko storimo tako, da se z zanko *for* sprehajamo po elementih seznama, tj. terkah, ki jih znotraj zanke razpakiramo (glej 3. vrstico v spodnji kodi):

```
low_temp_t = []
for snov in spojine:
    spojina, molska_masa, SMILES, temp_t = snov
    if temp_t < 0:
        low_temp_t.append(spojina)
```

ali pa razpakiranje opravimo že v glavi zanke *for*:

```
formule = []
for spojina, molska_masa, SMILES, temp_t in spojine:
    if len(SMILES) > 1:
        formule.append(SMILES)
```

Če nas kateri od elementov terke ne zanima, lahko ime zamenjamo s podčrtajem:

```
imena = []
for spojina,_,_,_ in seznam_full:
    imena.append(spojina)
```

Zgornja načina z uporabo zanke *for* bosta delovala le, če imajo vse terke v seznamu enako število elementov.

6.2.1 Funkcija `zip()`

S funkcijo `zip()` lahko združimo več enako dolgih seznamov v terke, ki vsebujejo elemente, ki se na različnih seznamih nahajajo na istem indeksu. Če imamo seznama `s1 = [1, 2, 3]` in `s2 = ['a', 'b', 'c']` in ju združimo s funkcijo `zip(s1,s2)`, bo ustvarjen generatorski objekt s terkami `(1, 'a'), (2, 'b'), (3, 'c')`. Generatorski objekt lahko direktno uporabimo v zanki *for*, tj. `for i,j in zip(s1, s2)`, za shranjevanje terk pa moramo objekt pretvoriti v seznam s funkcijo `list()`, tj. `list(zip(s1,s2))` (glej 6. vrstico v spodnji kodi).

```
imena = ["H2O","CO2","NaCl","CH4","NH3"]
molske_mase = [18.01528, 44.01, 58.44, 16.042, 17.031]
SMILES = ["O","C(=O)=O","[Na+].[Cl-]","C","N","OC(=O)CC(O)(C(=O)O)CC(=O)O"]
temp_tal = [0, -57, 801, -182, -78, 156]

seznam = list(zip(imena, molske_mase, SMILES, temp_tal))
print(seznam)
```

Zgornji program izpiše:

```
[('H2O', 18.01528, 'O', 0), ('CO2', 44.01, 'C(=O)=O', -57), ('NaCl', 58.44,
↪  '[Na+].[Cl-]', 801), ('CH4', 16.042, 'C', -182), ('NH3', 17.031, 'N', -78)]
```

6.2.2 Naloge za utrjevanje

1. Napišite funkcijo `potenciranje`, ki kot argumenta sprejme seznam števil in število. Vrača naj seznam terk oblike: `(element_seznama, (element_seznama)**število)`.
2. Napišite funkcijo, ki iz seznama terk oblike: `(spojina, molska_masa, SMILES, temp_t)` izpiše tiste spojine, ki imajo molsko maso višjo od 50 g/mol.
3. Funkcijo `titracija`, ki ste jo napisali v poglavju 5.4, preuredite tako, da vrača dva seznama - v prvem seznamu naj bo količina dodane baze v vsaki točki, v drugem seznamu pa pripadajoč pH. Kateri podatkovni tip vrne funkcija, ki vrača več kot eno spremenljivko?

6.3 Naloge

6.3.1 Spremenljivost

Vse funkcije tega sklopa pišite v datoteko `spremenljivost.py`

Korenjenje v seznamu (1)

Napišite funkcijo `koreni_seznam(seznam)`, ki prejme seznam in spremeni vse vrednosti v seznamu v njihove korene. Če je vrednost pod korenom negativna, jo v seznamu prepiše v `None`.

Primera pravilnega delovanja funkcije:

```
>>> s = [0, 1, 4, 9, 16, 25]
>>> koreni_seznam(s)
>>> s
[0.0, 1.0, 2.0, 3.0, 4.0, 5.0]
>>> s1 = [400, -100, 256, -1089]
>>> koreni_seznam(s1)
>>> s1
[20.0, None, 16.0, None]
```

Korenjenje v seznamu (2)

Napišite funkcijo `koreni_seznam2(seznam)`, ki prejme seznam in vrne seznam korenov vrednosti podanega seznama. Če je vrednost pod korenom negativna, naj doda v seznam vrednost `None`.

1. primer uporabe:

```
>>> s = [0, 1, 4, 9, 16, 25]
>>> s2 = koreni_seznam2(s)
>>> s2
[0.0, 1.0, 2.0, 3.0, 4.0, 5.0]
>>> s
[0, 1, 4, 9, 16, 25]
>>> s3 = [400, -100, 256, -1089]
>>> s4 = koreni_seznam2(s3)
>>> s4
[20.0, None, 16.0, None]
```

Unikaten seznam

Napišite funkcijo `unikaten_seznam(seznam)`, ki prejme seznam elementov, vrne pa seznam, ki vsebuje elemente prvega seznama samo po enkrat. Vrstni red elementov naj bo isti, kot je vrstni red prve pojavitve v podanem seznamu.

Primeri pravilnega delovanja funkcije:

```
>>> unikaten_seznam([1, 2, 3, 3, 3, 3, 4, 5])
[1, 2, 3, 4, 5]
>>> unikaten_seznam([1, 2, 3, 4, 5, 1, 3, 5])
[1, 2, 3, 4, 5]
>>> unikaten_seznam([1, 2, 3, 4, 5])
[1, 2, 3, 4, 5]
```

6.3.2 Terke

Vse funkcije tega sklopa pišite v datoteko `IMDb.py`.

Podan je seznam serij, ki poleg imena serije vsebuje tudi njeno oceno na IMDb-ju ter leto začetka predvajanja:

```
serije = [('Friends', 9.0, 1994),
    ('The Big Bang Theory', 8.4, 2007),
    ('Game of Thrones', 9.5, 2011),
    ('Mr. Robot', 8.7, 2015),
    ('Humans', 8.1, 2015)]
```

Ocene

Napišite funkcijo `ocene(serije)`, ki sprejme seznam `serije` in vrne seznam naslovov serij, ki imajo oceno večjo ali enako 9.

Primer pravilnega delovanja funkcije:

```
>>> ocene(serije)
['Friends', 'Game of Thrones']
```

Naslove vrnite v istem vrstnem redu, kot so podatki v podanem seznamu.

Najstarejša serija

Napišite funkcijo `najstarejsa(serije)`, ki vrne ime najstarejše serije. Predpostavite, da so vsi testni podatki taki, da je najstarejša serija samo ena.

Primer pravilnega delovanja funkcije:

```
>>> najstarejsa(serije)
'Friends'
```

Povprečna ocena

Napišite funkcijo `povprecna_ocena(serije)`, ki vrne povprečno oceno vseh serij v seznamu.

Primer pravilnega delovanja funkcije:

```
>>> povprecna_ocena(serije)
8.74
```

Dolga imena serij

Napišite funkcijo `dolga_imena(serije)`, ki vrne seznam imen serij, ki so daljša od dveh besed.

Primer pravilnega delovanja funkcije:

```
>>> dolga_imena(serije)
['The Big Bang Theory', 'Game of Thrones']
```

Namig: pomagajte si z metodo `split()`, ki niz razbije po presledkih na seznam nizov.

Primer delovanja metode `split()`:

```
>>> niz = 'The Big Bang Theory'
>>> niz.split()
['The', 'Big', 'Bang', 'Theory']
```

Krajšanje

Napišite funkcijo `krajsanje(serije)`, ki vrne seznam terk oblike: `(naslov, ocena)`.

Primer pravilnega delovanja funkcije:

```
>>> krajsanje(serije)
[('Friends', 9.0), ('The Big Bang Theory', 8.4), ('Game of Thrones', 9.5),
↪  ('Mr. Robot', 8.7), ('Humans', 8.1)]
```

Najnovejše serije

Napišite funkcijo `najnovejse(serije)`, ki vrne seznam imen najnovejših serij iz podanega seznama serij. Naslovi serij naj bodo v istem vrstnem redu kot v seznamu serij.

Primer pravilnega delovanja funkcije:

```
>>> najnovejse(serije)
['Mr. Robot', 'Humans']
```

6.4 Rešitve

6.3.1 Spremenljivost

Korenjenje v seznamu (1)

```
def koreni_seznam(seznam):
    for i in range(len(seznam)):
        if seznam[i] >= 0:
            seznam[i] = seznam[i] ** 0.5
        else:
            seznam[i] = None
```

Korenjenje v seznamu (2)

```
def koreni_seznam2(seznam):
    s=[]
    for elt in seznam:
        if elt >= 0:
            s.append(elt ** 0.5)
        else:
            s.append(None)
    return s
```

Unikaten seznam

```
def unikaten_seznam(s):
    unikati = []
    for elt in s:
        if elt not in unikati:
            unikati += [elt]
    return unikati
```

6.3.2 Terke

Ocene

```
def ocene(serije):
    seznam = []
    for naslov, ocena, leto in serije:
        if ocena >= 9.0:
            seznam.append(naslov)
    return seznam
```

Najstarejša serija

```
def najstarejsa(serije):
    naj_naslov, naj_ocena, naj_leto =serije[0]
    for naslov, ocena, leto in serije:
        if leto < naj_leto:
            naj_naslov = naslov
            naj_leto = leto
    return naj_naslov
```

Povprečna ocena

```
def povprecna_ocena(serije):
    if len(serije) == 0:
        return 0
    suma = 0
    for naslov, ocena, leto in serije:
        suma += ocena
    return suma / len(serije)
```

Dolga imena serij

```
def dolga_imena(serije):
    seznam = []
    for naslov, ocena, leto in serije:
        if len(naslov.split()) > 2:
            seznam.append(naslov)
    return seznam
```

Krajšanje

```
def krajsanje(serije):
    seznam = []
    for naslov, ocena, leto in serije:
        seznam.append((naslov, ocena))
    return seznam
```

Najnovejše serije

```
def najnovejse(serije):
    naj_naslov = []
    if len(serije) == 0:
        return naj_naslov

    naj_leto =serije[0][2]
    for serija in serije:
        if serija[2] > naj_leto:
            naj_leto = serija[2]

    for naslov, ocena, leto in serije:
        if leto == naj_leto:
            naj_naslov.append(naslov)

    return naj_naslov
```

7. vaje: Slovarji

Slovarji so podatkovni tip oblike `ključ:vrednost`. Vrednosti, spravljene pod določenim ključem, lahko hitro pokličemo s sintakso `slovar[ključ]` - podobno kot pri indeksiranju seznamov, le da indekse tukaj nadomeščajo ključi, ki so največkrat nizi ali cela števila. Ključi omogočajo zelo hiter dostop do želene vrednosti, saj nam za razliko od seznamov ni treba poznati indeksa ali iterirati preko celotnega seznama.

Za primer vzemimo slovar, kjer so ključi posamezne aminokisline, vrednosti pa njihove pripadajoče molske mase:

```
slovar_ak = {'A':71.03711,'C':103.00919,'D':115.02694,
             'E':129.04259,'F':147.06841,'G':57.02146,
             'H':137.05891,'I':113.08406,'K':128.09496,
             'L':113.08406,'M':131.04049,'N':114.04293,
             'P':97.05276,'Q':128.05858,'R':156.10111,
             'S':87.03203,'T':101.04768,'V':99.06841,
             'W':186.07931}
```

Dodajanje novega para `ključ:vrednost` v slovar izvedemo tako, da imenu slovarja dodamo "indeks" s ključem in mu priredimo neko vrednost: `slovar_ak['Y'] = 163`. Na ta način lahko tudi popravimo oz. spremenimo vrednost pod obstoječim ključem: `slovar_ak['Y'] = 163.06333`.

Takšen slovar lahko uporabimo pri izračunu molske mase proteina. Spremenljivko `molska_masa` pred začetkom zanke *for* nastavimo na 0, nato pa se z zanko *for* pomikamo po nizu in spremenljivki `molska_masa` prištevamo molsko maso trenutne aminokisline.

```
protein = 'VYFSATMRVLHASLFPQNMNRKRGNCRNNYYDVKMKQISHLKLMKFDFMNHNSKYVRHTLCKVEEHY
↪  YSEQESTKYIPRTWISLKDCINSFVYPRCDCCH'

molska_masa = 0

for aminokislina in protein:
    molska_masa += slovar_ak[aminokislina]

print(round(molska_masa,2))
```

Kadar nas zanima, ali je v slovarju prisoten nek ključ, lahko to preverimo na preprost način:

```
>>> 'W' in slovar_ak
True
```

Enako pa žal ne velja za preverjanje vrednosti. Če uporabimo enako sintakso kot zgoraj, le da ključ zamenjamo s pripadajočo vrednostjo, bo Python zanikal obstoj tega števila:

```
>>> 186.07931 in slovar_ak
False
```

To se zgodi, ker v privzetem načinu Python "vidi" le ključe, ne pa tudi pripadajočih vrednosti. Če bi želeli preveriti, ali neka vrednost obstaja v slovarju, bi to morali storiti z zanko *for* ali uporabiti eno od metod, opisanih v naslednjem poglavju.

```
for key in slovar_ak:
    if slovar_ak[key] == 186.07931:
        print(True)
        break
```

7.1 Metode `.keys()`, `.values()` in `.items()`

Metode, opisane v tej sekciji, so namenjene izključno uporabi s slovarji.

Seznam vseh ključev v slovarju dobimo z metodo `.keys()`.

```
>>> slovar_ak.keys()
dict_keys(['A', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'K', 'L', 'M', 'N', 'P',
↪  'Q', 'R', 'S', 'T', 'V', 'W', 'Y'])
```

Z zanko *for* se lahko po ključih pomikamo tudi brez uporabe metode `.keys()`, saj po privzeti nastavitvi Python iterira le po ključih.

```
>>> s = []
>>> for i in slovar_ak:
...     s.append(i)
>>> print(s)
['A', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'K', 'L', 'M', 'N', 'P', 'Q', 'R',
↪  'S', 'T', 'V', 'W', 'Y']
```

Metoda `.values()` vrača seznam vseh vrednosti slovarja. Pri tem pazite, saj vrednosti niso omejene na števila, temveč so lahko tudi slovarji, seznami, terke in nizi (glejte podpoglavje 7.3). V našem primeru ta metoda vrne sledeče:

```
>>> slovar_ak.values()
dict_values([71.03711, 103.00919, 115.02694, 129.04259, 147.06841, 57.02146,
↪  137.05891, 113.08406, 128.09496, 113.08406, 131.04049, 114.04293, 97.05276,
↪  128.05858, 156.10111, 87.03203, 101.04768, 99.06841, 186.07931, 163.06333])
```

Če so vse vrednosti v slovarju številske (bodisi cela ali decimalna števila), kakor v našem primeru `slovar_ak`, lahko to metodo uporabimo za računanje povprečja:

```
1 povprecje = sum(slovar_ak.values())/len(slovar_ak.values())
```

Metoda `.items()` vrne seznam terk oblike `(ključ, vrednost)`.

```
>>> slovar_ak.items()
dict_items([('A', 71.03711), ('C', 103.00919), ('D', 115.02694), ('E',
↪  129.04259), ('F', 147.06841), ('G', 57.02146), ('H', 137.05891), ('I',
↪  113.08406), ('K', 128.09496), ('L', 113.08406), ('M', 131.04049), ('N',
↪  114.04293), ('P', 97.05276), ('Q', 128.05858), ('R', 156.10111), ('S',
↪  87.03203), ('T', 101.04768), ('V', 99.06841), ('W', 186.07931)), ('Y',
↪  163.06333)]
```

Seznam terk je uporaben, kadar želimo dostopati tudi do ključa, povezanega z neko vrednostjo. S spodnjim programom tako iščemo ključe, katerih vrednosti presegajo 100:

```
1 kljuci = []
2 for kljuc, vrednost in slovar_ak.items():
3     if vrednost > 100:
4         kljuci.append(kljuc)
5 print(kljuci)
```

Zgornji program vrne:

```
['C', 'D', 'E', 'F', 'H', 'I', 'K', 'L', 'M', 'N', 'Q', 'R', 'T', 'W', 'Y']
```

česar brez uporabe metode `.items()` ne bi mogli izvesti oz. bi bila izvedba precej zahtevnejša.

7.2 Seznami in množice kot vrednosti v slovarjih

Poglejmo si primer priprave slovarja, kjer bodo ključi celine, vrednosti pa seznami terk oblike `(drzava, prestolnica)`. Slovar bomo pripravili iz seznama terk:

```
seznam = [('Evropa', 'Francija', 'Pariz'), ('Evropa', 'Nemčija', 'Berlin'),
↪  ('Azija', 'Japonska', 'Tokio'), ('Avstralija', 'Avstralija', 'Canberra'),
↪  ('Azija', 'Kitajska', 'Peking'), ('Afrika', 'Egipt', 'Kairo'), ('Afrika',
↪  'Nigerija', 'Abuja'), ('Severna Amerika', 'ZDA', 'Washington D.C.'),
↪  ('Južna Amerika', 'Brazilija', 'Brasilia')]
```

Ob pregledu seznama terk vidimo, da se nekateri kontinenti ponavljajo. Ker bodo vrednosti seznami terk, moramo pri pripravi slovarja v zanki preveriti, ali ključ (kontinent) že obstaja v slovarju (4. vrstica). Če ključ še ne obstaja, ustvarimo prazen seznam kot vrednost (5. vrstica) in vanj postopoma dodajamo nove terke (država, prestolnica).

```
slovar = {}

for kontinent, drzava, prestolnica in seznam:
    if kontinent not in slovar:
        slovar[kontinent] = []
    slovar[kontinent].append((drzava,prestolnica))
```

Pripravljen slovar:

```
{'Evropa': [('Francija', 'Pariz'), ('Nemčija', 'Berlin')], 'Azija':
↪  [('Japonska', 'Tokio'), ('Kitajska', 'Peking')], 'Avstralija':
↪  [('Avstralija', 'Canberra')], 'Afrika': [('Egipt', 'Kairo'), ('Nigerija',
↪  'Abuja')], 'Severna Amerika': [('ZDA', 'Washington D.C.')], 'Južna
↪  Amerika': [('Brazilija', 'Brasilia')]}
```

Če bi izpustili preverjanje v 4. vrstici, bi s kodo `slovar[kontinent] = [(drzava, prestolnica)]` ob vsakem prehodu zanke prepisali obstoječo vsebino in v slovarju ohranili le zadnjo terko vsake celine.

7.3 Slovar slovarjev

Vrednosti v slovarju so lahko števila, nizi, seznami, terke in celo slovarji. Če imamo opravka z "zahtevnejšimi" vrstami slovarjev, ki med vrednostmi vsebujejo zelo različne podatkovne tipe, je postopno indeksiranje najboljša strategija dela z njimi.

Kot primer bomo obravnavali skrajšan slovar slovarjev `star_wars_short`:

```
star_wars_short = {'Luke Skywalker': {'characteristics': ('Human', 'male', '172.0',
↪  '77.0'),
  'homeworld': 'Tatooine',
  'films': ['The Empire Strikes Back',
   'Revenge of the Sith',
   'Return of the Jedi',
   'A New Hope',
   'The Force Awakens']},
 'R2-D2': {'characteristics': ('Droid', 'none', '96.0', '32.0'),
  'homeworld': 'Naboo',
  'films': ['The Empire Strikes Back',
   'Attack of the Clones',
   'The Phantom Menace',
   'Revenge of the Sith',
   'Return of the Jedi',
   'A New Hope',
   'The Force Awakens']},
```

... se nadaljuje na naslednji strani.

```
'Darth Vader': {'characteristics': ('Human', 'male', '202.0', '136.0'),
 'homeworld': 'Tatooine',
 'films': ['The Empire Strikes Back',
  'Revenge of the Sith',
  'Return of the Jedi',
  'A New Hope']},
'Leia Organa': {'characteristics': ('Human', 'female', '150.0', '49.0'),
 'homeworld': 'Alderaan',
 'films': ['The Empire Strikes Back',
  'Revenge of the Sith',
  'Return of the Jedi',
  'A New Hope',
  'The Force Awakens']},
'Obi-Wan Kenobi': {'characteristics': ('Human', 'male', '182.0', '77.0'),
 'homeworld': 'Stewjon',
 'films': ['The Empire Strikes Back',
  'Attack of the Clones',
  'The Phantom Menace',
  'Revenge of the Sith',
  'Return of the Jedi',
  'A New Hope']},
'Anakin Skywalker': {'characteristics': ('Human', 'male', '188.0', '84.0'),
 'homeworld': 'Tatooine',
 'films': ['Attack of the Clones',
  'The Phantom Menace',
  'Revenge of the Sith']},
'Chewbacca': {'characteristics': ('Wookiee', 'male', '228.0', '112.0'),
 'homeworld': 'Kashyyyk',
 'films': ['The Empire Strikes Back',
  'Revenge of the Sith',
  'Return of the Jedi',
  'A New Hope',
  'The Force Awakens']}}
```

Po hitrem pregledu tega slovarja opazimo, da imamo pod vsakim ključem - tj. imenom lika -, kot vrednost nov slovar, ki vsebuje ključe `characteristics`, `homeworld` in `films`. Vrednosti pod ključem `characteristics` so zapakirane v terko, pod `homeworld` imamo niz, pod `films` pa seznam.

Razumevanja strukture slovarja se lotimo postopoma - najprej si oglejmo, kaj dobimo, če uporabimo prvi ključ:

```
star_wars_short['Anakin Skywalker']
```

Dobimo slovar, ki ima 4 ključe:

```
{'characteristics': ('Human', 'male', '188.0', '84.0'), 'homeworld':
↪  'Tatooine', 'films': ['Attack of the Clones', 'The Phantom Menace',
↪  'Revenge of the Sith']}
```

Če nas zanima npr. višina lika, potrebujemo podatek iz terke, ki se nahaja pod prvim ključem. Do te terke lahko dostopamo z dvojnim indeksiranjem - prvi indeks (ime lika) prikliče celoten slovar, nato pa iz tega slovarja terko prikličemo z uporabo ključa `'characteristics'`:

```
star_wars_short['Anakin Skywalker']['characteristics']
```

Zagon zgornje kode vrne:

```
('Human', 'male', '188.0', '84.0')
```

Do podatka o višini lika lahko tako dostopamo z razpakiranjem terke:

```
species, gender, height, weight = star_wars['Anakin
↪  Skywalker']['characteristics']
```

Na podoben način dostopamo do drugih podatkov v slovarju slovarjev. Če bi želeli izpisati prvi film v seznamu, ki se nahaja pod ključem `'films'`, bi uporabili sledečo kodo:

```
star_wars['Anakin Skywalker']['films'][0]
```

> *Kadar operirate s kombinacijami podatkovnih tipov, kot so seznami seznamov, slovarji slovarjev ipd., vedno rešujte situacijo postopoma in sproti preverjajte, ali vaši indeksi res vrnejo to, kar pričakujete.*

7.4 *Hitra izpeljava slovarja (angl. *dictionary comprehension*)

Podobno kot pri seznamih obstaja hiter način izpeljave slovarja (angl. *dictionary comprehension*). Pogledali si bomo izpeljavo sledečega slovarja molskih mas nukleotidov:

```
{'A': 135.13, 'T': 126.11, 'G': 151.13, 'C': 111.1, 'U': 112.09}
```

Slovar bomo izpeljali iz dveh seznamov, ki imata na istih indeksih podatke, ki spadajo skupaj. Obenem si pripravimo še prazen slovar, v katerega bomo shranjevali ključe in vrednosti:

```
nukleotidi = ['A','T','G','C','U']
molske_mase = [135.13, 126.11, 151.13, 111.10, 112.09]
slovar_nt = {}
```

Brez uporabe hitrega načina lahko slovar tvorimo z uporabo indeksiranja ali pa z uporabo funkcije `zip()`:

1. ```
   for i in range(0,5):
       slovar_nt[nukleotidi[i]] = molske_mase[i]
   ```

2. ```
   for nukleotid, masa in zip(nukleotidi, molske_mase):
       slovar_nt[nukleotid] = masa
   ```

Hitri način sledi sintaksi `slovar = {kljuc:vrednost for kljuc, vrednost in x}`, pri čemer je `x` seznam, `zip(seznam)` ali katera druga podatkovna oblika, ki služi kot osnova slovarja. Pri tem ne rabimo predhodno pripraviti praznega slovarja, saj se bo tvoril avtomatsko.

Hiter način lahko uporabimo tako z indeksiranjem kot z uporabo funkcije `zip()`:

1. ```
   slovar_nt = {nukleotidi[i]:molske_mase[i] for i in range(0,5)}
   ```

2. ```
   slovar_nt = {nukleotid:masa for nukleotid, masa in zip(nukleotidi,
   ↪  molske_mase)}
   ```

7.5 Dodatne naloge

Vojna zvezd

V podpoglavju 7.3 smo si pogledali del slovarja `star_wars`. Naloge 1-3 se nanašajo na uporabo tega slovarja, ki je v celoti dostopen na repozitorju GitHub (https://github.com/ahaluzan/OP_FKKT) v mapi `Dodatna_gradiva`.

1. Napišite funkcijo `vrste`, ki kot argument sprejme slovar likov. Funkcija naj vrne slovar, kjer ključe predstavljajo vrste (`'species'`), vrednosti pa imena likov, ki pripadajo tisti vrsti.
2. Napišite funkcijo, ki kot argument sprejme slovar likov, vrne pa seznam terk oblike `(planet, stevilo_likov)`, pri čemer `stevilo_likov` predstavlja število likov, ki prihajajo z istega planeta.
3. Napišite funkcijo, ki kot argument sprejme slovar likov, vrne pa slovar, kjer ključe predstavljajo imena filmov, vrednosti pa seznam likov, ki se pojavijo v danem filmu. Pri tem bodite pozorni, da številni liki nastopajo v več filmih, pri med vrednostmi novega seznama pa se lahko njihovo ime pojavi le enkrat.

Izračun pufrov

4. Napišite funkcijo `izracun_pufrov`, ki kot argumenta sprejme ime pufra, zapisano kot niz, in želen volumen pufra v litrih. Funkcija naj vrne seznam terk z ustreznimi količinami sestavin, ki tvorijo želen pufer v podanem volumnu. Slovar pufrov, kjer so navedene sestavine v gramih za 1 L posameznega pufra, je:

```
pufri = {"fosfatni": {"NaH2PO4_H2O": 13.8, "Na2HPO4_7H2O": 35.6},
"TRIS": {"Tris base": 12.1, "HCl": 8.5},
"acetatni": {"CH3COOH": 60.0, "CH3COONa_3H2O": 82.0}}
```

Primer klica funkcije:

```
>>> izracun_pufrov('TRIS', 0.5)
NaH2PO4_H2O ... 6.9 g,
Na2HPO4_7H2O ... 17.8 g
```

7.6 Naloge

Funkcije shranite v datoteko `trgovina.py`.

Podan je seznam izdelkov v trgovini, ki poleg imena izdelka vsebuje tudi njegovo ceno in zalogo:

```
izdelki = [('mleko', 0.86, 128),
    ('jogurt', 0.49, 56),
    ('piškoti', 2.99, 73),
    ('sok', 1.79, 104),
    ('jajce', 0.1, 103),
    ('moka', 1.39, 99),
    ('makaroni', 1.89, 67),
    ('paradižnik', 0.23, 35),
    ('kruh', 2.19, 43),
    ('hrenovka', 1.99, 28),
    ('gorgonzola', 2.69, 32)]
```

7.6.1 V slovar

Napišite funkcijo `v_slovar`, ki kot argument sprejme seznam `izdelki`. Funkcija naj vrača slovar, kjer kot ključi nastopajo imena izdelkov, vrednosti pa so seznami oblike `[cena, zaloga]`.

Primer pravilnega delovanja funkcije:

```
>>> v_slovar(izdelki)
{'mleko': [0.86, 128], 'hrenovka': [1.99, 28], 'jogurt': [0.49, 56], 'sok':
↪  [1.79, 104], 'moka': [1.39, 99], 'paradižnik': [0.23, 35], 'gorgonzola':
↪  [2.69, 32], 'makaroni': [1.89, 67], 'kruh': [2.19, 43], 'piškoti': [2.99,
↪  73], 'jajce': [0.1, 103]}
```

7.6.2 Število izdelkov

Napišite funkcijo `stevilo_izdelkov(zaloga)`, ki kot argument sprejme slovar, kot ga vrača funkcija `v_slovar` (tj. slovar `zaloga`). Funkcija naj vrne število vseh izdelkov v trgovini.

Primer pravilnega delovanja funkcije:

```
>>> zaloga = v_slovar(izdelki)
>>> stevilo_izdelkov(zaloga)
768
```

7.6.3 Nakupovalna košara

Napišite funkcijo `nakupovalna_kosara`, ki kot argument prejme seznam vsebine nakupovalne košare. Funkcija naj vrača slovar, kjer so ključi imena izdelkov, vrednosti pa število izdelkov.

Primer pravilnega delovanja funkcije:

```
>>> seznam = ['jogurt', 'mleko', 'jogurt', 'jogurt', 'piškoti']
>>> nakupovalna_kosara(seznam)
{'jogurt': 3, 'mleko': 1, 'piškoti': 1}
```

7.6.4 Cena

Napišite funkcijo `cena`, ki sprejme dva argumenta: slovar zaloge, kot ga vrača funkcija `v_slovar` (tj. slovar `zaloga`) in slovar nakupov, kot ga vrača funkcija `nakupovalna_kosara`. Funkcija `cena` naj vrne znesek, ki ga mora plačati kupec.

Primer pravilnega delovanja funkcije:

```
>>> zaloga = v_slovar(izdelki)
>>> seznam = ['jogurt', 'mleko', 'jogurt', 'jogurt', 'piškoti']
>>> slovar_nakupov = nakupovalna_kosara(seznam)
>>> cena(zaloga, slovar_nakupov)
5.32
```

7.6.5 Popravek zaloge

Napišite funkcijo `popravek_zaloge(zaloga, izdelek, popravek)`, ki spremeni slovar `zaloga` tako, da zalogo izdelka, podanega v argumentu `izdelek`, spremeni glede na argument `popravek` in vrne novo vrednost zaloge. Popravek je lahko negativen ali pozitiven. Če bi popravek spremenil zalogo na negativno, naj funkcija zaloge ne spremeni in vrne `None`.

Primera pravilnega delovanja funkcije:

```
>>> zaloga = v_slovar(izdelki)
>>> popravek_zaloge(zaloga, 'sok', 100)
204
>>> zaloga
{'mleko': [0.86, 128], 'hrenovka': [1.99, 28], 'moka': [1.39, 99], 'sok':
↪  [1.79, 204], 'makaroni': [1.89, 67], 'kruh': [2.19, 43], 'jogurt': [0.49,
↪  56], 'paradižnik': [0.23, 35], 'piškoti': [2.99, 73], 'gorgonzola': [2.69,
↪  32], 'jajce': [0.1, 103]}
```

```
>>> zaloga = v_slovar(izdelki)
>>> popravek_zaloge(zaloga, 'jogurt', -60)
>>>
```

7.6.6 Blagajna

Napišite funkcijo `blagajna`, ki kot argumenta sprejme slovar zaloge, kot ga vrača funkcija `v_slovar` in slovar nakupov, kot ga vrača funkcija `nakupovalna_kosara`. Funkcija `blagajna` naj spremeni slovar `zaloga` glede na količino izdelkov v slovarju nakupov in hkrati vrne znesek, ki ga mora plačati kupec. Kupec ne more kupiti več izdelkov kot jih je v zalogi.

Pomagajte si s funkcijami, ki ste jih napisali do sedaj.

Primer pravilnega delovanja funkcije:

```
>>> zaloga = v_slovar(izdelki)
>>> seznam = ['jogurt', 'mleko', 'jogurt', 'jogurt', 'piškoti']
>>> slovar_nakupov = nakupovalna_kosara(seznam)
>>> blagajna(zaloga, slovar_nakupov)
5.32
>>> zaloga
{'paradižnik': [0.23, 35], 'kruh': [2.19, 43], 'mleko': [0.86, 127], 'jajce':
↪  [0.1, 103], 'piškoti': [2.99, 72], 'moka': [1.39, 99], 'jogurt': [0.49,
↪  53], 'hrenovka': [1.99, 28], 'sok': [1.79, 104], 'makaroni': [1.89, 67],
↪  'gorgonzola': [2.69, 32]}
```

7.6.7 Kuhanje

V slovarju `jedi` kot ključi nastopajo imena jedi, vrednosti pa so slovarji z imenom sestavine in potrebno količino za pripravo 1 obroka:

```
jedi = {    "palačinke": {"jajce": 3, "mleko": 1, "moka": 1},
    "šmorn": {"jajce": 3, "mleko": 1, "moka": 1},
    "hrenovke": {"hrenovka": 2, "kruh": 1},
    "makaroni": {"makaroni": 1, "paradižnik": 3, "gorgonzola": 1},
    "piškoti": {"piškoti": 1}}
```

Napišite funkcijo `kuhanje(zaloga, jedi, jed)`, ki vrne ceno izdelkov potrebnih za pripravo izbrane jedi.

Primer pravilnega delovanja funkcije:

```
>>> zaloga = v_slovar(izdelki)
>>> kuhanje(zaloga, jedi, 'palačinke')
2.55
```

7.6.8 Obrok

Napišite funkcijo `posamezna_jed(jedi, jed, obrokov)`, ki kot argumente sprejme slovar receptov jedi (tj. `jedi`), ime jedi (argument `jed` v obliki niza (`str`)), ki jo želimo pripraviti in število obrokov v obliki števila (`int`). Funkcija naj vrača slovar s potrebnimi količinami sestavin za izbrano jed.

Primer pravilnega delovanja funkcije:

```
>>> posamezna_jed(jedi, "šmorn", 4)
{'moka': 4, 'mleko': 4, 'jajce': 12}
```

7.6.9 Nakup

Napišite funkcijo `nakup(jedi, obroki)`, ki kot argumenta sprejme slovar jedi (tj. `jedi`) in število obrokov. Funkcija naj vrača slovar, kjer kot ključi nastopajo potrebne sestavine, kot vrednosti pa količine teh sestavin, potrebnih za pripravo vseh obrokov.

Primer pravilnega delovanja funkcije:

```
>>> obroki = [("makaroni", 20), ("palačinke", 15), ("šmorn", 10), ("hrenovke",
↪  5)]
>>> nakup(jedi, obroki)
{'paradižnik': 60, 'gorgonzola': 20, 'moka': 25, 'kruh': 5, 'hrenovka': 10,
↪  'jajce': 75, 'makaroni': 20, 'mleko': 25}
```

7.6.10 Primanjkljaj

Napišite funkcijo `primanjkljaj(zaloga, jedi, obroki)`, ki kot argumente sprejme zalogo trgovine, seznam jedi in količino obrokov. Funkcija naj vrača slovar, v katerem kot ključi nastopajo sestavine, kot vrednosti pa pripadajoče količine sestavin, ki jih morajo v trgovini dokupiti glede na obroke.

Primer pravilnega delovanja funkcije:

```
>>> zaloga = v_slovar(izdelki)
>>> obroki = [("makaroni", 20), ("palačinke", 15), ("šmorn", 10), ("hrenovke",
↪  25)]
>>> primanjkljaj(zaloga, jedi, obroki)
{'hrenovka': 22, 'paradižnik': 25}
```

7.7 Rešitve nalog

7.6.1 V slovar

```
def v_slovar(izdelki):
    cene = {}
    for izdelek, cena, zaloga in izdelki:
        cene[izdelek] = [cena, zaloga]
    return cene
```

7.6.2 Število izdelkov

```
def stevilo_izdelkov(zaloga):
    vsota = 0
    for izdelek, vrednost in zaloga.items():
        cena, zaloga = vrednost
        vsota += zaloga
    return vsota
```

7.6.3 Nakupovalna košara

```
def nakupovalna_kosara(seznam):
    slovar = {}
    for izdelek in seznam:
        if izdelek not in slovar:
            slovar[izdelek] = 0
        slovar[izdelek] += 1
    return slovar
```

7.6.4 Cena

```
def cena(zaloga, slovar_nakupov):
    znesek = 0
    for izdelek, kolicina in slovar_nakupov.items():
        znesek += kolicina * zaloga[izdelek][0]
    return znesek
```

7.6.5 Popravek zaloge

```
def popravek_zaloge(zaloga, izdelek, popravek):
    nova_zaloga = zaloga[izdelek][1] + popravek
    if nova_zaloga >= 0:
        zaloga[izdelek][1] = nova_zaloga
        return nova_zaloga
    else:
        return None
```

7.6.6 Blagajna

```
def blagajna(zaloga, slovar_nakupov):
    for izdelek, kolicina in slovar_nakupov.items():
        popravek_zaloge(zaloga, izdelek, -kolicina)
    return cena(zaloga, slovar_nakupov)
```

7.6.7 Kuhanje

```
def kuhanje(zaloga, jedi, jed):
    cena = 0
    for sestavina, kolicina in jedi[jed].items():
        cena += kolicina * zaloga[sestavina][0]
    return cena
```

7.6.8 Obrok

```
def posamezna_jed(jedi, jed, obrokov):
    sestavine = {}
    for sestavina, kolicina in jedi[jed].items():
        sestavine[sestavina] = kolicina * obrokov
    return sestavine
```

7.6.9 Nakup

```
def nakup(jedi, obroki):
    sestavine = {}
    for jed, obrokov in obroki:
        for sestavina, kolicina in posamezna_jed(jedi, jed, obrokov).items():
            if sestavina not in sestavine:
                sestavine[sestavina] = 0
            sestavine[sestavina] += kolicina
    return sestavine
```

7.6.10 Primanjkljaj

```
def primanjkljaj(zaloga, jedi, obroki):
    sestavine = nakup(jedi, obroki)
    slovar = {}
    for sestavina, kolicina in sestavine.items():
        if zaloga[sestavina][1] < kolicina:
            slovar[sestavina] = kolicina - zaloga[sestavina][1]
    return slovar
```

8. vaje: Množice in metode

8.1 Množice

Množice (angl. *set*) so nabori unikatnih elementov nekega podatkovnega tipa, ki jih pripravimo tako, da nad podatkovnim tipom izvedemo funkcijo `set()`. Kljub temu, da so množice označene z zavitimi oklepaji, tj. { }, prazno množico pripravimo z uporabo funkcije `set()`, in sicer: `mnozica = set()` in ne z dvema zavitima oklepajema, saj bi to tvorilo slovar, ne pa množice.

1. **Tvorba množic iz nizov** - množica bo vsebovala posamezne unikatne znake, ki so se pojavili v nizu:

```
protein1 = 'QLTWLLRFFMYVHTVEELCRIHQSHCFEIFKWKINNIIAHCCSVINTICF'
aminokisline_p1 = set(protein1)

protein2 = 'YGDHVIMDELRSCAMTSIHEYFECKTSIYHTKCGTLCNPALSYLGIWGQT'
aminokisline_p2 = set(protein2)

print(f'Aminokisline v prvem zaporedju: {aminokisline_p1} \nAminokisline v
↪  drugem zaporedju: {aminokisline_p2}.')
```

   Zgornji program vrne le tiste aminokisline (znake), ki so se pojavile v nizu.

```
Aminokisline v prvem zaporedju: {'K', 'A', 'W', 'M', 'N', 'L', 'E', 'F',
↪  'T', 'I', 'C', 'V', 'H', 'Q', 'Y', 'R', 'S'}
Aminokisline v drugem zaporedju: {'W', 'C', 'V', 'H', 'A', 'G', 'Y', 'L',
↪  'I', 'Q', 'R', 'P', 'S', 'M', 'N', 'E', 'F', 'T', 'D', 'K'}.
```

2. **Tvorba množic iz seznama nizov** - množica bo vsebovala unikatne elemente, ki so se pojavili v seznamu:

```
zaporedja_proteinov =
↪  ['QLTWLLRFFMYVHTVEELCRIHQSHCFEIFKWKINNIIAHCCSVINTICF',
↪  'YGDHVIMDELRSCAMTSIHEYFECKTSIYHTKCGTLCNPALSYLGIWGQT',
↪  'VKASPQGWHDKASRCPTGEDRHVNTNGFSIDDIEALESNMQPSPEVYCMH',
↪  'MRVYFYQNAEPGRMAAHLKCFQYCVGGVCKAVESYDDSRTGRIMRVDWLN',
↪  'FRKVRWWMLSLLPFDIYETNPHQQHYLAHFKICGKNRDLKICPDEFKFML',
↪  'LGDRKIIPIETIWPRYNWLKRHLPNNHSRPCLQIEHALECLEKEMPHGDI',
↪  'QLTWLLRFFMYVHTVEELCRIHQSHCFEIFKWKINNIIAHCCSVINTICF',
↪  'LIPTQIYCCLQRPKYISVRQCWTHILATWTIKTRHTWEHLEARECWSPWH',
↪  'DVFIKACQGPGMQTMFSFETEPMLPKPMMYSGKGYVFDRVRQNRIETVWV',
↪  'QLTWLLRFFMYVHTVEELCRIHQSHCFEIFKWKINNIIAHCCSVINTICF',
↪  'CNQQLWFEERFREGRRVNQNGCEMKIPIKGYGEPTVEWRINWTFDMEDWP',
↪  'VYICLGKATEPWWFQFELAYKNSNEVAWVINCCKCINHVYPTNMINQKMF']

print(set(zaporedja_proteinov))
```

V množici je manj elementov kot jih je bilo v seznamu, kar pomeni, da so se določeni elementi v seznamu ponovili večkrat:

```
{'LGDRKIIPIETIWPRYNWLKRHLPNNHSRPCLQIEHALECLEKEMPHGDI',
↪  'CNQQLWFEERFREGRRVNQNGCEMKIPIKGYGEPTVEWRINWTFDMEDWP',
↪  'LIPTQIYCCLQRPKYISVRQCWTHILATWTIKTRHTWEHLEARECWSPWH',
↪  'QLTWLLRFFMYVHTVEELCRIHQSHCFEIFKWKINNIIAHCCSVINTICF',
↪  'DVFIKACQGPGMQTMFSFETEPMLPKPMMYSGKGYVFDRVRQNRIETVWV',
↪  'FRKVRWWMLSLLPFDIYETNPHQQHYLAHFKICGKNRDLKICPDEFKFML',
↪  'VKASPQGWHDKASRCPTGEDRHVNTNGFSIDDIEALESNMQPSPEVYCMH',
↪  'YGDHVIMDELRSCAMTSIHEYFECKTSIYHTKCGTLCNPALSYLGIWGQT',
↪  'VYICLGKATEPWWFQFELAYKNSNEVAWVINCCKCINHVYPTNMINQKMF',
↪  'MRVYFYQNAEPGRMAAHLKCFQYCVGGVCKAVESYDDSRTGRIMRVDWLN'}
```

Dodajanje elementov v množico izvedemo z uporabo metode `.add()` - bodite pozorni, da pri seznamih uporabljamo `.append()`, pri množicah pa `.add()`. Če v množici že obstaja element, ki ga dodajamo, se ne bo ponovno dodal, saj lahko množice vsebujejo le eno ponovitev določenega elementa - samo unikatne elemente.

Med množicami lahko izvajamo računske operacije unije, preseka, razlike in simetrične razlike. Operacija unije vrne vse elemente v množicah, s katerimi računamo. Presek vrača le elemente, ki so danim množicam skupni. Razlika dveh množic vsebuje elemente prve množice, ne pa tudi druge. Simetrična razlika množic je množica elementov, ki pripadajo danim množicam, vendar niso skupni. Metode in operatorji, s katerimi izvajamo računske operacije med množicami v Pythonu, so povzeti v Tabeli 4.

Tabela 4: Metode in operatorji za izvajanje računskih operacij med množicami. Z rdečo barvo je poudarjena metoda/operator, s črno sta dopisani množici `mn1` in `mn2`, s katerima prikažemo uporabo. Pri razliki in simetrični razliki bodite pozorni na vrstni red množic.

Operacija:	Metoda:	Operator:
unija	`mn1.union(mn2)`	`mn1 | mn2`
presek	`mn1.intersection(mn2)`	`mn1 & mn2`
razlika	`mn1.difference(mn2)`	`mn1 - mn2`
simetrična razlika	`mn1.symmetric_difference(mn2)`	`mn1 ^ mn2`

Poglejmo si primer iskanja aminokislin, ki se pojavijo v obeh proteinskih zaporedjih in aminokislin, ki se pojavijo samo v `protein2`, ne pa tudi v `protein1`:

```
1 >>> aminokisline_p1 | aminokisline_p2
2 {'G', 'M', 'K', 'V', 'H', 'L', 'N', 'P', 'E', 'A', 'R', 'T', 'C', 'F', 'Y',
  ↪  'D', 'I', 'W', 'Q', 'S'}
3 >>> aminokisline_p2 - aminokisline_p1
4 {'G', 'P', 'D'}
```

Množice so neurejene, kar pomeni, da jih ne moremo indeksirati. Vrstni red elementov v njih je naključen in neodvisen od zaporedja njihovega dodajanja - vsakič, ko poženemo kodo, se lahko vrstni red spremeni.

8.2 Metode

Vse, kar ima svoj tip (angl. *class*) in atribute (podatke), je v Pythonu obravnavano kot objekt (angl. *object*). Nad objekti lahko izvajamo operacije, imenovane metode - gre za funkcije, ki so vezane na določene podatkovne tipe. Z njimi smo se že srečali tekom vaj, primer so `.union()`, vezana na množice, metoda `.items()`, vezana na slovarje, itd. Pogosto uporabljene metode, skupaj z objekti, nad katerimi jih lahko izvajamo, so zbrane v Tabeli 5.

Pri uporabi metod nad objekti v Pythonu moramo razmisliti, ali želimo, da sprememba objekta postane trajna ali ne. Če želimo aminokislinsko zaporedje iz primera v poglavju 8.1 pretvoriti v male črke, lahko uporabimo ukaz `protein1.lower()` ali `protein1 = protein1.lower()`. Prvi ukaz zgolj ustvari novo različico niza z malimi črkami, pri čemer izvorna spremenljivka ostane nespremenjena. Če pa želimo, da sprememba postane trajna – torej da spremenljivka `protein1` od tega trenutka dalje hrani preoblikovan niz – moramo rezultat metode prirediti izvorni spremenljivki z ukazom `protein1 = protein1.lower()`. S tem prepišemo staro vsebino in spremenljivka bo trajno vsebovala spremenjeno obliko niza.

Tabela 5: Pregled pogosto uporabljenih metod. Poleg vsake metode je naveden objekt, nad katerim lahko metodo izvajamo, kratek opis metode in primer uporabe.

Metoda:	Objekt:	Opis:	Primer uporabe:
`X.lower()`	niz	niz `X` spremenimo v male tiskane črke	`"Abeceda".lower()` `abeceda`
`X.upper()`	niz	niz `X` spremenimo v velike tiskane črke	`"Abeceda".upper()` `ABECEDA`
`X.capitalize()`	niz	prvo črko niza `X` spremenimo v veliko tiskano črko	`"abeceda".capitalize()` `Abeceda`
`X.strip(Y)`	niz	v nizu `X` odstrani vse znake `Y`, ki so na začetku ali na koncu niza če `Y` ne določimo, se odstranijo beli prostori	`"Hello world!a".strip("a")` `Hello world!`
`X.count(Y)`	niz, seznam	prešteje število pojavitev elementa `Y` v nizu ali seznamu `X`	`"aabb".count("a")` `2`
`X.split(Y)`	niz	niz `X` razdeli po znaku `Y`	`"a,b,c".split(",")` `['a','b','c']`
`Y.join(X)`	seznam	združi elemente seznama `X` v niz z ločilom `Y`	`",".join(["a","b","c"])` `"a,b,c"`
`X.append(Y)`	seznam	doda element `Y` na konec seznama `X`	`seznam.append(5)` `["a","b","c",5]`
`X.append(Y)`	seznam	izpiše indeks seznama `X`, na katerem se nahaja element `Y`	`seznam.index("b")` `1`
`X.keys()`	slovar	vrne vse ključe slovarja `X`	`slovar.keys()`
`X.values()`	slovar	vrne vse vrednosti slovarja `X`	`slovar.values()`
`X.items()`	slovar	vrne vse (ključ,vrednost) pare slovarja `X`	`slovar.items()`
`X.add(Y)`	množica	doda element `Y` v množico `X`	`mn.add(5)`

8.3 Naloge za utrjevanje

V nalogah 1-4 predpostavljajte, da je seznam proteinov po obliki enak seznamu `zaporedja_proteinov` iz poglavja 8.1.

1. Napišite funkcijo `iskanje_proteinov`, ki kot argument sprejme seznam proteinov, vrne pa le tista zaporedja, ki vsebujejo 2 ali več argininov (R). V seznamu se lahko določeno zaporedje ponovi večkrat, pri vračanju pa želimo, da se pojavi le enkrat.
2. Napišite funkcijo `stetje`, ki kot argument sprejme seznam proteinov, vrne pa seznam terk oblike `(zaporedje, ponovitev)`, pri čemer je `ponovitev` številka, ki pove, kolikokrat se določeno zaporedje ponovi v podanem seznamu.
3. Napišite funkcijo `dolzine`, ki kot argument sprejme seznam proteinov, vrne pa seznam terk oblike `(stevilo_aminokislin, stevilo_zaporedij)`, pri čemer je `stevilo_zaporedij` število proteinov iz seznama, ki dosegajo določeno dolžino, tj. `stevilo_aminokislin`.
4. Napišite funkcijo `manjkajoce`, ki kot argument sprejme seznam proteinov, vrne pa množico tistih aminokislin, ki se ne pojavijo v nobenem od podanih proteinov v danem seznamu.
5. Z uporabo metod pretvorite niz `"You still have much to learn."` v niz `"Much to learn, you still have."` Po enakem principu napišite funkcijo `yoda`, ki podanemu nizu spremeni vrstni red besed na enak način.

8.4 Naloge

Funkcije shranite v datoteko `metode.py`.

8.4.1 Besede z a-ji

Napišite funkcijo `aa_besede`, ki kot argument sprejme niz, vrne pa množico besed iz niza, ki vsebujejo vsaj dve črki `a`. Funkcija naj ne bo občutljiva na velikost črk, v množici naj ne bo ločil. Predpostavite lahko, da bo edino ločilo v testnih primerih pika (tj. `.`).

Primer pravilnega delovanja funkcije:

```
>>> aa_besede('Anja je v trgovini kupila ananas in jabolka.')
{'ananas', 'Anja', 'jabolka'}
```

8.4.2 Najdaljše besede

Napišite funkcijo `najdaljse_besede`, ki kot argument sprejme niz, vrne pa množico najdaljših besed v podanem nizu. Predpostavite lahko, da bosta edini ločili v testnih nizih pika (`.`) in vejica (`,`).

Primer pravilnega delovanja funkcije:

```
>>> najdaljse_besede('Univerza v Ljubljani')
{'Ljubljani'}
```

8.4.3 Inicialke

Napišite funkcijo `velike_zacetnice`, ki kot argument sprejme niz. Funkcija naj vrača spremenjen niz, v katerem vsem besedam, daljšim od dveh znakov, spremeni prvo črko v veliko začetnico.

Primer pravilnega delovanja funkcije:

```
>>> velike_zacetnice('V trgovini je kupila ananas in jabolka.')
'V Trgovini je Kupila Ananas in Jabolka.'
```

8.4.4 Unikaten seznam

Napišite funkcijo `razlicne_skladbe`, ki kot argument sprejme seznam poslušanih skladb in vrne število različnih skladb.

Primer pravilnega delovanja funkcije:

```
>>> razlicne_skladbe(['Let It Be', 'Imagine', 'The River', 'One', 'Stand By
↪  Me', 'Imagine', 'Stairway To Heaven', 'One', 'Imagine'])
6
```

8.4.5 Prijatelji

Napišite funkcijo `skupne`, ki kot argumenta sprejme dva seznama. Seznama vsebujeta naslove poslušanih skladb dveh prijateljev. Funkcija naj vrne množico skladb, ki so na obeh seznamih.

Primer pravilnega delovanja funkcije:

```
>>> skupne(['Let It Be', 'Imagine', 'The River', 'One', 'Stand By Me'],
↪  ['Imagine', 'Stairway To Heaven', 'One', 'Imagine'])
{'Imagine', 'One'}
```

8.4.6 Repertoar

Napišite funkcijo `repertoar`, ki kot argumenta sprejme dva seznama predvajanj dveh prijateljev. Funkcija naj vrača množico skladb, ki so na enem ali drugem ali obeh seznamih.

Primer pravilnega delovanja funkcije:

```
>>> repertoar(['Let It Be', 'Imagine', 'The River', 'One', 'Stand By Me'],
↪  ['Imagine', 'Stairway To Heaven', 'One', 'Imagine'])
{'Let It Be', 'Imagine', 'The River', 'One', 'Stand By Me', 'Stairway To
↪  Heaven'}
```

8.4.7 Unikati

Napišite funkcijo `unikatna_predvajanja(seznam1, seznam2)`, ki sprejme seznama predvajanj dveh prijateljev. Funkcija naj vrne množico skladb, ki so na enem ali drugem seznamu, ne pa na obeh.

Primer pravilnega delovanja funkcije:

```
>>> unikatna_predvajanja(['Let It Be', 'Imagine', 'The River', 'One', 'Stand By
↪  Me'], ['Imagine', 'Stairway To Heaven', 'One', 'Imagine'])
{'Let It Be', 'The River', 'Stand By Me', 'Stairway To Heaven'}
```

8.4.8 Ponavljajoči se znaki

Napišite funkcijo `ponavljajoci_znaki`, ki kot argument sprejme niz. Funkcija naj vrne množico terk oblike: `(znak, število)`, pri čemer `število` ponazarja število ponovitev znaka v izbranem nizu. Funkcija naj vrne le tiste znake, ki se pojavijo več kot enkrat. Prav tako naj bo občutljiva na velikost črk.

Primer pravilnega delovanja funkcije:

```
>>> ponavljajoci_znaki('otorinolaringologija')
{('l', 2), ('i', 3), ('r', 2), ('a', 2), ('g', 2), ('o', 5), ('n', 2)}
```

8.4.9 Onesnaženost z delci PM10

Agencija Republike Slovenije za okolje (ARSO) nas je prosila, da naredimo analizo onesnaženosti z delci PM10 v treh največjih slovenskih mestih (Ljubljana, Maribor in Celje) ter primerjamo podatke z merilno postajo Iskrba[13].

Napišite funkcijo `najveckrat_onesnazena_mesta`, ki sprejme dva argumenta: seznam mesečnih podatkov o onesnaženosti (podan kot seznam terk z dnevnimi meritvami merilnih postaj) in seznam krajev. Funkcija naj vrne množico vseh krajev, v katerih je bila največkrat dosežena oz. presežena mejna vrednost 50 μg/m^3. Takih krajev je lahko več ali pa jih sploh ni – v tem primeru naj funkcija vrne prazno množico.

Seznam dnevnih meritev za en mesec vsebuje 28, 30 ali 31 terk, vsaka izmed njih pa predstavlja podatke za en dan v mesecu. Vsaka terka ima dva elementa: dan v mesecu in terko, v kateri so podatki o povprečni dnevni koncentraciji delcev PM10 merilnih postaj Ljubljana, Maribor, Celje ter Iskrba. Podatki so vedno podani v tem vrstnem redu postaj. Podatki so podani v enotah μg/m^3. Lahko se zgodi, da kakšne meritve manjkajo, kar je v terki označeno z `None`.

Primer podatkov za januar 2017:

```
kraji = ["Ljubljana", "Maribor", "Celje", "Iskrba"]
januar = [
    (1, (126, 88, None, 12)),
    (2, (53, 68, 77, 13)),
    (3, (34, 81, 96, 8)),
    (4, (None, 69, 117, 12)),
    (5, (31, 19, 51, 6)),
    (6, (13, 16, 16, 5)),
    (7, (22, 34, 42, 11)),
    (8, (39, 50, 58, 8)),
    (9, (66, 71, 90, 16)),
    (10, (41, 50, 52, 28)),
    (11, (62, 67, 82, 18)),
    (12, (73, 77, 29, 5)),
    (13, (12, 15, 19, 3)),
    (14, (34, 14, 57, 8)),
    (15, (48, 18, 82, 11)),
    (16, (32, 23, 72, 12)),
    (17, (15, 24, 28, 19)),
    (18, (23, 34, 36, 22)),
    (19, (38, 56, 65, 27)),
    (20, (51, 72, None, 17)),
    (21, (67, 101, None, 25)),
```

[13]Merilno mesto Iskrba se nahaja ob Kočevski Reki, kjer v bližini ni virov, ki bi povzročali emisije delcev.

```
    (22, (70, 138, 142, 40)),
    (23, (78, 170, 146, 67)),
    (24, (96, 106, 105, 82)),
    (25, (56, 74, 73, 26)),
    (26, (31, 51, 75, 22)),
    (27, (40, 62, 66, 22)),
    (28, (81, 94, 116, 29)),
    (29, (101, 100, 109, 68)),
    (30, (83, 99, 102, 42)),
    (31, (110, 146, 115, 5))
]
```

Primer pravilnega delovanja funkcije:

```
>>> najveckrat_onesnazena_mesta(januar, kraji)
{'Maribor', 'Celje'}
```

8.5 Rešitve nalog

8.4.1 Besede z a-ji

```
def aa_besede(s):
    mn = set()
    for beseda in s.replace('.','').split():
        if beseda.lower().count('a') > 1:
            mn.add(beseda)
    return mn
```

8.4.2 Najdaljše besede

```
def najdaljse_besede(s):
    najdaljsa = 0
    besede = s.replace('.','').replace(',','').split()
    for beseda in besede:
        if len(beseda) > najdaljsa:
            najdaljsa = len(beseda)

    mn = set()
    for beseda in besede:
        if len(beseda) == najdaljsa:
            mn.add(beseda)
    return mn
```

8.4.3 Inicialke

```
def velike_zacetnice(s):
    seznam = []
    for beseda in s.split():
        if len(beseda.replace('.','').replace(',','')) > 2:
            beseda = beseda.capitalize()
        seznam.append(beseda)
    return " ".join(seznam)
```

Če niste našli metode `capitalize`, lahko nalogo rešite tudi tako:
`beseda = beseda[0].upper()+beseda[1:]`

8.4.4 Unikaten seznam

```
def razlicne_skladbe(seznam):
    return len(set(seznam))
```

8.4.5 Prijatelji

```
def skupne(seznam1, seznam2):
    return set(seznam1) & set(seznam2)
```

8.4.6 Repertoar

```
def repertoar(seznam1, seznam2):
    return set(seznam1) | set(seznam2)
```

8.4.7 Unikati

```
def unikatna_predvajanja(seznam1, seznam2):
    return set(seznam1) ^ set(seznam2)
```

8.4.8 Ponavljajoči se znaki

```
def ponavljajoci_znaki(niz):
    mn = set()
    for znak in niz:
        c=niz.count(znak)
        if c > 1:
            mn.add((znak,c))
    return mn
```

8.4.9 Onesnaženost z delci PM10

```
def najveckrat_onesnazena_mesta(podatki, kraji):
    mesta = [0] * len(kraji)
    for dan,postaje in podatki:
        for i in range(len(postaje)):
            if postaje[i] is not None and postaje[i] >= 50:
                mesta[i] += 1
    naj = max(mesta)
    if naj == 0:
        return(set())
    else:
        maksimumi = set()
        for i in range(len(kraji)):
            if mesta[i] == naj:
                maksimumi.add(kraji[i])
        return maksimumi
```

9. vaje: Datoteke

Kadar delamo z datotekami v Pythonu, datoteko vedno najprej odpremo, preberemo/uredimo podatke in na koncu datoteko zapremo.

9.1 Odpiranje datotek

Datoteko lahko odpremo na dva načina:

1. `f = open("datoteka.txt", "r", encoding = "UTF-8")`
2. `with open("datoteka.txt", "r", encoding="UTF-8") as f:`
   Pri uporabi `with open() as` pazite, da naslednjo vrstico zamaknete.

Če imamo datoteko shranjeno na isti lokaciji kot program, s katerim jo odpiramo, zadostuje le ime datoteke. Če se lokacija razlikuje od programa, moramo navesti pot do datoteke.

Pri odpiranju datoteke moramo biti pozorni na načine dostopa, ki so:

- `'w'` (angl. *write*): pisanje od začetka datoteke, pri čemer se morebitna obstoječa vsebina datoteke prepiše z novo vsebino, v primeru da datoteka še ne obstaja, pa bo ustvarjena,
- `'r'`, (angl. *read*): branje podatkov,
- `'a'`, (angl. *append*): dodajanje vsebine na konec datoteke, pri čemer se obstoječa vsebina ne spremeni,
- `'w+'`, `'r+'`, `'a+'`: omogočajo branje in pisanje podatkov.

Običajno k odpiranju, sploh, če delamo na več različnih računalnikih, dodamo argument `encoding`, kjer definiramo kodirno tabelo znakov za našo datoteko. Tako zagotovimo, da se vsi znaki pravilno izpisujejo.

9.2 Branje podatkov

Do podatkov v datotekah lahko dostopamo na štiri načine:

1. z `f.read()` celotno vsebino datoteke shranimo kot niz, ki ne vsebuje posebnih znakov za novo vrstico,
2. `f.readlines()` ustvari seznam nizov, pri čemer je en niz ena vrstica, seznam ima torej toliko nizov kot ima datoteka vrstic,
3. z `f.readline()` se pomaknemo le za eno vrstico naenkrat,
4. najpogostejši način je uporaba zanke *for*, `for vrstica in f:`, ki omogoči pomikanje po datoteki, za eno vrstico naenkrat.

Za lažje razumevanje si poglejmo vse štiri načine na primeru branja datoteke `hamlet.txt`, ki vsebuje sledečo vsebino[14]:

```
To be, or not to be, that is the question:
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune,
Or to take arms against a sea of troubles,
And by opposing end them? To die|to sleep,
No more; and by a sleep to say we end
The heart-ache, and the thousand natural shocks
That flesh is heir to: 'tis a consummation
Devoutly to be wish'd. To die, to sleep.
```

1.
```
>>> tekst = f.read()
>>> tekst
'To be, or not to be, that is the question:'tis nobler in the mind to
↪  sufferslings and arrows of outrageous fortune,to take arms against a sea
↪  of troubles,by opposing end them? To die|to sleep,more; and by a sleep
↪  to say we endheart-ache, and the thousand natural shocksflesh is heir
↪  to: 'tis a consummationto be wish'd. To die, to sleep.'
```
Celotno besedilo je en niz. Znakov za prelom ni.

2.
```
>>> tekst = f.readlines()
>>> tekst
['To be, or not to be, that is the question:', 'Whether 'tis nobler in the
↪  mind to suffer', 'The slings and arrows of outrageous fortune,', 'Or to
↪  take arms against a sea of troubles,', 'And by opposing end them? To
↪  die|to sleep,', 'No more; and by a sleep to say we end', 'The
↪  heart-ache, and the thousand natural shocks', 'That flesh is heir to:
↪  'tis a consummation', 'Devoutly to be wish'd. To die, to sleep.']
```
`f.readlines()` tvori seznam, kjer je posamezna vrstica en niz. Posebej bodite pozorni, da se vsak niz (vrstica) konča z znakom za nov odstavek `\n`.

3.
```
>>> f.readline()
'To be, or not to be, that is the question:'
>>> f.readline()
'Whether 'tis nobler in the mind to suffer'
```
`f.readline()` omogoči pomikanje po datoteki za zgolj eno vrstico naenkrat - tolikokrat, kot uporabimo `f.readline()`. Ta način je zelo uporaben, kadar želimo kakšno vrstico (npr. naslovno) izvzeti iz nadaljnje obravnave.

[14]Vsebina datoteke je izsek iz drame Hamlet Williama Shakespearja, dostopne na https://www.gutenberg.org/cache/epub/1524/pg1524.txt.

4. ```
   for vrstica in f:
       print(vrstica)
   ```
   Pri uporabi tega načina se vsaka vrstica izpiše posebej. Ker smo uporabili funkcijo `print()`, se znaki za prelom besedila ne izpišejo na zaslonu, prav tako ni narekovajev, ki bi označevali niz:
   ```
   To be, or not to be, that is the question:

   Whether 'tis nobler in the mind to suffer

   The slings and arrows of outrageous fortune,

   Or to take arms against a sea of troubles,

   And by opposing end them? To die|to sleep,

   No more; and by a sleep to say we end

   The heart-ache, and the thousand natural shocks

   That flesh is heir to: 'tis a consummation

   Devoutly to be wish'd. To die, to sleep.
   ```

9.3 Pisanje v datoteko

Če želimo pisati v datoteko, moramo datoteko odpreti z načini `w`, `w+`, `a`, `a+` ali `r+`. Za pisanje lahko uporabimo:

1. metodo `f.write()`:
   ```
   f.write("Besedilo, ki se napiše v datoteko.")
   ```
2. funkcijo `print()`, pri kateri moramo kot opcijski argument obvezno navesti v katero datoteko želimo pisati:
   ```
   print("Besedilo, ki se napiše v datoteko.", file = f)
   ```

9.4 Zapiranje datoteke

Datoteko po končanem delu zapremo s preprostim ukazom `f.close()`. Zapiranje datoteke zagotovi shranjevanje podatkov v datoteko, razbremeni sistemske resurse (predvsem kadar delamo bodisi z veliko datotekami hkrati ali z velikimi datotekami) in omogoči, da lahko po končanem programskem delu datoteko odpiramo in urejamo še z drugimi programi.

Če smo datoteko odpirali z `with open("datoteka.txt", "r", encoding="UTF-8") as f:`, se bo datoteka sama zaprla po končanem bloku kode, ki sledi temu stavku.
   ```

9.5 Zgledi

9.5.1 Zaporedja

V datoteki `zaporedja.txt` so zapisana številna kratka zaporedja DNA s pripadajočimi oznakami. Za nadaljnje delo nas zanimajo zgolj zaporedja DNA, ki jih želimo shraniti v seznam.

Pri delu z datotekami imamo običajno opravka z več sto vrsticami. Vsakič, ko delamo z datoteko, poskusimo najti vzorce, ki nam bodo pomagali pri ekstrakciji želenih podatkov. Poglejmo si nekaj prvih vrstic datoteke `zaporedja.txt`:

```
>zaporedje1
gtgataatgagagaaaggtagaagg

>zaporedje2
tatagaaccgatctactcaatcccc

>zaporedje3
gggcagtcggcatcatcaccgaagg
```

Opazimo lahko, da so imena zaporedij v vrsticah, ki se začnejo z znakom `>`. V vrstici pod njo se nahaja zaporedje, za njim pa je prazna vrstica. Na osnovi teh nekaj vrstic lahko sklepamo, da preostanek datoteke sledi istemu vzorcu.

Ker želimo v seznam shraniti le zaporedja, datoteko odpremo in se z zanko *for* začnemo pomikati po vrsticah. S pogojnim stavkom *if* preverimo, ali se vrstica ne začne z znakom `>` in ali ni prazna (prazne vrstice vsebujejo `\n`) (glej 5. vrstico v spodnji kodi). Če oba pogoja držita - torej se vrstica ne začne s `>` in ni prazna, vsebino vrstice dodamo v seznam. Pri tem odstranimo znak za novo vrstico, ki se nahaja na koncu (v izpisu teh znakov ne vidimo) in zaporedje spremenimo v velike tiskane črke z metodo `.upper()` (glej 6. vrstico v spodnji kodi).

```
1  seznam = []
2  f = open('zaporedja.txt', 'r', encoding = 'UTF-8')
3
4  for vrstica in f:
5      if not vrstica.startswith('>') and vrstica != '\n':
6          seznam.append(vrstica.strip('\n').upper())
7
8  f.close()
9
10 print(seznam)
```

Ko zgornji program zaženemo, se izpiše seznam zaporedij, ki se nahajajo v datoteki `zaporedja.txt`:

```
['GTGATAATGAGAGAAAGGTAGAAGG', 'TATAGAACCGATCTACTCAATCCCC',
'GGGCAGTCGGCATCATCACCGAAGG', 'TGAACCGGGAGTTGAGCCGAGATAC',
'TGAACGTCATTTGGCAAAATTGATT', 'TACAAACGTAGTAGGTCACCACTCG',
'CCCTTCAAGTCTTATCGCTCCATAT', 'TACTATAACCTGGGAGTCATAGAAT',
'CATCATTTCGCGAGCTCAAAGGGTT', 'AATGCGTCTAAAGCGTGCGAATAGT']
```

Če bi v datoteko `zaporedja.txt` želeli dodati novo zaporedje, bi datoteko odprli v načinu `a`. S tem omogočimo le dodajanje na konec datoteke in ne prepisovanje obstoječe vsebine, kar bi se zgodilo v primeru uporabe `w`. Pisanje v datoteko lahko izvedemo na dva načina (glej poglavje 9.3), v spodnji kodi sta za namen ponazoritve uporabljena oba načina - pri pisanju svoje kode izberite tistega, ki vam je bližje.

```
1 f = open('zaporedja.txt', 'a', encoding = 'UTF-8')
2
3 f.write('\n>zaporedje11\n')
4 print('aaacgccacgtcagacaagaataca', file=f)
5
6 f.close()
```

Dodatek `\n` pred in za imenom zaporedja (glej 3. vrstico) zagotovi pravilno oblikovane odstavke pred besedilom `>zaporedje` in za njim.

V datoteki `zaporedja.txt` imamo sedaj naslednjo vsebino:

```
>zaporedje1
gtgataatgagagaaaggtagaagg

>zaporedje2
tatagaaccgatctactcaatcccc

>zaporedje3
gggcagtcggcatcatcaccgaagg

>zaporedje4
tgaaccgggagttgagccgagatac

>zaporedje5
tgaacgtcatttggcaaaattgatt

>zaporedje6
tacaaacgtagtaggtcaccactcg

>zaporedje7
cccttcaagtcttatcgctccatat

>zaporedje8
tactataacctgggagtcatagaat

>zaporedje9
catcatttcgcgagctcaaagggtt

>zaporedje10
aatgcgtctaaagcgtgcgaatagt

>zaporedje11
aaacgccacgtcagacaagaataca
```

9.5.2 Napišite funkcijo `preberi_podatke()` ...

Ena izmed nalog, s katero se boste gotovo srečali na izpitih, je kodiranje funkcije `preberi_podatke(datoteka)`.

Naj ima v danem zgledu datoteka sledečo obliko:

```
avtor;naslov;zbirka;ocena;strani;zanri
Frances Hodgson Burnett;The Secret Garden;;413;331;Classics,Fiction,Childrens
John Green;Looking for Alaska;;402;221;Young Adult,Fiction,Contemporary
Louis Sachar;Holes;Holes;397;233;Young Adult,Fiction,Childrens
Arthur Golden;Memoirs of a Geisha;;412;503;Fiction,Historical Fiction,Romance
Stephenie Meyer;Twilight;The Twilight Saga;360;501;Young Adult,Fantasy,Romance
```

V prvi vrstici imamo navedena imena stolpcev - kateri podatek se nahaja na katerem mestu v vrstici. Kot ločilo med podatki je uporabljeno podpičje (`;`). Opazimo lahko, da na določenih mestih podatki manjkajo (glej 2., 3. in 5. vrstico), saj si dve ločili sledita ena za drugo. Zadnji podatek v vsaki vrstici je žanr, ki je sestavljen iz več besed, ločenih z vejicami (`,`).

Funkcija `preberi_podatke(datoteka)` naj v tem primeru vrne seznam terk oblike `(avtor, naslov, strani, prvi_zanr, ocena)`. Pri tem želimo, da je podatek `strani` število (`int`), `ocena` pa realno število `float()`, zaokroženo na 2 decimalni mesti. Podatek `prvi_zanr` naj bo ime žanra, ki je prvi naveden v stolpcu `zanri`. Izpustiti želimo vrstice, kjer imamo manjkajoče podatke.

```
1  def preberi_podatke(datoteka):
2      seznam = []
3      f = open(datoteka, "r", encoding="UTF-8")
4      f.readline()
5      for vrstica in f:
6          vrstica_s = vrstica.split(';')
7          if '' in vrstica_s:
8              continue
9          avtor, naslov, zbirka, ocena, strani, zanri = vrstica_s
10         prvi_zanr = zanri.split(',')[0]
11         strani = int(strani)
12         ocena = round(float(ocena),2)
13         seznam.append((avtor, naslov, strani, zanr, ocena))
14     return seznam
```

Po odpiranju datoteke v 3. vrstici sledi ukaz `f.readline()`, s katerim se pomaknemo za prvo vrstico, ki vsebuje imena stolpcev. Nato se z zanko *for* pomikamo vrstico po vrstico. Vsako vrstico razbijemo glede na položaj podpičij v seznam nizov z metodo `.split(';')`. Če se v seznamu pojavi prazen niz, ki je posledica razbijanja dveh zaporednih podpičij, to vrstico

spustimo in se s `continue` pomaknemo na naslednjo vrstico.[15] Če vrstica nima manjkajočih podatkov, priredimo imena posameznim elementom v seznamu ter podatke uredimo (pretvorimo v zahtevani podatkovni tip, upoštevamo vrstni red in zapakiramo v terko) pred dodajanjem v seznam.

[15]Manjkajoče podatke bi lahko preverili tudi z `if ';;' in vrstica:`, pri čemer nad vrstico še nismo izvedli metode `.split()`.

9.6 Naloge

9.6.1 Filmi

Vse funkcije tega sklopa shranite v datoteko `filmi.py`.

Datoteka `filmi.txt` se nahaja v podimeniku `podatki` imenika s testi.

Preberi

Napišite funkcijo `preberi`, ki kot argument sprejme ime datoteke v obliki niza (`str`). Funkcija naj prebere datoteko in vrne niz, zapisan v njej.

Primer pravilne izvedbe funkcije:

```
>>> print(preberi('podatki/filmi.txt'))
The Godfather;9.2;Kriminalka;1972
Schindler's List;8.9;Drama;1993
Casablanca;8.6;Drama;1942
Forrest Gump;8.8;Komedija;1994
The Sound of Music;8.0;Glasbena biografija;1965
Gladiator;8.5;Akcija;2000
Titanic;7.7;Romantična drama;1997
Saving Private Ryan;8.6;Akcija;1998
```

V seznam

Napišite funkcijo `v_seznam(datoteka)`, ki prebere datoteko in vrne vsebino v obliki seznama seznamov oblike: `[naslov, ocena, zanr, leto]`.

Pri tem naj bosta spremenljivki `naslov` in `zanr` v obliki niza (`str`), spremenljivka `ocena` naj bo decimalno število (`float`), `leto` pa v obliki celega števila (`int`).

Primer pravilne izvedbe funkcije:

```
>>> v_seznam('podatki/filmi.txt')
[['The Godfather', 9.2, 'Kriminalka', 1972], ["Schindler's List", 8.9, 'Drama',
↪  1993], ['Casablanca', 8.6, 'Drama', 1942], ['Forrest Gump', 8.8,
↪  'Komedija', 1994], ['The Sound of Music', 8.0, 'Glasbena biografija',
↪  1965], ['Gladiator', 8.5, 'Akcija', 2000], ['Titanic', 7.7, 'Romantična
↪  drama', 1997], ['Saving Private Ryan', 8.6, 'Akcija', 1998]]
```

V slovar

Napišite funkcijo `v_slovar`, ki kot argument sprejme ime datoteke. Funkcija naj vrne slovar, kjer so ključi žanr filma, vrednosti pa seznam naslovov, ki ustrezajo temu žanru.

Primer pravilne izvedbe funkcije:

```
>>> v_slovar('podatki/filmi.txt')
{'Romantična drama': ['Titanic'], 'Kriminalka': ['The Godfather'], 'Akcija':
↪  ['Gladiator', 'Saving Private Ryan'], 'Drama': ["Schindler's List",
↪  'Casablanca'], 'Komedija': ['Forrest Gump'], 'Glasbena biografija': ['The
↪  Sound of Music']}
```

Najljubši film

Napišite funkcijo `najljubsi(datoteka_pisanja)`, ki vpraša uporabnika po najljubšem filmu in v novo datoteko, poimenovano `datoteka_pisanja`, zapiše naslov tega filma.

Primer pravilne izvedbe funkcije:

```
>>> najljubsi('podatki/najljubsi_film.txt')
Najljubši film: Forrest Gump
```

V datoteki `najljubsi_film.txt` je sedaj viden zapis:

```
Forrest Gump
```

Zapis

Napišite funkcijo `zapis(datoteka_branja, datoteka_pisanja)`, ki kot argumenta sprejme dva niza z imenoma datoteke branja in datoteke pisanja. Funkcija naj prebere vsebino datoteke branja in v novo datoteko, tj. datoteko pisanja, zapiše naslove filmov, urejene po abecedi.

Primer pravilne izvedbe funkcije:

```
>>> zapis('podatki/filmi.txt', 'podatki/imena_filmov.txt')
```

V datoteki `imena_filmov.txt` je sedaj viden zapis:

```
Casablanca
Forrest Gump
Gladiator
Saving Private Ryan
Schindler's List
The Godfather
The Sound of Music
Titanic
```

CSV

Datoteka `ratings.csv` vsebuje podatke o ocenah več filmov s strani več tisoč ocenjevalcev.

Primer vsebine datoteke `ratings.csv`:

```
userId,movieId,rating,timestamp
1,2,3.5,1112486027
1,29,3.5,1112484676
1,32,3.5,1112484819
1,47,3.5,1112484727
1,50,3.5,1112484580
1,112,3.5,1094785740
1,151,4.0,1094785734
1,223,4.0,1112485573
1,253,4.0,1112484940
```

Napišite funkcijo `ocena_filma`, ki kot argumenta sprejme id_filma, zapisanega kot `int` in ime csv_datoteke. Funkcija naj za podani film vrne njegovo povprečno oceno.

Primer pravilne izvedbe funkcije:

```
>>> ocena_filma(2, 'podatki/ratings.csv')
3.2296978281397544
```

9.6.2 Oliver Twist

Vse naloge tega sklopa shranite v datoteko `knjiga.py`.

Za reševanje potrebujete datoteko `OliverTwist.txt`[16].

Število besed

Napišite funkcijo `stevilo_besed(datoteka)`, ki prebere izbrano datoteko in vrne število besed, ki se pojavijo v knjigi. Besedilo lahko le ločite po presledkih, odstranjevanje drugih ločil ni potrebno.

Primer pravilne izvedbe funkcije:

```
>>> stevilo_besed('OliverTwist.txt')
158041
```

[16]Vir: http://www.gutenberg.org/ebooks/730

Različni znaki

Napišite funkcijo `razlicni_znaki(datoteka)`, ki prebere izbrano datoteko in vrne število različnih znakov, ki se pojavijo v knjigi.

Primer pravilne izvedbe funkcije:

```
>>> razlicni_znaki('OliverTwist.txt')
70
```

Najpogostejši znak

Napišite funkcijo `najpogostejsi_znak(datoteka)`, ki prebere datoteko in vrne **znak**, ki se v datoteki pojavi največkrat.

Primer pravilne izvedbe funkcije:[17]

```
>>> najpogostejsi_znak('OliverTwist.txt')
' '
```

Hapax legomenon

Napišite funkcijo `hapax(datoteka)`, ki prebere izbrano datoteko in vrne število besed, ki se v knjigi Oliver Twist pojavijo natanko enkrat. Besedilo lahko le ločite po presledkih, odstranjevanje drugih ločil ni potrebno.

Primer pravilne izvedbe funkcije:

```
>>> hapax('OliverTwist.txt')
11851
```

[17] Vrnjen znak je presledek.

9.7 Rešitve nalog

9.6.1 Filmi

Preberi

```
def preberi(datoteka):
    f = open(datoteka, 'r', encoding='utf8')
    tekst = f.read()
    f.close()
    return tekst
```

V seznam

```
def v_seznam(datoteka):
    seznam = []
    for vrstica in open(datoteka, 'r', encoding='utf8'):
        film, ocena, zanr, leto = vrstica.split(';')
        seznam.append([film, float(ocena), zanr, int(leto)])
    return seznam
```

V slovar

```
def v_slovar(datoteka):
    slovar = {}
    for vrstica in open(datoteka, 'r', encoding='utf8'):
        film, ocena, zanr, leto = vrstica.split(';')
        if zanr not in slovar:
            slovar[zanr] = []
        slovar[zanr].append(film)
    return slovar
```

Najljubši film

```
def najljubsi(datoteka_pisanja):
    naj = input("Najljubši film: ")
    f = open(datoteka_pisanja, 'w', encoding='UTF-8')
    print(naj, file=f)
    f.close()
```

Zapis

```python
def zapis(datoteka_branja, datoteka_pisanja):
    seznam = []
    for vrstica in open(datoteka_branja, 'r', encoding='utf8'):
        film, ocena, zanr, leto = vrstica.split(';')
        seznam.append(film)
    f = open(datoteka_pisanja, 'w', encoding = 'UTF-8')
    for ime in sorted(seznam):
        print(ime, file = f)
    f.close()
```

CSV

```python
def ocena_filma(id_filma, csv_datoteka):
    f = open(csv_datoteka, "r", encoding = "utf-8")
    stevec = 0
    vsota = 0
    for vrstica in f:
        userId, movieId, rating, timestamp = vrstica
        if movieId == str(id_filma):
            vsota += float(rating)
            stevec += 1
    return vsota / stevec
```

9.6.2 Oliver Twist

Število besed

```
def stevilo_besed(datoteka):
    f = open(datoteka, 'r', encoding='UTF-8')
    tekst = f.read()
    f.close()
    return len(tekst.split())
```

Različni znaki

```
def razlicni_znaki(datoteka):
    f = open(datoteka, 'r', encoding = 'UTF-8')
    tekst = f.read()
    f.close()
    slovar = {}
    for znak in tekst:
        if znak not in slovar:
            slovar[znak] = 0
        slovar[znak] += 1
    return len(slovar)
```

Najpogostejši znak

```
def najpogostejsi_znak(datoteka):
    f = open(datoteka, 'r', encoding = 'UTF-8')
    tekst = f.read()
    f.close()
    slovar = {}
    for znak in tekst:
        if znak not in slovar:
            slovar[znak] = 0
        slovar[znak] += 1
    maksi = max(slovar.values())
    for znak, st in slovar.items():
        if st == maksi:
            return znak
```

Hapax legomenon

```
def hapax(datoteka):
    f = open(datoteka, 'r', encoding = 'UTF-8')
    tekst = f.read()
    tekst_split = tekst.split()
```

```
    f.close()
    slovar = {}
    for beseda in tekst_split:
        if beseda not in slovar:
            slovar[beseda] = 0
        slovar[beseda] += 1
    seznam = []
    for beseda, st in slovar.items():
        if st == 1:
            seznam.append(beseda)
    return len(seznam)
```

10. vaje: Knjižnice

Knjižnice v Pythonu omogočajo dostop do številnih dodatnih funkcionalnosti, kot so uporaba specifičnih matematičnih algoritmov, orodij za risanje grafov, ipd., ki jih osnovni jezik sam po sebi ne omogoča. Knjižnice vsebujejo več modulov, tj. zapakirano kodo s funkcijami in algoritmi, ki so jih pripravili drugi uporabniki/programerji.

Standardne knjižnice, kot sta `math` in `random`, so že vključene v standardno namestitev Pythona in jih ne rabimo posebej nameščati. Številne druge zunanje knjižnice, kot so `numpy`, `matplotlib` in `pandas`, pa razvija in deli skupnost uporabnikov Pythona, zato jih pred uporabo namestimo.

10.1 Nameščanje knjižnic

Najbolj uporabljane knjižnice za znanstvene analize in strojno učenje[18] so že vključene vmesniku Anaconda[19].

Knjižnice lahko brez uporabe vmesnika nameščamo preko ukazne vrstice (`cmd` v operacijskem sistemu Windows oziroma Terminal v operacijskem sistemu iOS). To storimo z ukazom `pip install`, ki mu dodamo ime knjižnice, ki jo želimo namestiti - npr. `pip install pandas`. Pred namestitvijo je dobro preveriti, ali imamo zadnjo verzijo namestitvenega programa `pip` - če ne, jo je dobro posodobiti, kar storimo z ukazom: `pip install --upgrade pip`. Po vsaki uspešni namestitvi nam ukazna vrstica to tudi izpiše.

10.2 Uporaba knjižnic

Če želimo neko knjižnico uporabljati v našem programu, jo na vrhu datoteke uvozimo z ukazom `import`, ki mu dodamo ime želene knjižnice:

```
import numpy
```

Znotraj posamezne datoteke lahko uvozimo poljubno število knjižnic, informacije o posamezni knjižnici in funkcijah pa so zbrane v dokumentaciji, dostopni na spletu. Dokumentacija standardne knjižnice je združena in dostopna na naslovu https://docs.python.org/3/library/index.html, dokumentacije ostalih knjižnic pa se nahajajo na različnih naslovih in jih najlaže najdemo preko iskalnika.

Ko želimo uporabiti funkcijo iz uvožene knjižnice, to storimo tako, da najprej zapišemo ime knjižnice, iz katere prihaja, za piko pa ime funkcije - če želimo uporabiti funkcijo `sin` iz modula `numpy`, zapišemo: `numpy.sin` in pokličemo `x = numpy.sin(y)`.

Pogosto imena knjižnic okrajšamo:

```
import numpy as np
```

[18] Primeri vključenih knjižnico so `matplotlib`, `numpy`, `pandas`, `seaborn` in `pytorch`.
[19] https://anaconda.org

kar nam omogoča, da funkcijo pokličemo `x = np.sin(y)`.

Včasih potrebujemo le eno funkcijo iz celotne knjižnice, ki jo lahko uvozimo z ukazom:

```
from numpy import sin
```

Funkcijo v tem primeru pokličemo brez navedbe knjižnice, v kateri se nahaja: `x = sin(y)`.

Če zapišemo:

```
from numpy import *
```

bo Python uvozil vse funkcije iz te knjižnice, pri čemer jim ob klicu ne rabimo dodajati imena knjižnice. Zveni priročno, vendar lahko hitro privede do zmede, saj imajo številne funkcije iz različnih knjižnic isto ime - tako v knjižnici `math` kot v knjižnici `numpy` imamo funkcijo `log`, ki pa deluje različno. Če iz obeh knjižnic uvozimo vse funkcije, Python ne bo nujno uporabil prave za naš primer. Najboljša praksa je tako uvoz z uporabo krajšav, kot smo videli zgoraj.

10.3 Uporaba lastnih modulov

Pri programiranju večkrat potrebujemo za naše delo specifične funkcije, ki jih običajno zberemo v eno ali več datotek. Ko jih želimo uvoziti v drug program, moramo, tako kot v prešnjih primerih izvesti uvoz modula z ukazom `import`.

Če imamo svoje funkcije shranjene v datoteki `funkcije.py`, jih uvozimo z ukazom `import funkcije`, pod pogojem, da je datoteka, v katero jih uvažamo, v isti mapi. Če se lokaciji datotek razlikujeta, je treba dodati bodisi še ime podmape, v kateri je shranjen modul, dodati modul na PATH ali pa uporabiti orodja iz knjižnice `importlib`.

10.4 Utrjevanje

Vse spodnje naloge se navezujejo na zgled iz poglavja 3.3.1 in 3. nalogo iz poglavja 5.4.

1. Spremenite funkcijo `titracija` tako, da vrača dva seznama - količino dodane baze za vsako točko dodajanja in pH raztopine po vsakem dodatku baze.
2. Napišite funkcijo `graf_titracije`, ki vrne graf pH raztopine v odvisnosti od količine dodane baze.
3. S podatki, ki jih dobite pri uporabi funkcije `titracija`, pripravite Pandas DataFrame. Prvi stolpec naj vsebuje količino dodane baze, drugi pH raztopine, tretji pa koncentracijo H_3O^+ ionov.

10.5 Naloge

Za reševanje nalog potrebujete knjižnici `matplotlib` in `numpy`.

10.5.1 Onesnaženost

Funkcije shranite v datoteko `onesnazenost.py`.

Branje podatkov

Podatki o onesnaženosti z delci PM10 za izbrane kraje so shranjeni v datotekah, kjer stolpci predstavljajo kraje, vrstice pa dneve v mesecu.

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke v obliki niza (`str`). Funkcija naj vrača terko z dvodimenzionalno tabelo (angl. *array*), ki jo lahko iz datoteke razčlenite z uporabo funkcije `np.genfromtext`. Če se odločite za uporabo te funkcije, v dokumentaciji poiščite, katere parametre morate prilagoditi.

Primer pravilne izvedbe funkcije:

```
>>> preberi_podatke('podatki/januar.csv')
(array([[152,  88, 134,  35],
       [ 61,  68,  94,  31],
       [ 40,  81,  83,  61],
       [ 57,  69,  86,  51],
       [ 39,  19,  39,  11],
       [ 10,  16,  13,   5],
       [ 19,  34,  43,  14],
       [ 34,  50,  48,  32],
       [ 71,  71,  67,  51],
       [ 37,  47,  47,  31],
       [ 67,  67,  82,  43],
       [ 82,  77,  34,  33],
       [ 11,  15,  19,   5],
       [ 33,  14,  60,  17],
       [ 47,  18,  67,  17],
       [ 29,  23,  36,  15],
       [ 11,  24,  16,  11],
       [ 20,  34,  31,  28],
       [ 42,  56,  51,  46],
       [ 65,  72, 101,  58],
       [ 88, 101, 105,  55],
       [ 79, 138,  93,  74],
       [ 91, 170,  97,  87],
       [104, 106,  97, 111],
```

```
       [ 62,  74,  64,  74],
       [ 31,  51,  54,  38],
       [ 53,  62,  75,  36],
       [ 93,  94, 100,  91],
       [124, 100, 112, 120],
       [108,  99, 101, 113],
       [132, 146,  93,  90]]),
['Ljubljana', 'Maribor', 'Zagorje', 'Velenje'])
```

Povprečje

Napišite tri funkcije, ki kot argument sprejmejo podatke, kot jih vrača funkcija `preberi_podatke`. Funkcije so:

- `povprecje`, ki vrača skupno povprečje,
- `povprecje_po_dnevih`, ki vrača povprečje po dnevih in
- `povprecje_po_krajih`, ki vrača povprečje po krajih.

Pri pisanju funkcij uporabite funkijo `np.average`.

Primer pravilne izvedbe funkcije:

```
>>> podatki, _ = preberi_podatke('podatki/januar.csv')
>>> povprecje(podatki)
61.306451612903224
>>> povprecje_po_dnevih(podatki)
array([102.25,  63.5 ,  66.25,  65.75,  27.  ,  11.  ,  27.5 ,  41.  ,
       65.  ,  40.5 ,  64.75,  56.5 ,  12.5 ,  31.  ,  37.25,  25.75,
       15.5 ,  28.25,  48.75,  74.  ,  87.25,  96.  , 111.25, 104.5 ,
       68.5 ,  43.5 ,  56.5 ,  94.5 , 114.  , 105.25, 115.25])
>>> povprecje_po_krajih(podatki)
array([ 61.03225806,  67.22580645,  69.09677419,  47.87096774])
```

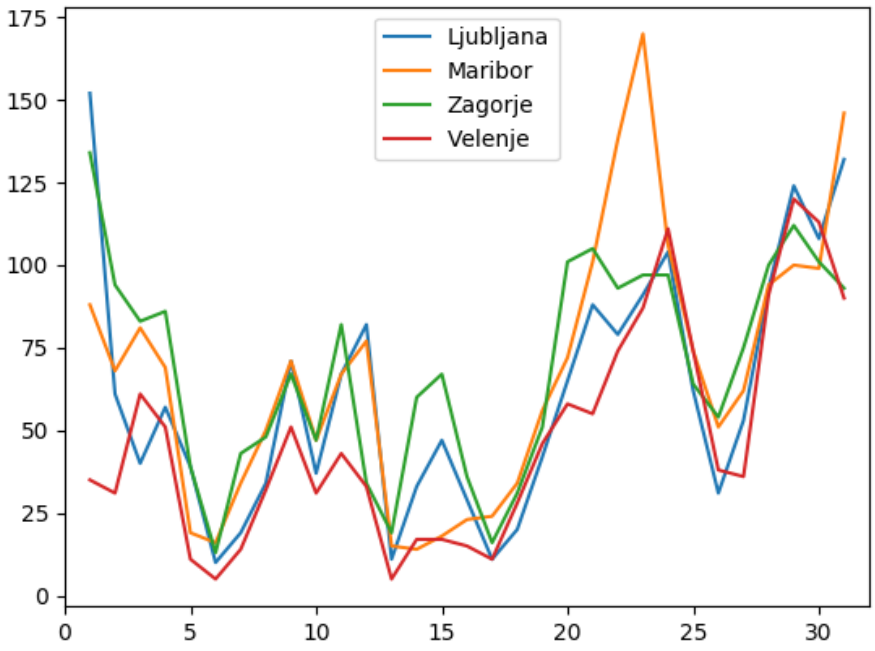
Prikaz onesnaženosti

Napišite funkcijo `shrani_graf`, ki sprejme dva argumenta - podatke, kot jih vrača funkcija `preberi_podatke` in seznam mest. Funkcija naj izriše graf onesnaženosti za podana mesta s seznama in ga shrani v datoteko `onesnazenost.png`. Za shranjevanja grafa uporabite funkcijo `plt.savefig`. Funkcija naj deluje tudi za drugačne sezname mest.

Primer izvedbe funkcije:

```
>>> podatki, kraji = preberi_podatke('podatki/januar.csv')
>>> shrani_graf(podatki, kraji)
```

Primer grafa:



10.5.2 Modeliranje poteka kemijskih reakcij

Funkcije shranite v datoteko `reakcije.py`.

Preprost kemijski sistem

Imamo preprost kemijski sistem, ponazorjen z naslednjo shemo:

$$\xrightarrow{k_1} S_1 \xrightarrow{k_{-1}}$$

S_1 je koncentracija spojine, k_1 stopnja nastajanja te spojine, k_{-1} pa stopnja razpada spojine. k_1 in k_{-1} sta podani v nMh^{-1}.

Sistem lahko opišemo z naslednjo diferencialno enačbo:

$$\frac{dS_1}{dt} = k_1 - k_{-1}S_1(t)$$

Začetna koncentracija spojine S_1 naj bo 0 nM. Napišite funkcijo `kemijski_sistem`, ki kot argumenta sprejme stopnji nastajanja in razpada (tj. k_1 in k_{-1}), vrne pa koncentracijo S_1, pri kateri sistem doseže stabilno stanje - kjer se koncentracija spojine se med dvema časovnima točkama razlikuje za manj kot 0.00001. Rezultat naj bo zaokrožen na 3 decimalke. Časovni korak, dt, naj bo 1.

Primera pravilne izvedbe funkcije:

```
>>> kemijski_sistem(0.01, 0.01)
0.999
>>> kemijski_sistem(0.02, 0.01)
1.999
```

Namig: Z zanko *while* trenutni koncentraciji S_1 prištevajte vrednost izraza $k_1 - k_{-1} \cdot S_1$, vse dokler slednji ne doseže 0.0001.

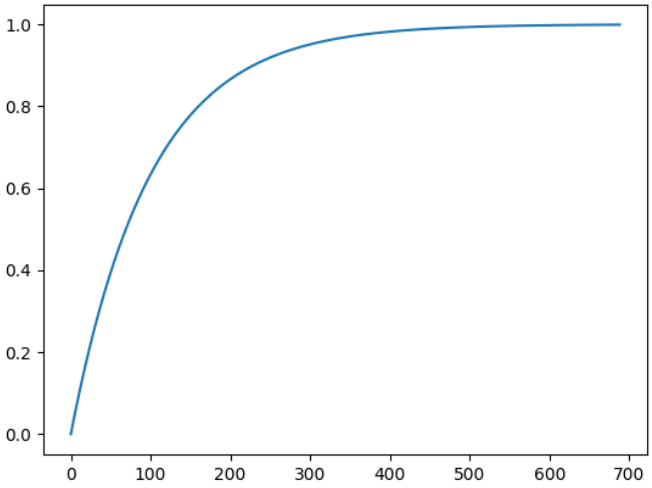
Izris

Napišite funkcijo `izris(k1, k_1)`, ki izriše spreminjanje koncentracije S_1, kot ga izračunava funkcija `kemijski_sistem`.

Primer izvedbe funkcije:

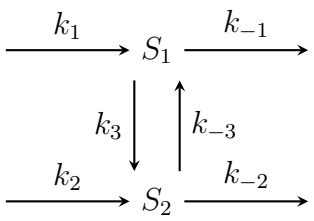
```
>>> izris(0.01, 0.01)
```

Primer grafa:



Razširjen kemijski sistem

V sistem dodamo spojino S_2, ki se vključuje v sistem na naslednji način:



k_2 je stopnja nastajanja, k_{-2} pa stopnja razpada S_2. k_3 in k_{-3} sta stopnji pretvorbe spojine S_1 v S_2 in obratno. Vse konstante so podane v enotah nMh^{-1} Takšen sistem opišemo z dvema diferencialnimi enačbama:

$$\frac{dS_1}{dt} = k_1 + k_3 \cdot S_2(t) - (k_{-1} + k_{-3}) \cdot S_1(t)$$

$$\frac{dS_2}{dt} = k_2 + k_{-3} \cdot S_1(t) - (k_{-2} + k_3) \cdot S_2(t)$$

Začetna koncentracije spojin S_1 in S_2 naj bo 0 nM. Napišite funkcijo `kemijski_sistem_dva`, ki kot argumente sprejme konstante k_1, k_{-1}, k_2, k_{-2}, k_3, k_{-3} in vrne terko koncentracij, $(S_{1,ss}, S_{2,ss})$, pri katerih sistem doseže ravnotežno stanje. Kot ravnotežno stanje štejte koncentraciji, ki se v zaporednih časovnih točkah spremenita za manj kot $10^{-5}\,nM$.

Ravnotežni koncentraciji naj bosta zaokroženi na 3 decimalke.

Primer pravilne izvedbe funkcije:

```
>>> kemijski_sistem_dva(0.01, 0.01, 0.2, 0.01, 0.02, 0.004)
(12.646, 8.353)
```

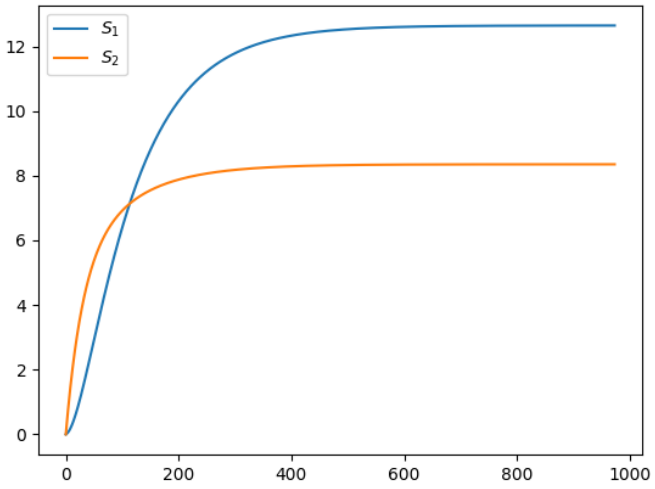
Izris II

Napišite funkcijo `izris_dva`, ki kot argumente sprejme konstante k_1, k_{-1}, k_2, k_{-2}, k_3, k_{-3} in izriše spreminjanje koncentracij S_1 in S_2, kot ju izračunava funkcija `kemijski_sistem_dva`.

Primer izvedbe funkcije:

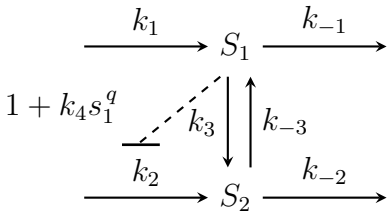
```
>>> izris_dva(0.01, 0.01, 0.2, 0.01, 0.02, 0.004)
```

Primer grafa:



Še bolj razširjen kemijski sistem

Nastajanje spojine S_2 naj negativno regulira spojina S_1:



Sistem je sedaj opisan s parom diferencialnih enačb:

$$\frac{dS_1}{dt} = k_1 + k_3 \cdot S_2(t) - (k_{-1} + k_{-3}) \cdot S_1(t)$$

$$\frac{dS_2}{dt} = \frac{k_2}{1 + k_4 \cdot S_1^q(t)} + k_{-3} \cdot S_1(t) - (k_{-2} + k_3) \cdot S_2(t)$$

Napišite funkcijo `kemijski_sistem_tri`, ki kot argumente sprejme konstante k_1, k_{-1}, k_2, k_{-2}, k_3, k_{-3}, k_4 in q in vrne terko koncentracij, $(S_{1,ss}, S_{2,ss})$, pri katerih sistem doseže ravnotežno stanje. Kot ravnotežno stanje ponovno upoštevajte koncentraciji, ki se v zaporednih časovnih točkah spremenita za manj kot $10^{-5}\, nM$.

Rezultata zaokrožite na 3 decimalke.

Primer pravilne izvedbe funkcije:

```
>>> kemijski_sistem_tri(0.01, 0.01, 0.2, 0.01, 0.02, 0.004, 0.05, 2)
(5.531, 3.372)
```

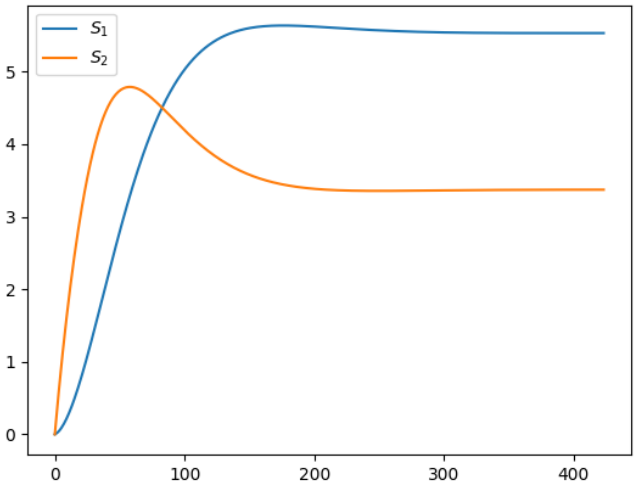
Izris III

Napišite funkcijo `izris_tri`, ki kot argumente sprejme konstante k_1, k_{-1}, k_2, k_{-2}, k_3, k_{-3}, k_4 in q, in izriše spreminjanje koncentracij S_1 in S_2, kot ju izračunava funkcija `kemijski_sistem_tri`.

Primer klica funkcije:

```
>>> izris_tri(0.01, 0.01, 0.2, 0.01, 0.02, 0.004, 0.05, 2)
```

Primer grafa:



10.6 Rešitve

10.5.1 Onesnaženost

Branje podatkov

```
import numpy as np
def preberi_podatke(datoteka):
    with open(datoteka, encoding='utf-8') as f:
        kraji = f.readline().strip().split(';')
    return np.genfromtxt(datoteka, dtype=int, delimiter=';', skip_header=True),
    ↪ kraji
```

Povprečje

```
import numpy as np
def povprecje(podatki):
    return np.average(podatki)

def povprecje_po_dnevih(podatki):
    return np.average(podatki, axis = 1)

def povprecje_po_krajih(podatki):
    return np.average(podatki, axis = 0)
```

Prikaz onesnaženosti

Izpis na zaslon:

```
import numpy as np
import matplotlib.pyplot as plt

def shrani_graf(podatki, napisi):
    plt.plot(range(1, len(podatki)+1), podatki)
    plt.xlim(0, len(podatki)+1)
    plt.legend(napisi, loc = 'upper center').set_draggable(True)
    plt.show()
```

Shranjevanje v datoteko:

```
import numpy as np
import matplotlib.pyplot as plt

def shrani_graf(podatki, napisi):
    plt.plot(range(1, len(podatki)+1), podatki)
    plt.xlim(0, len(podatki)+1)
    plt.legend(napisi, loc = 'upper center').set_draggable(True)
    plt.savefig('onesnazenost.png')
```

10.5.2 Modeliranje poteka kemijskih reakcij

Preprost kemijski sistem

```
def kemijski_sistem(k1, k_1):
    s1 = 0
    d1 = k1 - k_1 * s1
    while abs(d1) > 0.00001:
        s1 = s1 + d1
        d1 = k1 - k_1 * s1
    return round(s1, 3)
```

Izris

```
import matplotlib.pyplot as plt
def izris(k1, k_1):
    s1 = 0
    d1 = k1 - k_1 * s1
    S = [s1]
    while abs(d1) > 0.00001:
        s1 = s1 + d1
        S.append(s1)
        d1 = k1 - k_1 * s1
    plt.plot(S)
    plt.show()
```

Razširjen kemijski sistem

```
def kemijski_sistem_dva(k1, k_1, k2, k_2, k3, k_3):
    s1 = 0
    s2 = 0
    d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
    d2 = k2 + k_3 * s1 - (k_2 + k3) * s2
    while abs(d1) > 0.00001 or abs(d2) > 0.00001:
        s1 += d1
        s2 += d2
        d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
        d2 = k2 + k_3 * s1 - (k_2 + k3) * s2
    return round(s1, 3), round(s2, 3)
```

Izris II

```python
import matplotlib.pyplot as plt
def izris_dva(k1, k_1, k2, k_2, k3, k_3):
    s1 = 0
    s2 = 0
    S1 = [s1]
    S2 = [s2]
    d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
    d2 = k2 + k_3 * s1 - (k_2 + k3) * s2
    while abs(d1) > 0.00001 or abs(d2) > 0.00001:
        s1 += d1
        s2 += d2
        S1.append(s1)
        S2.append(s2)
        d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
        d2 = k2 + k_3 * s1 - (k_2 + k3) * s2
    plt.plot(S1, label = '$S_1$')
    plt.plot(S2, label = '$S_2$')
    plt.legend()
    plt.show()
```

Še bolj razširjen kemijski sistem

```python
def kemijski_sistem_tri(k1, k_1, k2, k_2, k3, k_3, k4, q):
    s1 = 0
    s2 = 0
    d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
    d2 = k2 / (1 + k4 * s1 ** q) + k_3 * s1 - (k_2 + k3) * s2
    while abs(d1) > 0.00001 or abs(d2) > 0.00001:
        s1 += d1
        s2 += d2
        d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
        d2 = k2 / (1 + k4 * s1 ** q) + k_3 * s1 - (k_2 + k3) * s2
    return round(s1, 3), round(s2, 3)
```

Izris III

```
import matplotlib.pyplot as plt
def izris_tri(k1, k_1, k2, k_2, k3, k_3, k4, q):
    s1 = 0
    s2 = 0
    S1 = [s1]
    S2 = [s2]
    d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
    d2 = k2 / (1 + k4 * s1 ** q) + k_3 * s1 - (k_2 + k3) * s2
    while abs(d1) > 0.00001 or abs(d2) > 0.00001:
        s1 += d1
        s2 += d2
        S1.append(s1)
        S2.append(s2)
        d1 = k1 + k3 * s2 - (k_1 + k_3) * s1
        d2 = k2 / (1 + k4 * s1 ** q) + k_3 * s1 - (k_2 + k3) * s2
    plt.plot(S1, label = '$S_1$')
    plt.plot(S2, label = '$S_2$')
    plt.legend()
    plt.show()
```

Razhroščevanje

Prva pomoč pri nedelujočem programu

Če program ne deluje, preverite, ali ste pravilno uporabili:

- ločila (dvopičja, pike, vejice, oklepaje in zaklepaje),
- ustrezen zamik pred vrsticami (skozi cel program uporabljajte bodisi tabulator ali 4 presledke),
- ustrezne operacije glede na uporabljene podatkovne tipe in
- poimenovanje spremenljivk (izbrana imena niso na rezerviranih seznamih in pri večkratni ponovitvi imena spremenljivke ne vsebujejo zatipkov).

Pythonova sporočila o napakah

Python ob nedelovanju programa javi napako. Najpogostejša sporočila o napakah (angl. *Error Messages*) so:

- SyntaxError: koda vsebuje sintaktične napake (npr. manjkajoči zaklepaji, napačni zamiki, neustrezna uporaba (dvojnega) enačaja, uporaba nove sintakse na starejših verzijah Pythona),
- IndetationError: napačno urejeni/uporabljeni zamiki,
- NameError: spremenljivka z danim imenom ni definirana,
- ValueError: argument funkcije ni ustreznega podatkovnega tipa (npr. `stevilo = int("dve")`,
- UnboundLocalError: spremenljivka je bila uporabljena preden ji je bila prirejena vrednost,
- TypeError: uporabljena operacija ne deluje na podanem podatkovnem tipu (npr. `"Hello"/3`),
- ZeroDivisionError: prišlo je do deljenja z 0,
- IndexError: indeks je izven obsega seznama/terke/niza,
- KeyError: dani ključ v slovarju ne obstaja,
- AttributeError: izbran podatkovni tip ne podpira uporabljenega atributa.

Poseben tip napake je EOF Error, kjer "EOF" pomeni "End of File". Takšni programi se predčasno zaključijo, vrstica z javljeno napako pa ni nujno vzrok težave - ta se lahko pojavi prej ali kasneje. Najpogostejši razlogi za to napako so manjkajoči oklepaji/zaklepaji, nepričakovana vsebina v `input`, neskladje med pričakovanimi in dejanskimi vhodi pri testiranju ter slabo definirani pogoji izstopa iz zank ali neskončna zanka.

Razhroščevanje s testi

Ko ekstrahirate mapo s testi, ki ste jo prenesli iz spletne učilnice, se v njej nahaja podmapa `public`, znotraj katere so datoteke `config.json`, `navodila.txt` in `test.py` ter tri podmape - `input`, `output` in `results`.

V mapi `input` se nahajajo tekstovne datoteke s končnico `.in`, v katerih so zbrani vhodi, s katerimi testiramo vašo funkcijo. Izhodi, ki jih želimo dobiti pri teh vhodnih podatkih, se nahajajo v mapi `output` v tekstovnih datotekah s končnico `.out`.

Pripadajoči vhodi in izhodi so poimenovani enako - vhodi v datoteki `test_1.in` vrnejo izhod v datoteki `test_1.out`. Datoteke s končnicami `.in` ali `.out` lahko odprete z IDLE ali s katerimkoli programom za urejanje golega besedila.

Razhroščevanje s funkcijo `print()`

Kadar Python ne javi sporočila o napaki, kljub temu pa se naša koda ne obnaša, kakor smo si zamislili, je najboljši način preverjanja pravilnosti logike z uporabo funkcije `print()`. Takšno razhroščevanje je še posebej uporabno, kadar nismo prepričani, v katerem koraku pride do napake in v primerih, kadar uporabljamo več zank v kombinaciji z različnimi pogoji.

Poglejmo si uporabo tega na primeru programa, ki naj sešteva vsa liha števila v podanem seznamu. Spisana koda je sledeča:

```
1 seznam = [18, 92, 94, 77, 33, 7, 54, 37]
2 vsota = 0
3
4 for i in range(len(seznam)):
5     if i%2 == 0:
6         vsota += i
7
8 print("Vsota je:", vsota)
```

Ko zaženemo program in vpišemo seznam , nam vrne rezultat `12`, kar ni ok. Ker je napako lažje odkriti s pomočjo funkcije `print()`, vstavimo funkcijo in pripadajoče spremenljivke na mesta v kodi, kjer želimo slediti poteku programa:

```
1 seznam = [18, 92, 94, 77, 33, 7, 54, 37]
2 vsota = 0
3
4 for i in range(len(seznam)):
5     print(f'To je i: {i}')
6     if i%2 == 0:
7         vsota += i
8         print(f'Vsota: {vsota}')
```

```
9
10 print("Vsota je", vsota)
```

Ko program poženemo, Python izpiše:

```
To je i: 0
Vsota: 0
To je i: 1
To je i: 2
Vsota: 2
To je i: 3
To je i: 4
Vsota: 6
To je i: 5
To je i: 6
Vsota: 12
To je i: 7
Vsota je 12
```

Na ta način lahko lepo sledimo poteku kode. Najprej opazimo, da naš `i` ni element seznama, temveč indeks. Nadalje vidimo, da do seštevanja pride le v primeru sodih števil. To kaže, da moramo našo kodo popraviti na dveh mestih:

```
1 seznam = [18, 92, 94, 77, 33, 7, 54, 37]
2 vsota = 0
3
4 for i in seznam:
5     print(f'To je i: {i}')
6     if i%2 != 0:
7         vsota += i
8         print(f'Vsota: {vsota}')
9
10 print("Vsota je", vsota)
```

Sedaj program vrača:

```
To je i: 18
To je i: 92
To je i: 94
To je i: 77
Vsota: 77
To je i: 33
Vsota: 110
To je i: 7
```

```
Vsota: 117
To je i: 54
To je i: 37
Vsota: 154
Vsota je 154
```

kar ustreza želenemu delovanju. Vrstice, namenjene razhroščevanju pred oddajo programa pobrišemo, če pa bomo program še nadgrajevali, jih lahko zakomentiramo in si s tem prihranimo čas v primeru ponovne potrebe po razhroščevanju.

V funkcijah `print()`, namenjenih razhroščevanju, lahko uporabljamo tudi samo spremenljivke, brez spremnega teksta. Kadar imamo veliko spremenljivk ali zank, ki jim želimo slediti, lahko uporaba brez spremnega teksta povzroči nekaj več zmede pri interpretaciji, katera številka sodi h kateri spremenljivki.

Pregled najpogostejših napak

V tem poglavju so zbrane najpogostejše napake, ki jih zasledimo pri pregledovanju rešenih nalog in/ali izpitov. Večina napak je že podrobno razložena v ustreznih poglavjih v skripti, tukaj pa so zbrane za hitrejši pregled in ponovitev.

Pogojni stavek *if*

Spremenljivki `x` neka funkcija priredi vrednost, za katero nas zanima, ali je enaka kateri od naslednjih spremenljivk: `a = 1`, `b = "a"` in `c = (1,"a")`. To lahko preverimo s pogojnim stavkom *if*.

Če pogojni stavek *if* napišemo takole:

```
1 if x == a or b or c:
2     print("Se ujema.")
```

bo Python preveril, ali je `x` enak `a` oziroma ali je v spremenljivki `b` neničelna vrednost oziroma ali je v spremenljivki `c` neničelna vrednost[20]. To pomeni, da se bo koda pod stavkom izvedla vedno, kadar vsaj ena od spremenljivk `b` ali c vsebuje neničelno vrednost. Pravilna sintaksa pogojnega stavka *if*, ki bo preverjala, ali je `x` enak bodisi `b` ali c se glasi:

```
1 if x == a or x == b or x == c:
2     print("Se ujema.")
```

Poglejmo si še tri primere uporabe pogojnih stavkov *if*.

1. primer: kombinacija treh pogojnih stavkov *if*

```
1 if x < 3:
2     print("a")
3 if x < 2:
4     print("b")
5 if x < 1:
6     print("c")
```

Če je `x = 0`, bo program izpisal sledeče:

```
a
b
c
```

Ker so nanizani 3 pogojni stavki *if*, se bodo navedeni pogoji vseh treh preverili neodvisno eden od drugega, saj imamo 3 sklope *if*. Ker so za `x = 0` vsi trije pogoji pravilni, se bodo izvedle vse vrstice pod *if*.

[20] Python si druga dva pogoja tolmači kot `b == True` in `c == True`.

2. primer: kombinacija *if-elif-else*

```
1 if x < 3:
2     print("a")
3 elif x < 2:
4     print("b")
5 else:
6     print("c")
```

V kombinaciji *if-elif-else* se vedno izvede le koda pod stavkom, ki vsebuje prvi pravilen pogoj. Če je `x = 0`, sta sicer pravilna tako pogoj pri *if* kot pri *elif*, vendar se bo na zaslonu izpisal le `a`, saj Python v tej kombinaciji izvede le del kode pod prvim pravilnim pogojem, naslednjih pogojev pa ne gleda več.

3. primer: kombinacija *if-if-else*

```
1 if x < 3:
2     print("a")
3 if x < 2:
4     print("b")
5 else:
6     print("c")
```

Če je `x = 2.5`, se bo na zaslonu izpisalo:

```
a
c
```

Kombinacijo *if-if-else* Python tolmači kot 2 neodvisna sklopa pogojnih stavkov - prvi sklop se začne v 1. vrstici, drugi sklop pa v 3. vrstici. Pogoj pri pogojnem stavku *if* je pravilen, zato se izvede koda pod tem stavkom. V drugem sklopu `x` ne izpolnjuje pogoja, navedenega pod *if*, zato se izvede *else*. Zadnji *else* je vezan na zadnji *if* (torej *if* v zadnjem sklopu, ki se začne v 3. vrstici) in ni povezan z nobenih predhodnim sklopom pogojnih stavkov *if*.

Zanka *for*

Z zanko *for* se želimo sprehoditi po prileţnih elementih seznamov:

`s1 = [0, 1, 2]` in `s2 = ['a', 'b', 'c']`.

Zanka *for* ne omogoča uporabe več kot ene kombinacije "`element in seznam`" naenkrat - povedano drugače: Python ne sprejme sintakse `for i in s1 and j in s2`. Za izvedbo sprehoda po istoležnih elementih seznamov imamo več možnosti (ob pogoju, da velja `len(s1) == len(s2)`):

```
1 for i in range(len(s1)):
```

```
    print(s1[i], s2[i])

for i1, i2 in zip(s1, s2):
    print(i1, i2)
```

Če bi uporabili ugnezdeni zanki:

```
for i in s1:
    for j in s2:
        print(i, j)
```

bi program izpisal vse možne kombinacije elementov obeh seznamov:

```
0 a
0 b
0 c
1 a
1 b
1 c
2 a
2 b
2 c
```

To se zgodi, ker notranja zanka teče hitreje od zunanje - šele, ko se sprehodimo skozi celoten `s2`, se lahko premaknemo iz `s1[0]` na `s1[1]`.

Funkcije

Funkcija `vsota` sprejme dve števili in vrne njuno vsoto:

```
def vsota(x,y):
    return x + y
```

Navidez povsem enako deluje tudi funkcija `vsota_wrong`, ki na zaslon izpiše vsoto podanih argumentov:

```
def vsota_wrong(x,y):
    print(x + y)
```

Razliko med funkcijama `vsota` in `vsota_wrong` opazimo, kadar želimo rezultat funkcije shraniti v novo spremenljivko in to spremenljivko uporabiti v nadaljnji kodi:

```
a = vsota(2,1)
b = vsota_wrong(2,1)

c = a + 1
d = b + 1
```

Vrednost, prirejeno spremenljivki `c` bo Python izračunal brez težav, vrednost, prirejeno `d` pa ne. To se zgodi, ker funkcija `vsota_wrong` rezultat le izpiše na zaslon, a ga ne shrani kot število. Besedica `return` namreč zagotovi, da rezultat funkcije ohrani podatkovni tip, funkcija *print* pa tega ne omogoča.

V primeru, da pisanja funkcije nismo dokončali, dodamo besedo `pass`, saj s tem omogočimo, da se vsi testi izvedejo normalno in se nedokončana funkcija ne preverja:

```
def nedokoncana_funkcija(arg1, arg2):
    b = "abcd"
    if arg1 in b:
        pass

def nedokoncana_funkcija2(arg1, arg2):
    pass

def nedokoncana_funkcija3(arg1, arg2):
    b = "abcd"
    if arg1 in b:
        b += arg1
    pass
```

Terke

Imamo tri sezname enake dolžine, naš cilj pa je pripraviti seznam terk istoležnih elementov teh seznamov:

```
s1 = [0, 1, 2]
s2 = ["a", "b", "c"]
s3 = ["x", "y", "z"]

s4 = list(zip(s1,s2,s3))
```

Seznam `s4` bo enak `[(0,"a","x"),(1,"b","y"),(2,"c","z")]`.

Funkcija `zip` sama po sebi ne ustvari seznama, zato moramo pred njo dodati `list`, če želimo seznam uporabljati v nadaljnji kodi. Če terke istoležnih elementov potrebujemo v zanki `for`, ne potrebujemo funkcije `list`.

```
s4 = []
for j, k, l in zip(s1,s2,s3):
    s4.append(k+l)
```

Seznam neoprostljivih napak

Spodnji seznam vsebuje napake, ki so posledica napačnega razumevanja osnovnih konceptov in/ali sintakse programskega jezika. Če se katera od teh napak pojavi v vaši kodi, ocenjevanje prekinemo in preostanka naloge ne pogledamo do konca.

Napačno:	**Pravilno:**
`if x = a:`	`if x == a:`
`if x == a or b or c:`	`if x == a or x == b or x == c:`
`for el1 in sez1 and el2 in sez2:`	glej poglavje o zanki *for*
`for el1 and el2 in terka:`	glej poglavje o zanki *for*
`def funkcija(argument):` `  print(argument)`	`def funkcija(argument):` `  return argument`[†]
`for i in seznam:` `  a=seznam[i]**2`	glej poglavje o zanki *for*
`for i in range(len(seznam)):` `  a=i**2`	glej poglavje o zanki *for*

[†]Razen v primeru, kjer je v navodilih eksplicitno napisano, naj funkcija ne vrača ničesar in izpiše rezultat.

Naloge za ponavljanje

Naloge

Maraton

Vse funkcije shranite v datoteko `maraton.py`.

V datoteki `rezultati.csv` so zbrani rezultati maratona. Podatki si sledijo v sledečem vrstnem redu: uvrstitev, štartna številka, ime in priimek, letnica rojstva, država, čas, kategorija, uvrstitev znotraj kategorije, spol. Kot ločilo je uporabljena vejica, "`,`".

Primer vsebine datoteke `rezultati.csv`:

```
1;3;Getachew Limenih;1991;ETH;2:08:19;A;1;M
2;5;Wami Mulugeta;1982;ETH;2:08:37;B;1;M
3;10;Rutto Samuel;1994;KEN;2:08:59;A;2;M
4;1;Omari Levi Matebo;1989;KEN;2:09:54;A;3;M
5;6;Sigei Richard;1985;KEN;2:11:20;B;2;M
6;2;Matebor Albert Kiplagat;1980;KEN;2:12:32;C;1;M
7;16;Kipkemboi Paul;1982;KEN;2:15:29;B;3;M
8;13;Grigoryev Oleg;1989;RUS;2:17:10;A;4;M
9;9;Lagat Cosmas;1990;KEN;2:20:30;A;5;M
10;14;Kosovelj Mitja;1984;SLO;2:20:53;B;4;M
11;12;Dhaqabi Dadafo;1989;NOR;2:23:35;A;6;M
12;21;Gizaw Melkam;1990;ETH;2:25:42;A;1;Ž
13;17;Fayissa Guta Hunde;1988;ETH;2:25:43;A;7;M
14;8;Kiprugut Daniel;1978;KEN;2:30:14;C;2;M
15;23;Korir Rebecca;1992;KEN;2:30:36;A;2;Ž
```

Preberi podatke

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke in vrne seznam nizov, pri čemer je vsak niz posamezna vrstica datoteke.

Primer pravilne izvedbe funkcije z delnim izpisom rezultata:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> rezultati[:6]
['1;3;Getachew Limenih;1991;ETH;2:08:19;A;1;M\n', '2;5;Wami
↪  Mulugeta;1982;ETH;2:08:37;B;1;M\n', '3;10;Rutto
↪  Samuel;1994;KEN;2:08:59;A;2;M\n', '4;1;Omari Levi
↪  Matebo;1989;KEN;2:09:54;A;3;M\n', '5;6;Sigei
↪  Richard;1985;KEN;2:11:20;B;2;M\n', '6;2;Matebor Albert
↪  Kiplagat;1980;KEN;2:12:32;C;1;M\n']
```

Obdelava podatkov

Napišite funkcijo `obdelava_podatkov`, ki kot argument sprejme seznam nizov (tj. seznam, kot ga vrača funkcija `preberi_podatke`). Funkcija vrne seznam seznamov oblike: `[uvrstitev, startna_stevilka, ime_priimek, leto_rojstva, drzava, cas, kategorija, uvrstitev, spol]`. Pri tem čas pretvorite v sekunde, prav tako pa zagotovite, da bodo vsa števila pretvorjena v ustrezen podatkovni tip.

Primer pravilne izvedbe funkcije z delnim izpisom rezultata:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> rezultati[:7]
[[1, 3, 'Getachew Limenih', 1991, 'ETH', 7699, 'A', 1, 'M'], [2, 5, 'Wami
↪  Mulugeta', 1982, 'ETH', 7717, 'B', 1, 'M'], [3, 10, 'Rutto Samuel', 1994,
↪  'KEN', 7739, 'A', 2, 'M'], [4, 1, 'Omari Levi Matebo', 1989, 'KEN', 7794,
↪  'A', 3, 'M'], [5, 6, 'Sigei Richard', 1985, 'KEN', 7880, 'B', 2, 'M'], [6,
↪  2, 'Matebor Albert Kiplagat', 1980, 'KEN', 7952, 'C', 1, 'M'], [7, 16,
↪  'Kipkemboi Paul', 1982, 'KEN', 8129, 'B', 3, 'M']]
```

Kategorije

Napišite funkcijo `tekaci_v_kategoriji`, ki kot argumenta sprejme seznam seznamov, kot ga vrača funkcija `obdelava_podatkov` in kategorijo. Argument kategorija je niz (`str`), sestavljen iz dveh podatkov: spola in kategorije, ločenih s pomišljanjem: `"spol-kategorija"`.

Najprej lahko napišete funkcijo `po_kategorijah`, s katero seznam seznamov pretvorite v slovar, kjer so ključi kategorije. Opisana pomožna funkcija vam lahko pride prav še pri kakšni drugi nalogi.

Funkcija `tekaci_v_kategoriji` naj vrača število tekačev v želeni kategoriji.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> kategorija = 'M-A'
>>> tekaci_v_kategoriji(rezultati, kategorija)
186
```

Najmanj zastopana kategorija

Napišite funkcijo `najmanj_na_kategorijo`, ki kot argument sprejme seznam, kot ga vrača funkcija `obdelava_podatkov`. Funkcija naj vrne kategorijo v obliki `spol-kategorija`, v kateri je teklo najmanj tekačev.

Pomagate si lahko s pomožno funkcijo `po_kategorijah`, ki ste jo uporabili pri pisanju funkcije `tekaci_v_kategoriji`.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> najmanj_na_kategorijo(rezultati)
'Ž-I'
```

Zmagovalec kategorije Napišite funkcijo `zmagovalec_kategorije`, ki kot argumenta sprejme seznam, kot ga vrača funkcija `obdelava_podatkov` in kategorijo v obliki niza: `"spol-kategorija` (`str`). Funkcija naj vrača zmagovalca kategorije.

Če ste napisali pomožno funkcijo `po_kategorijah`, jo lahko uporabite, ni pa nujno.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> kategorija = 'M-A'
>>> zmagovalec = zmagovalec_kategorije(rezultati, kategorija)
>>> zmagovalec
'Getachew Limenih'
```

Zmagovalka

Napišite funkcijo `zmagovalka`, ki kot argument sprejme seznam, kot ga vrača funkcija `obdelava_podatkov` in vrne ime ženske zmagovalke.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> z = zmagovalka(rezultati)
>>> z
'Gizaw Melkam'
```

Letnica rojstva

Napišite funkcijo `tekaci_po_letu`, ki kot argumenta sprejme seznam kot ga vrača funkcija `obdelava_rezultatov` in letnico v obliki celega števila (`int`). Funkcija naj vrača množico tekačev s podano letnico rojstva.

Za pomoč pri reševanju si lahko najprej napišete funkcijo `po_letnicah`, ki seznam pretvori v slovar, kjer so ključi letnice rojstva.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> letnica = 1987
>>> tekaci = tekaci_po_letu(rezultati, letnica)
>>> tekaci
{'Pirh Barbara', 'Sleigh James', 'Heidebrecht Hans-jürgen', 'Levstek Luka',
↪  'Starič Rok', 'Mahmutović Elvis', 'Terčelj Matic', 'Jelovečki Marco',
↪  'Kopač Nataša', 'Nikolic Milos', 'Dimitrijevic Milos', 'Collett William',
↪  'Mikuž Domen', 'Arčon Matic', 'Zimmermann Patrick', 'Tuđa Andrea',
↪  'Tomasevic Natasa', 'Abrahamsberg Matej', 'Marton Minea', 'Jocic Bojan',
↪  'Spasojevic Ana', 'Poznič Jerca', 'Pock Kralj Yvonne', 'Zupan Primož',
↪  'Zrimšek Matej', 'Ožek Matjaž', 'Šipec Urša', 'Lucic Branko', 'Veselič
↪  Ivo', 'Mitrovic Milan', 'Pivk Matej', 'Šantej Marko', 'Šenk Primož',
↪  'Eberhart Nina', 'Žitnik Slavko', 'Kostanjevec Marko'}
>>> len(tekaci)
36
```

Najstarejši tekači

Napišite funkcijo `najstarejsi`, ki kot argument sprejme seznam, kot ga vrača funkcija `obdelava_podatkov` in vrne množico imen najstarejših tekačev na maratonu.

Če ste napisali pomožno funkcijo `po_letnicah`, jo lahko uporabite.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> najstarejsi_tekaci = najstarejsi(rezultati)
>>> najstarejsi_tekaci
{'Nadenić Josip'}
```

Čas

Napišite funkcijo `cas`, ki kot argumenta sprejme seznam, kot ga vrača funkcija `obdelava_podatkov` in ime tekača. Funkcija naj vrača čas, ki ga je izbrani tekač potreboval za zaključek maratona.

Najprej lahko napišete funkcijo `po_osebah`, ki seznam pretvori v slovar, kjer so ključi imena tekačev.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> oseba = 'Levstek Luka'
>>> time = cas(rezultati, oseba)
>>> time
'3:30:35'
```

Razlika

Napišite funkcijo `razlika`, ki sprejme 3 argumente: seznam, kot ga vrača funkcija `obdelava_podatkov`, ter dve imeni tekačev. Funkcija naj vrača absolutno razliko časov med izbranima tekačema.

Če ste napisali pomožno funkcijo `po_osebah`, jo lahko uporabite.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> oseba1 = 'Poznič Jerca'
>>> oseba2 = 'Levstek Luka'
>>> razlika(rezultati, oseba1, oseba2)
'0:48:35'
```

Soimenjaki

Napišite funkcijo `soimenjaki`, ki kot argument sprejme seznam, kot ga vrača funkcija `obdelava_podatkov`, vrne pa število imen, ki se pojavijo več kot enkrat.

Lahko si pomagate s pomožno funkcijo `po_osebah`.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> soimenjaki(rezultati)
3
```

Najboljši

Napišite funkcijo `najboljsi_po_letnicah`, ki kot argument sprejme seznam, kot ga vrača funkcija `obdelava_podatkov`, vrne pa množico terk, pri čemer posamezna terka vsebuje letnico rojstva in ime najhitrejšega tekača s to letnico rojstva, tj. `(letnica, ime)`.

Pomagate si lahko z vsemi pomožnimi funkcijami.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> najboljsi = najboljsi_po_letnicah(rezultati)
>>> najboljsi
{(1967, 'Benedetti Paolo'), (1971, 'Kustec Gregor'), (1951, 'Bechini Valter'),
↪  (1975, 'Vezilj Mario'), (1997, 'Kadivnik Jernej'), (1952, 'Ruparčič Ivan'),
↪  (1963, 'Božič Ivan'), (1953, 'Korhonen Veijo'), (1942, 'Jean-pierre
↪  Briard'), (1960, 'Savić Miro'), (1943, 'Savič Vladimir'), (1973,
↪  'Stanojevic Igor'), (1977, 'Miskeljin Ivan'), (1958, 'Kalan Tomaž'), (1987,
↪  'Zrimšek Matej'), (1966, 'Mccarter Dominic'), (1989, 'Omari Levi Matebo'),
↪  (1998, 'Kotnik Mojca'), (1939, 'Ambrož Janez'), (1991, 'Getachew Limenih'),
↪  (1962, 'Nottolini Claudio'), (1996, 'Svenšek Robin'), (1965, 'Majcen
↪  Marjan'), (1988, 'Fayissa Guta Hunde'), (1968, 'Meden Sašo'), (1954,
↪  'Lončar Franjo'), (1969, 'Bregar Metod'), (1995, 'Gradišar Žiga'), (1976,
↪  'Volkar Franci'), (1980, 'Matebor Albert Kiplagat'), (1972, 'Stacul
↪  Alessio'), (1956, 'Košnjek Tone'), (1985, 'Sigei Richard'), (1959, 'Jerič
↪  Miha'), (1990, 'Lagat Cosmas'), (1979, 'Balciunaite Zivile'), (1955,
↪  'Foršček Bojan'), (1978, 'Kiprugut Daniel'), (1994, 'Rutto Samuel'), (1961,
↪  'Švab Boštjan'), (1945, 'Posel Bojan'), (1948, 'Goleš Željko'), (1933,
↪  'Nadenić Josip'), (1970, 'Samec Aleš'), (1947, 'Marzona Adriano'), (1938,
↪  'Jean Berland'), (1993, 'Tuei Naomi'), (1964, 'Hasanović Izudin'), (1982,
↪  'Wami Mulugeta'), (1986, 'Tratnik Marko'), (1974, 'Štendler Martin'),
↪  (1981, 'Mulej Janez'), (1992, 'Korir Rebecca'), (1950, 'Koskinen Markku'),
↪  (1946, 'Rossi Adriano'), (1984, 'Kosovelj Mitja'), (1949, 'Pozar Janez'),
↪  (1957, 'Hickman Peter'), (1983, 'Lagoutte Julien')}
```

Histogram

Napišite funkcijo `histogram`, ki kot argument sprejme seznam, kot ga vrača funkcija `obdelava_podatkov`.

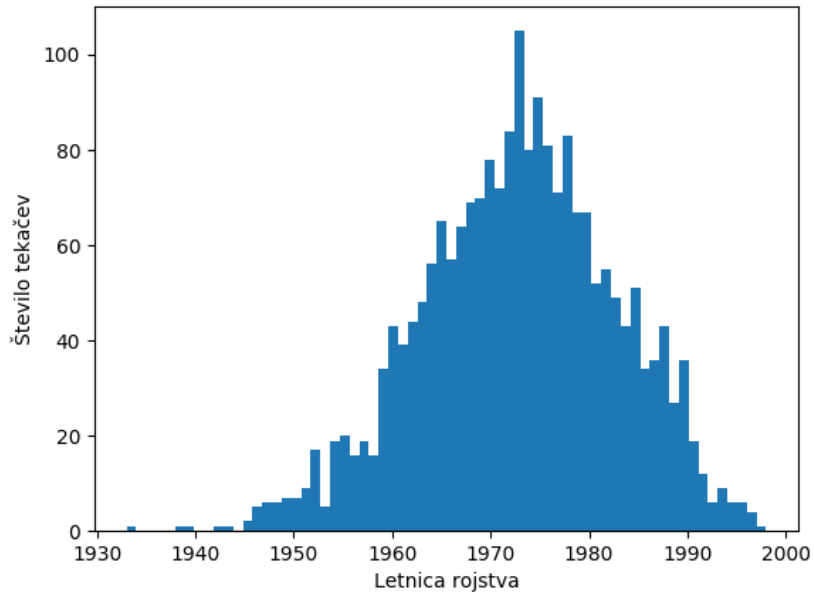
Funkcija naj izriše histogram števila tekačev v odvisnosti od letnice rojstev. Uporabite lahko pomožne funkcije, histogram pa izrišete s funkcijo `hist` iz modula `matplotlib`.

Za to nalogo testi niso na voljo.

Primer pravilne izvedbe funkcije:

```
>>> rezultati = preberi_podatke('rezultati.csv')
>>> obdelava_podatkov(rezultati)
>>> histogram(rezultati)
```

Primer izrisanega histograma:



Rešitve nalog

Maraton

Preberi podatke

```
def preberi_podatke(datoteka):
    f = open(datoteka, 'r', encoding = 'UTF-8')
    seznam = f.readlines()
    f.close()
    return seznam
```

ali:

```
def preberi_podatke(datoteka):
    seznam = []
    for vrstica in open(datoteka, 'r', encoding='UTF-8'):
        seznam.append(vrstica)
    return seznam
```

Obdelava podatkov

```
def obdelava_podatkov(rezultati):
    for i in range(len(rezultati)):
        podatki = rezultati[i].strip().split(";")
        for j in [0, 1, 3, 7]:
            podatki[j] = int(podatki[j])
        cas = podatki[5].split(':')
        podatki[5] = int(cas[2]) + 60*int(cas[1]) + 3600*int(cas[0])
        rezultati[i] = podatki
```

Kategorije

Pomožna funkcija:

```
def po_kategorijah(rezultati):
    slovar = {}
    for podatek in rezultati:
        kategorija = podatek[8] + "-" + podatek[6]
        if kategorija not in slovar:
            slovar[kategorija] = set()
        slovar[kategorija].add(podatek[2])
    return slovar
```

```python
def tekaci_v_kategoriji(rezultati, kategorija):
    slovar = po_kategorijah(rezultati)
    return len(slovar[kategorija])
```

Najmanj zastopana kategorija

```python
def najmanj_na_kategorijo(rezultati):
    slovar = po_kategorijah(rezultati)
    min_k, min_t = "", 2000
    for kategorija, tekaci in slovar.items():
        if len(tekaci) < min_t:
            min_t = len(tekaci)
            min_k = kategorija
    return min_k
```

Zmagovalec kategorije

```python
def zmagovalec_kategorije(rezultati, kategorija):
    for podatek in rezultati:
        kat = podatek[8] + "-" + podatek[6]
        if kat == kategorija:
            return podatek[2]
```

Zmagovalka

```python
def zmagovalka(rezultati):
    for podatek in rezultati:
        if podatek[8] == 'Ž':
            return podatek[2]
```

Letnica rojstva

Pomožna funkcija:

```python
def po_letnicah(rezultati):
    slovar = {}
    for podatek in rezultati:
        letnica = podatek[3]
        if letnica not in slovar:
            slovar[letnica] = set()
        slovar[letnica].add(podatek[2])
    return slovar
```

```python
def tekaci_po_letu(rezultati, letnica):
    slovar = po_letnicah(rezultati)
    if letnica in slovar:
        return slovar[letnica]
    else:
        return 0
```

Najstarejši tekači

```python
def najstarejsi(rezultati):
    slovar = po_letnicah(rezultati)
    return slovar[min(slovar.keys())]
```

Čas

Pomožna funkcija:

```python
def po_osebah(rezultati):
    slovar = {}
    for vrstica in rezultati:
        slovar[vrstica[2]] = vrstica[5]
    return slovar
```

```python
def cas(rezultati, oseba):
    slovar = po_osebah(rezultati)
    time = slovar[oseba]
    ure = time // 3600
    time -= ure * 3600
    minute = time // 60
    time -= minute * 60
    return "{}:{}:{}".format(ure, minute, time)
```

Razlika

```python
def razlika(rezultati, oseba1, oseba2):
    slovar = po_osebah(rezultati)
    time = abs(slovar[oseba1] - slovar[oseba2])
    ure = time // 3600
    time -= ure * 3600
    minute = time // 60
    time -= minute * 60
    return "{}:{}:{}".format(ure, minute, time)
```

Soimenjaki

```python
def soimenjaki(rezultati):
    seznam = []
    for podatek in rezultati:
        seznam.append(podatek[2])
    mnozica = set(seznam)
    return len(seznam) - len(mnozica)
```

Najboljši

```python
def najboljsi_po_letnicah(rezultati):
    slovar_letnice = po_letnicah(rezultati)
    slovar_osebe = po_osebah(rezultati)
    najboljsi = set()
    for letnica, tekaci in slovar_letnice.items():
        naj_cas = 100000
        naj_tekac = ''
        for tekac in tekaci:
            if slovar_osebe[tekac] < naj_cas:
                naj_cas = slovar_osebe[tekac]
                naj_tekac = tekac
        najboljsi.add((letnica, naj_tekac))
    return najboljsi
```

Histogram

```python
import matplotlib.pyplot as plt
def histogram(rezultati):
    letnice = []
    for podatek in rezultati:
        letnice.append(podatek[3])
    plt.hist(letnice, max(letnice) - min(letnice) + 1)
    plt.xlabel('Letnica rojstva')
    plt.ylabel('Število tekačev')
    plt.show()
```